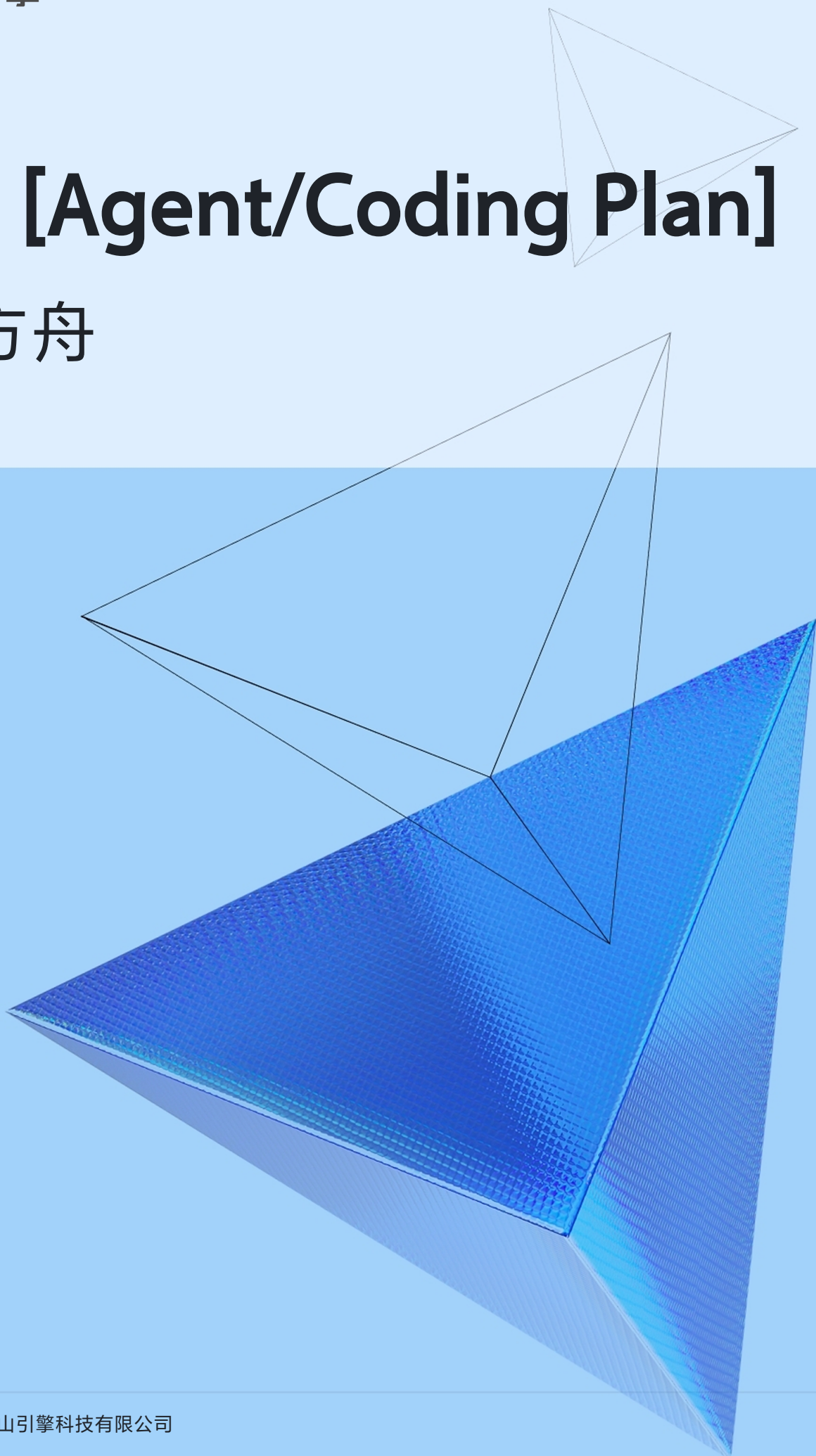


订阅 [Agent/Coding Plan]

火山方舟



法律声明

本《火山方舟》的所有内容，包括但不限于文字、商标、架构、图示、图片、页面布局等，其知识产权（著作权、商标权、专利权、商业秘密等）归属于北京火山引擎科技有限公司及其关联公司（火山引擎），非经火山引擎书面同意，任何个人和组织不得复制、使用、修改、转发或以任何违反本《火山方舟》所承载的目的进行传播。

本《火山方舟》陈述内容仅作为产品的通用性介绍和参考性指引，火山引擎保留按“现状”和“当前可用”的形式提供产品和服务的权利。火山引擎不对本《火山方舟》中所载的产品功能、性质、质量、标准等内容进行明示或默示的保证和承诺，最终以您与火山引擎实际签署的协议为准。

如您发现本《火山方舟》有任何错误或歧义，或发现有对本《火山方舟》、产品本身的侵权行为，请与火山引擎取得联系。

联系方式：service@volcengine.com，400-850-0030
(周一至周五 10:00-18:00)

目录

- 法律声明-----1
- 目录-----1
- 1. 套餐概览-----1
- 2. 计费说明-----11
 - 2.1 套餐内 AFP 抵扣规则-----11
 - 2.2 超额后付费规则-----18
 - 2.3 超额后付费管理-----21
- 3. 快速开始-----24
- 4. 接入 AI 工具-----32
 - 4.1 Codex-----32
 - 4.2 Claude Code-----39
 - 4.3 OpenCode-----50
 - 4.4 OpenClaw-----65
 - 4.5 Hermes Agent-----81
 - 4.6 TRAE-----88
 - 4.7 OpenViking-----90
 - 4.8 其他工具-----98
- 5. 接入多模态生成模型-----101
 - 5.1 接入向量化模型-----101
 - 5.2 接入视觉模型-----105
 - 5.3 接入语音模型-----123
- 6. 配置 Harness-----154
 - 6.1 豆包搜索-----154
 - 6.2 专业数据集-----163
 - 6.3 Agent 记忆-----173
 - 6.4 AI Native 应用开发底座-----186
 - 6.5 ArkClaw-----195
- 7. 实践指南-----198
 - 7.1 使用 Agent Plan 开发短视频网站-----198
 - 7.2 使用 Agent Plan 开发学习教育网站-----203
- 8. 常见问题-----208
- 9. 活动及公告-----209
 - 9.1 Agent/Coding Plan 指定模型抵扣系数限时折扣活动-----209
 - 9.2 Agent Plan Small & Medium 套餐 2.5 折普惠活动-----211
 - 9.3 Agent Plan Medium 套餐 seedance 2.0 系列模型调整公告-----213
 - 9.4 功能发布公告-----214

1. 套餐概览

方舟 Agent Plan 是面向个人用户推出的订阅式大模型服务套餐包，新增支持全模态模型及专属 Harness，采用精细化积分计费模式，助力开启 Agent 场景的全新体验。

核心优势

- **全模态模型及 Harness 升级**：提供文本生成、图像生成、视频生成、向量化等能力模型，新增豆包搜索等专属 Harness。
- **量大管饱性价比高**：灵活提供 Small、Medium、Large、Max 四档套餐，兼顾大容量与高性价比，相较于后付费模式，享受更高折扣。
- **兼容主流编程与 Agent 工具**：多生态兼容，无缝接入 OpenClaw、Hermes Agent、Claude Code、OpenCode 等主流编程及 Agent 工具。
- **包月省心管理**：包月形式订阅，减少预算管理负担。
- **用量清晰可查**：采用 Agent 燃料值（Agent Fuel Point，后简称 AFP）作为统一用量单位，额度用量清晰可查。

说明

- 已购买套餐用户？参考[快速开始](#) 开启 AI 工具使用之旅。
- 2026年6月10日18:00:00至2026年7月15日00:00:00，Agent/Coding Plan 个人版用户使用 deepseek-v4-pro、kimi-k2.6、kimi-k2.7-code、glm-5.2 模型时，模型抵扣系数在现有抵扣系数基础上享最低 2.5 折优惠。详情请参见[Agent/Coding Plan 指定模型抵扣系数限时折扣活动](#)。

方舟 Coding 开发者交流群
飞书扫码进群，交流 AI Coding 实践与心得



方舟 Agent 开发者交流群
飞书扫码进群，交流 Agent 开发与应用(如 OpenClaw)



套餐权益

注意

Small 套餐仅供轻量化体验，不支持视频生成，建议选购 Medium 及以上套餐。

支持模型及 Harness

注意

glm-5.2 、 deepseek-v4-flash 、 deepseek-v4-pro 支持 1M 上下文窗口用于包含大型代码库的长会话。在工具中开启 1M 上下文的方案可参见[Claude Code](#)、[OpenCode](#)、[OpenClaw](#)。

不同套餐支持的模型及 Harness 不同，详细信息如下：

分类	领域	模型名称	长度限制 (token)	Small	Medium	Large	Max
模型	文本生成（极速）	doubao-seed-2.0-mini	· 上下文窗口：256k · 最大输出：128k	✓	✓	✓	✓
	文本生成（标准）	doubao-seed-2.0-lite	· 上下文窗口：256k · 最大输出：128k	✓	✓	✓	✓
	文本生成（标准）	deepseek-v4-flash 注意	· 上下文窗口：1024k · 最大输出：384k	✓	✓	✓	✓

分类	领域	模型名称	长度限制 (token)	Small	Medium	Large	Max
		deepseek-v4-flash 为尝鲜体验版，如遇访问拥堵或频繁出现限流错误，建议切换至其他模					

分类	领域	模型名称	长度限制 (token)	Small	Medium	Large	Max
		模型。					
	文本生成 (进阶)	doubao-seed-2.0-code	· 上下文窗口: 256k · 最大输出: 128k	✓	✓	✓	✓
	文本生成 (进阶)	doubao-seed-2.0-pro	· 上下文窗口: 256k · 最大输出: 128k	✓	✓	✓	✓
	文本生成 (进阶)	minimax-m2.7	· 上下文窗口: 200k · 最大输出: 128k	✓	✓	✓	✓
	文本生成 (进阶)	minimax-m3	· 上下文窗口: 512k · 最大输出: 128k	✓	✓	✓	✓
	文本生成 (进阶)	glm-5.2 (glm-latest)	· 上下文窗口: 1024k	✓	✓	✓	✓

分类	领域	模型名称	长度限制 (token)	Small	Medium	Large	Max
			· 最大输出: 128k				
	文本生成 (进阶)	kimi-k2.6	· 上下文窗口: 256k · 最大输出: 32k	✓	✓	✓	✓
	文本生成 (进阶)	kimi-k2.7-code	· 上下文窗口: 256k · 最大输出: 32k	✓	✓	✓	✓
	文本生成 (进阶)	deepseek-v4-pro 注意 deepseek-v4-pro 为	· 上下文窗口: 1024k · 最大输出: 384k	✓	✓	✓	✓

分类	领域	模型名称	长度限制 (token)	Small	Medium	Large	Max
		尝鲜体验版，如遇访问拥堵或频繁出现限流错误，建议切换至其他模型。					
	向量化	doubao-embedding-vision	* 上下文窗口：128k	✓	✓	✓	✓
	图片生成	doubao-seedream-5.0-lite	-	✓	✓	✓	✓
	视频生成	doubao-seedance-1.5-pro	-	×	✓	✓	✓

分类	领域	模型名称	长度限制 (token)	Small	Medium	Large	Max
	视频生成	doubao-seedance-2.0	-	×	×	✓	✓
	视频生成	doubao-seedance-2.0-fast	-	×	×	✓	✓
	视频生成	doubao-seedance-2.0-mini	-	×	×	✓	✓
	语音模型	doubao-seed-tts-2.0	-	✓	✓	✓	✓
	语音模型	doubao-seed-asr-2.0	-	✓	✓	✓	✓
Harness	专业数据集	-	-	✓	✓	✓	✓
	豆包搜索	-	-	✓ 每月500次免费额度	✓ 每月500次免费额度	✓ 每月500次免费额度	✓ 每月500次免费额度
	Agent 记忆	-	-	✓	✓	✓	✓
	AI Native 应用开发底座	-	-	✓	✓	✓	✓

用量说明

采用积分（AFP）抵扣模式，详细抵扣规则请参见[套餐内 AFP 抵扣规则](#)。

注意

- 图片生成模型、视频生成模型没有五小时、周额度限制，仅受模型日额度、套餐月额度限制。
- 语音模型、Harness 没有五小时、周额度限制，仅受月额度限制。

套餐	Small	Medium	Large	Max
价格	40 元/月	200 元/月	500 元/月	1000 元/月
月额度 (AFP)	20,000	100,000	250,000	500,000
周额度 (AFP)	7,000	35,000	87,500	175,000
五小时额度 (AFP)	2,000	10,000	25,000	50,000
视觉模型日额度 (AFP)	10,000 仅图片生成模型	50,000	125,000	250,000

您可以在 [Agent Plan 控制台](#) 查看模型调用明细、Harness用量明细信息。模型调用明细数据存在 0.5 – 1 天的延迟。超额后付费部分用量仅作参考，实际以账单为准。

有效期

套餐有效期以自然月为单位，自购买成功当日开始计算。1 个月套餐有效期示例：01.31日订购成功，02.28日 23:59 到期（非闰年）。

额度刷新规则

- 未开启超额后付费：**套餐额度在时间周期内耗尽后，需要等待下一个周期自动恢复额度，不会消耗其他资源包或账户余额。抵扣规则参见[套餐内 AFP 抵扣规则](#)。
 - 五小时限额：根据首次请求发生时间，以 5 小时为周期定时刷新限额。
 - 周限额：每周一 00:00:00 重置周限额。
 - 日限额：每日 00:00:00 重置日限额（仅限视觉模型）。
 - 月限额：每订阅月第 1 日 00:00:00 重置月限额。
- 开启超额后付费：**在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息（如 Base URL、API Key、模型名称），系统将自动切换至超额后付费模式继续使用，并产生后付费账单。具体规则与操作参见[超额后付费规则](#)和[超额后付费管理](#)。

注意

- 文本生成模型、向量化模型：开启超额后付费后，当触达五小时限额、周限额或月限额时，将自动切换至超额后付费模式。套餐周期额度刷新后，会自动切换至套餐内抵扣。
- 语音模型、Harness：没有五小时限额与周限额；开启超额后付费后，仅在触达月限额时才会自动切换至超额后付费模式。

额度使用限制

- 同一订阅套餐可在所有支持的工具中使用，额度共享。
- 文本生成模型及向量化模型不可用于 API 调用，在非 AI 工具中使用 Agent Plan 权益对应的 Base URL 和 API Key 有可能被识别为滥用/违规，会导致订阅停用或账号封禁。

订阅规则

购买限制

单次下单套餐时长不能超过12个月，存量套餐时长 + 新购套餐时长不能超过24个月。

续费规则

- 自动续费：若您在下单时勾选自动续费，即连续包月，每次默认续费 1 个月，到期会自动扣除下一个月的套餐费用。
- 手动续费：若您在下单时没有勾选自动续费，可在[开通管理](#) 进行续费操作。

升配规则

套餐有效期不变，补齐升配套餐的差价，即可升配，可在[开通管理](#) 进行升配操作。套餐仅支持升配，不支持降配。

· 差价计算公式

升配需补差价金额 = (升配套餐月价 - 原套餐月价) / 30 天 * 套餐剩余时长

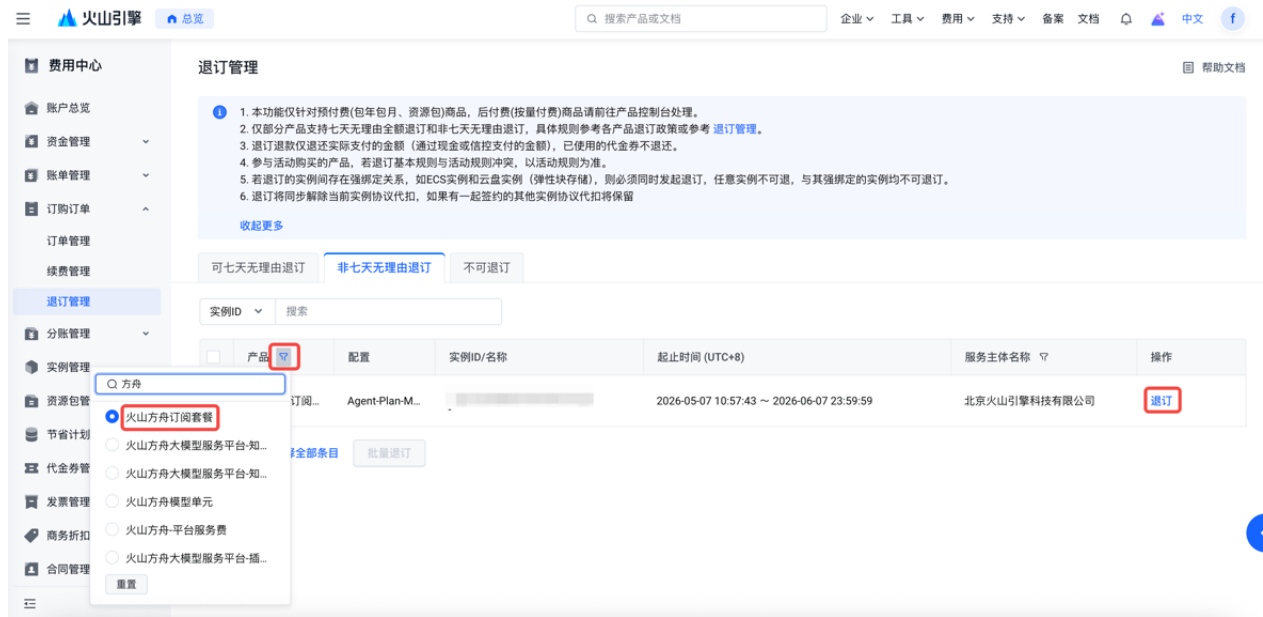
· 额度折算规则

- 月限额：升配后当月可用额度 = 升配套餐月额度 * 当月剩余时长占比 + 原套餐月额度 * 当月已用时占比
- 5小时限额与周限额：升配完成后，5小时限额、周限额均直接为升配套餐对应的限额。

退订规则

支持非七天无理由自助退订，退订说明及退款计算公式可详见[非七天无理由退订说明](#)。具体操作如下：

1. 进入火山引擎[费用中心-退订管理](#)。
2. 在 **非七天无理由退订** 页签，在 **产品** 列过滤 **火山方舟订阅套餐**，找到要退订的套餐，在操作列单击 **退订**，根据界面提示完成套餐退订。



接入 AI 工具

Claude Code

AI 终端编程助手，自然语言编程

TRAE

智能 IDE，构建新一代编程 workflow

Roo Code

轻量级编程助手

Codex CLI

OpenAI 推出的命令行编程工具

OpenCode

开源 AI 编程代理工具

Cline

VSCode 扩展，代码补全和调试

Kilo Code

轻量高性能编程工具

OpenClaw

开源、自托管个人 AI 助手

Cursor

AI 原生代码编辑器

Hermes Agent

开源自主 AI 智能体

常见问题

[常见问题](#)

数据授权使用协议

[火山引擎数据授权使用协议](#) [🔗](#)

2. 计费说明

2.1 套餐内 AFP 抵扣规则

方舟 Agent Plan 套餐内用量采用 AFP 作为统一用量单位，本文帮助您快速了解套餐内用量抵扣规则。

AFP 定义

Agent 燃料值（Agent Fuel Point，简称 AFP）是 Agent Plan 套餐的统一用量计费单位，用于量化智能体（Agent）资源的消耗。

抵扣公式

单次请求 AFP 消耗：

- 文本生成模型、向量化模型： $(\text{输入 token} * \text{输入抵扣系数} + \text{输出 token} * \text{输出抵扣系数}) / 10,000$
- 视频生成模型： $\text{消耗的 token} / 10,000 * \text{抵扣系数}$
- 图片生成模型： $\text{成功生成的图片张数} * \text{抵扣系数}$
- 语音模型：
 - doubao-seed-tts-2.0： $\text{文本字符数} / 10,000 * \text{抵扣系数}$
 - doubao-seed-asr-2.0： $\text{语音文件时长(小时)} * \text{抵扣系数}$
- 专业数据集： $\text{调用次数} * \text{抵扣系数}$

注意

生成视频预估消耗的 token 可参见[价格示例](#)中 token 用量估算公式。

抵扣系数

根据模型及 Harness 的层级不同，AFP 的抵扣系数也不同。

说明

2026年6月10日18:00:00至2026年7月15日00:00:00，Agent/Coding Plan 个人版用户使用 deepseek-v4-pro、kimi-k2.6、kimi-k2.7-code、glm-5.2 模型时，模型抵扣系数在现有抵扣系数基础上享最低 2.5 折优惠。详情请参见[Agent/Coding Plan 指定模型抵扣系数限时折扣活动](#)。

模型

文本生成模型与向量化模型

模型决定模型抵扣系数，输入长度决定输入分段系数。

模型类别	模型	输入长度	输入抵扣系数 (模型抵扣系数 × 输入分段系 数)	输出抵扣系数
文本生成（极 速）	doubao- seed-2.0-mini	输入长度 ≤32k	0.25×0.67	0.25
		32k<输入长度 ≤128k	0.25×1	0.25
		输入长度>128k	0.25×2	0.25
文本生成（标 准）	doubao- seed-2.0-lite deepseek-v4- flash	输入长度 ≤32k	0.5×0.67	0.5
		32k<输入长度 ≤128k	0.5×1	0.5
		输入长度>128k	0.5×2	0.5
文本生成（进 阶）	doubao- seed-2.0-code doubao- seed-2.0-pro minimax-m2.7 minimax-m3	输入长度 ≤32k	2.5×0.67	2.5
		32k<输入长度 ≤128k	2.5×1	2.5
		输入长度>128k	2.5×2	2.5
文本生成（进 阶）	kimi-k2.6	输入长度 ≤32k	4.5×0.67 (1.8×0.67 限时4折)	4.5 (1.8 限时4折)
		32k<输入长度 ≤128k	4.5×1 (1.8×1 限时4折)	4.5 (1.8 限时4折)
		输入长度>128k	4.5×2 (1.8×2 限时4折)	4.5 (1.8 限时4折)
文本生成（进 阶）	kimi-k2.7-code	输入长度 ≤32k	4.5×0.67 (1.125×0.67 限时2.5折)	4.5 (1.125 限时2.5折)

模型类别	模型	输入长度	输入抵扣系数 (模型抵扣系数 × 输入分段系 数)	输出抵扣系数
		32k<输入长度 ≤128k	4.5×1 (1.125×1 限时2.5折)	4.5 (1.125 限时2.5折)
		输入长度>128k	4.5×2 (1.125×2 限时2.5折)	4.5 (1.125 限时2.5折)
文本生成（进 阶）	glm-5.2 (glm- latest)	输入长度 ≤32k	4.5×0.67 (1.125×0.67 限时2.5折)	4.5 (1.125 限时2.5折)
		32k<输入长度 ≤128k	4.5×1 (1.125×1 限时2.5折)	4.5 (1.125 限时2.5折)
		输入长度>128k	4.5×2 (1.125×2 限时2.5折)	4.5 (1.125 限时2.5折)
文本生成（进 阶）	deepseek-v4- pro	输入长度 ≤32k	5.5×0.67 (2.2×0.67 限 时4折)	5.5 (2.2 限时4折)
		32k<输入长度 ≤128k	5.5×1 (2.2×1 限时4折)	5.5 (2.2 限时4折)
		输入长度>128k	5.5×2 (2.2×2 限时4折)	5.5 (2.2 限时4折)
向量化	doubao- embedding- vision	输入长度 ≤32k	0.5×0.67	0.5
			0.5×1	0.5

模型类别	模型	输入长度	输入抵扣系数 (模型抵扣系数 × 输入分段系数)	输出抵扣系数
		32k<输入长度 ≤128k		
		输入长度>128k	0.5 × 2	0.5

视觉模型

模型决定抵扣系数。

模型类别	模型	抵扣系数	单位
视频生成	doubao-seedance-1.5-pro	输出 是否包含声音 · 无声视频：36 · 有声视频：72	token
视频生成	doubao-seedance-2.0	· 输出视频分辨率为480p，720p ◦ 输入包含视频：140 ◦ 输入不含视频：230 · 输出视频分辨率为1080p ◦ 输入包含视频：155 ◦ 输入不含视频：255 · 输出视频分辨率为4k ◦ 输入包含视频：80 ◦ 输入不含视频：130	token
视频生成	doubao-seedance-2.0-fast	输入 是否包含视频 · 输入包含视频：110	token

模型类别	模型	抵扣系数	单位
		· 输入不含视频： 185	
视频生成	doubao-seedance-2.0-mini	输入 是否包含视频 · 输入包含视频：70 · 输入不含视频： 115	token
图片生成	doubao-seedream-5.0-lite	1张成功生成的 图片 = 99 AFP 99	张

语音模型

模型决定抵扣系数。

模型类别	模型	抵扣系数	单位
语音模型	doubao-seed-tts-2.0	1350	万字符
语音模型	doubao-seed-asr-2.0	450	小时

Harness

专业数据集

Harness	数据类型	抵扣系数	单位
专业数据集	中国金融数据库	12	次
专业数据集	企业工商数据库	12	次
专业数据集	企业风险数据库	12	次
专业数据集	科研学术数据搜索服务	24	次
专业数据集	其他专业数据搜索服务	12	次

豆包搜索

豆包搜索支持使用 Agent Plan 套餐内 AFP 进行抵扣，抵扣系数如下：

Harness	抵扣系数	单位
豆包搜索	5	次

Agent 记忆

每个数据库独立计费，每个库抵扣情况如下：

文件情况	抵扣系数	单位
前50个文件	免费	-
50 < 文件数 ≤ 4 万	5	小时
文件数 > 4万	1.5	万个文件/小时

注意

- 一旦开通抵扣，该账号下 OpenViking 所有数据库均从 Agent Plan 抵扣，不支持部分抵扣。
- 关闭 Agent Plan 抵扣会立即自动关停 OpenViking 商品，并不定期清理数据，请谨慎操作。
- 请确保您的官网账户余额 ≥ 0，避免由于欠费导致数据回收。

AI Native 应用开发底座

AI Native 应用开发底座采用按量付费模式，各计费项按标准换算为 AFP，由方舟 Agent Plan 统一抵扣。
AI Native 应用开发底座的计费项与抵扣系数如下：

计费项	说明	抵扣系数	单位
数据库算力	Workspace 计算资源 的费用，以 CU 作为 计费单元。1 个 CU 等 价于 1 核 CPU 4 GiB 内存的计算资源。	250	元/CU/小时
Supabase 通用组件算 力	Supabase 服务通用组 件计算资源的费用。	250	元/CU/小时
Edge Function 执行次 数	Supabase 服务调用 Edge Function 产生的 费用。	70	元/万次
对象存储容量	Supabase 服务使用的 对象存储容量的费 用。	0.125	元/GiB/小时
数据库存储容量	Workspace 的数据存 储空间的费用。	0.625	元/GiB/小时

说明

计费项对应的规格非 Supabase 的最小可用规格，若采用最小规格（数据库算力 0.5 CU、Supabase 通用组件算力 0.25 CU），每小时仅需消耗 187.5 AFP 及少量存储相关 AFP。

抵扣示例

- 文本生成模型示例：当模型设置为 doubao-seed-2.0-code，某次请求输入为 50k token，输出为 0.5k token。

Plain

本次 AFP 消耗
= (输入 token * 输入抵扣系数 + 输出 token * 输出抵扣系数) / 10,000
= (50000 * 2.5 + 500 * 2.5) / 10000
= 12.625 AFP

- 视频生成模型示例：当模型设置为 doubao-seedance-2.0-fast，某次视频任务输入包含视频，成功生成视频，消耗 150k token。

Plain

本次 AFP 消耗
= 消耗的 token / 10000 * 抵扣系数
= 150000 / 10000 * 110
= 1650 AFP

- 图片生成模型示例：当模型设置为 doubao-seedream-5.0-lite，成功生成一张图片，消耗 99 AFP。
- 语音模型示例：当模型设置为 doubao-seed-tts-2.0，某次请求文本为 2000 字符数，消耗 270 AFP。

Plain

本次 AFP 消耗
= 文本字符数 / 10,000 * 抵扣系数
= 2000 / 10000 * 1350
= 270 AFP

- 专业数据集：成功调用一次中国金融数据库，消耗 12 AFP。

2.2 超额后付费规则

开启超额后付费后，系统将在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，不需要修改任何配置信息（如 Base URL、API Key、模型名称），将自动切换至超额后付费模式继续使用支持的模型或 Harness，相关调用将产生后付费账单。

模型超额后付费规则

如果需要在套餐额度耗尽后继续使用模型，可开启模型超额后付费，具体操作参见[管理模型超额后付费](#)。
支持开启超额后付费的模型及具体的后付费规则见下表。

说明

auto 模式不支持超额后付费使用。

模型类别	模型名称	是否支持开启超额后付费	后付费模型价格	后付费计费说明
文本生成（极速）	doubao-seed-2.0-mini	✓	在线推理（常规）	模型后付费计费说明
文本生成（标准）	doubao-seed-2.0-lite	✓	在线推理（常规）	模型后付费计费说明
文本生成（标准）	deepseek-v4-flash	✓	在线推理（常规）	模型后付费计费说明
文本生成（进阶）	doubao-seed-2.0-code	✓	在线推理（常规）	模型后付费计费说明
文本生成（进阶）	doubao-seed-2.0-pro	✓	在线推理（常规）	模型后付费计费说明
文本生成（进阶）	minimax-m2.7	×	—	—
文本生成（进阶）	minimax-m3	×	—	—
文本生成（进阶）	glm-5.2 (glm-latest)	×	—	—
文本生成（进阶）	kimi-k2.6	×	—	—
文本生成（进阶）	deepseek-v4-pro	✓	在线推理（常规）	模型后付费计费说明
向量化	doubao-embedding-vision	✓	向量模型	模型后付费计费说明

模型类别	模型名称	是否支持开启超额后付费	后付费模型价格	后付费计费说明
图片生成	doubao-seedream-5.0-lite	×	-	-
视频生成	doubao-seedance-1.5-pro	×	-	-
视频生成	doubao-seedance-2.0	×	-	-
视频生成	doubao-seedance-2.0-fast	×	-	-
视频生成	doubao-seedance-2.0-mini	×	-	-
语音模型	doubao-seed-tts-2.0	✓	3元/万字符	按调用后付费 ↗
语音模型	doubao-seed-asr-2.0	✓	1元/小时	按调用后付费 ↗

Harness 超额后付费规则

如果需要在套餐额度耗尽后继续使用 Harness，可开启 Harness 超额后付费，具体操作参见[管理 Harness 超额后付费](#)。

专业数据集

单次调用超额后付费的费用如下，具体的后付费规则参见：[高质量数据集计费](#) [↗](#)。

数据类型	超额后付费费用
中国金融数据库	0.024 元/次
企业工商数据库	0.024 元/次
企业风险数据库	0.024 元/次
科研学术数据搜索服务	0.048 元/次
其他专业数据搜索服务	0.024 元/次

豆包搜索

单次调用超额后付费的费用如下，具体后付费规则参见：[web信息搜索](#) [↗](#)。

Harness	超额后付费费用
豆包搜索	0.020元/次

Agent 记忆

Agent 记忆支持开启超额后付费，具体的后付费规则参见：[计费说明](#)。

AI Native 应用开发底座

AI Native 应用开发底座支持开启超额后付费，具体的后付费规则参见：[计费项与价格](#)。

2.3 超额后付费管理

当套餐额度不足但仍需在当前套餐继续使用支持的模型或 Harness 时，可开启超额后付费。本文介绍 Agent Plan 个人版中模型和 Harness 超额后付费的开启/关闭方式及相关说明。

管理模型超额后付费

前提条件：已在[开通管理页面](#) 开通模型服务。

1. 在 [Agent Plan](#) 套餐信息区域，单击超额后付费管理。

套餐信息

套餐类型

套餐限额

订阅状态

开始时间

结束时间

剩余时间

计费模式

自动续费

查看权益

♥Large 套餐

25000 AFP/5小时
0 AFP/周
0 AFP/月
计算规则详见 [说明文档](#)

生效中

2026-06-02 16:45

2027-04-02 23:59

303天

包月

[查看文档](#)

续费套餐

升级套餐

超额后付费管理

2. 在超额后付费管理的模型页签中，按需开启或关闭模型的超额后付费开关。

超额后付费管理

×

模型

Harness

ⓘ 当前auto模式不支持超额付费使用。

[如何修改?](#)

全部

语言模型

向量化模型

视觉模型

语音模型

模型名/提供方	超额后付费	推理定价	开启超额后付费
Doubao-Seed-TTS-2.0	☑ 已开启	-	
Doubao-Seed-ASR-2.0	☑ 已开启	-	

注意

- 如果在[开通管理页面](#) 关闭模型服务，则不能继续使用 Agent Plan 的模型超额后付费。
- 支持开启超额后付费的模型及后付费规则，参见[模型超额后付费规则](#)。

管理 Harness 超额后付费

开启/关闭抵扣

- 开启抵扣：开启后，对应 Harness 在套餐额度充足时会优先消耗套餐内 AFP。开启抵扣的方式如下：
 - 在购买套餐时，选中相关协议，支持的 Harness 会开启抵扣。
 - 在 [Agent Plan](#) 配置 Harness 区域，在对应 Harness 卡片开启抵扣。

使用配置

用量统计

配置文档

1 配置模型及Base URL

2 配置Harness (可选)

3 配置专属API Key

4 接入使用

专属 Harness开箱即用，解锁 Agent 场景全新体验，更多能力持续扩展中

联网搜索 Beta版

限时免费额度 Skill/MCP

权益生效时间：2026/04/16-2028/03/27

剩余免费额度：9994/10000

超额后付费

查看联网密钥 查看文档

专业数据集

金融股票 工商企业库 学术论文 MCP

面向AI agent场景的搜索基础设施，聚合多个垂直专业领域的数据源，提供高质量专业数据检索能力

抵扣说明：金融数据库、企业工商数据库、企业风险数据库 12AFP/次，科研学术数据搜索服务 24AFP/次

开启抵扣 超额后付费

首次上线 Hot

能力介绍 配置说明

- 关闭抵扣：关闭后，不能使用套餐内该 Harness 服务，即对应 Harness 不再消耗套餐内 AFP 额度，同时其超额后付费也会关闭。关闭抵扣的方式如下：
在配置Harness区域，在对应 Harness 卡片关闭抵扣。

开启/关闭超额后付费

开启超额后付费后，在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息（如 Base URL、API Key、模型名称），系统将自动切换至超额后付费模式继续使用对应 Harness，并产生后付费账单；关闭后，在套餐周期内额度用尽时不再支持通过当前调用方式继续使用对

版权所有©北京火山引擎科技有限公司

22/215

应 Harness。
可通过以下方式开启或关闭 Harness 超额后付费：

- 在Agent Plan 套餐信息区域，单击超额后付费管理，并在 Harness 页签中按需开启或关闭对应的超额后付费开关。

超额后付费管理

×

模型

Harness

! 当前auto模式不支持超额付费使用。

如何修改?

Harness

超额后付费

开启超额后付费

专业数据集

已开启

- 在配置 Harness区域，按需开启或关闭对应 Harness 卡片的超额后付费开关。

使用配置

用量统计

配置文档

1 配置模型及Base URL

2 配置Harness (可选)

3 配置专属API Key

4 接入使用

专属 Harness开箱即用，解锁 Agent 场景全新体验，更多能力持续扩展中

联网搜索 Beta版

限时免费额度 Skill/MCP

权益生效时间：2026/04/16-2028/03/27

剩余免费额度：9994/10000

超额后付费

查看联网密钥 查看文档

专业数据集 首次上线 Hot

金融股票 工商企业库 学术论文 MCP

面向AI agent场景的搜索基础设施，聚合多个垂直专业领域的数据库，提供高质量专业数据检索能力

抵扣说明：金融数据库、企业工商数据库、企业风险数据库 12AFP/次，科研学术数据搜索服务 24AFP/次

开启抵扣

超额后付费

能力介绍 配置说明

注意
支持开启超额后付费的 Harness 及后付费规则，参见Harness 超额后付费规则。

版权所有©北京火山引擎科技有限公司

23/215

3. 快速开始

本文介绍如何在 AI 工具中快速配置和使用 Agent Plan。

一、购买 Agent Plan

进入 [Agent Plan 控制台](#)，按需购买套餐。套餐介绍参见[套餐概览](#)。

二、选择 AI 工具

Agent Plan 支持主流的 AI 工具，可按需选择。

Claude Code

AI 终端编程助手，自然语言编程

TRAE

智能 IDE，构建新一代编程 workflow

Roo Code

轻量级编程助手

Codex CLI

OpenAI 推出的命令行编程工具

OpenCode

开源 AI 编程代理工具

Cline

VSCode 扩展，代码补全和调试

Kilo Code

轻量高性能编程工具

OpenClaw

开源、自托管个人 AI 助手

Cursor

AI 原生代码编辑器

Hermes Agent

开源自主 AI 智能体

三、配置 AI 工具

下面以 Claude Code 为例介绍如何使用 Agent Plan。

1.安装 Claude Code

请参考 [官方文档的安装章节](#) 完成 Claude Code 安装。

2. 配置 Claude Code

需配置的信息

如您对 Claude Code 熟悉 / 使用 cc-switch 等配置工具 / 本地已有配置 / 已有本地 Agent等，可获取下列信息直接配置/让本地 Agent 帮您配置。

- **ANTHROPIC_BASE_URL**：`https://ark.cn-beijing.volces.com/api/plan`，为专属 Base URL，其他 Base URL 无法在 Agent Plan 中使用。
- **ANTHROPIC_AUTH_TOKEN**：[获取 Agent Plan API Key](#)，为专属 API Key，其他方舟 API Key 如Coding Plan API Key 无法在 Agent Plan 中使用。



- **ANTHROPIC_MODEL**：见 [支持模型及 Harness](#)，不同 Agent Plan 套餐支持的模型不同。请先配置文本生成模型，后续图片生成模型、视频生成模型等可在 Claude Code 中使用自然语言/Skill 等方式完成配置。

注意

若您本地已有Claude Code配置（如终端启动命令 `.bashrc` / `.zshrc` 等），可能因为配置冲突导致 Claude Code 不生效，可以尝试备份并清理本地配置后再次运行。

方式 1：自动化助手（推荐）

ArkCLI Helper 是一个编码工具助手，支持快速配置选择的工具接入 Agent Plan。安装并运行该助手，根据界面提示操作可自动完成工具配置，降低手动配置的时间成本和出错风险。

注意

- ArkCLI Helper 是基于 Ark Helper 升级演进的工具配置服务，相关能力将持续在 ArkCLI Helper 上迭代更新。建议优先使用 ArkCLI Helper 完成工具配置。
- ArkCLI Helper 支持 macOS、Linux、Windows 系统。

1. 安装 Ark CLI。

Bash

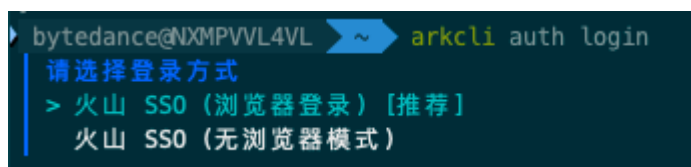
```
# 1. 通过 NPM 安装
npm install -g @volcengine/ark-cli@latest

# 2. 验证安装
arkcli --version
```

2. 登录 Ark CLI。

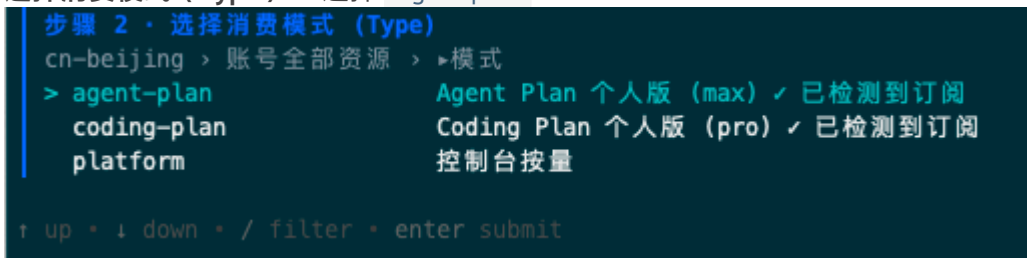
Bash

```
arkcli auth login
```



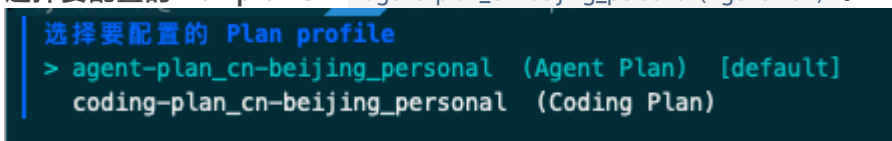
根据界面提示选择登录方式。首次登录时，按提示完成以下配置：

- 选择项目（Project）：可按需选择，或默认选择 账号全部资源。
- 选择消费模式（Type）：选择 `agent-plan`。



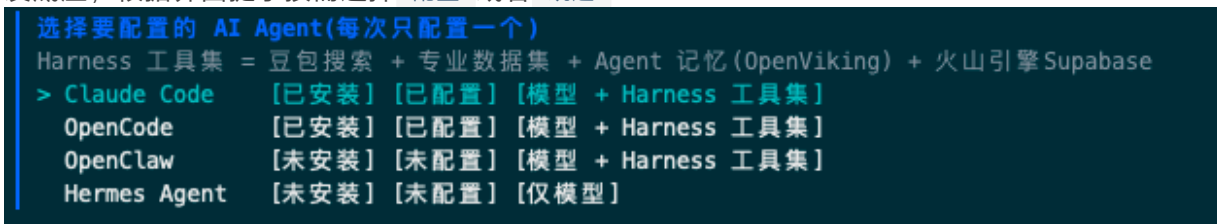
3. 运行 ArkCLI Helper，根据界面提示完成 Claude Code 的配置。

- 在命令行界面输入 `arkcli helper` 命令，启动 ArkCLI Helper。
- 选择要配置的 Plan profile： `agent-plan_cn-beijing_personal` (Agent Plan)。



- 选择默认 model：按需选择。
- 选择要配置的 AI Agent： `Claude Code`。

当前支持的 Harness 包括：豆包搜索、专业数据集、Agent 记忆（OpenViking）、AI Native 应用开发底座，根据界面提示按需选择 配置 或者 跳过。



完成上述操作后，将自动完成 Claude Code 的基础配置，并按选择结果自动安装对应的 Harness 工具，无需手动安装。

说明

Claude Code 的原生搜索工具仅支持 Claude 系列模型，第三方模型（如 Doubao、GPT 等）无法直接调用。但通过 [豆包搜索](#) 可在第三方模型上同样可以正常使用搜索能力。

方式 2：手动配置

下面详细介绍如何手动完成上述信息配置。

1. 创建并打开 `settings.json` 配置文件。文件路径因系统而异，具体操作如下：

macOS/Linux

macOS/Linux 系统 Claude Code 配置文件路径：`~/claude/settings.json`。

1. 如果在主目录下没有 `~/.claude` 目录，执行以下命令创建目录。

Bash

```
mkdir -p ~/.claude
```

2. 创建并打开配置文件。

Bash

```
nano ~/.claude/settings.json
```

Windows

Windows 系统 Claude Code 配置文件路径：`C:\Users\<用户名>\.claude\settings.json`。以 CMD 命令行方式为例，操作如下。

1. 如果当前用户目录下没有 `~\.claude` 目录，执行以下命令创建目录。

Bash

```
if not exist "%USERPROFILE%\.claude" mkdir "%USERPROFILE%\.claude"
```

2. 创建并打开配置文件。

Bash

```
notepad "%USERPROFILE%\.claude\settings.json"
```

2. 编辑 `settings.json`，需修改的配置信息如下：

- `<ARK_API_KEY>`：替换为您的 [ARK Agent Plan API Key](#) [🔗](#)
- `<MODEL_NAME>`：结合套餐及您的需求，更新 **文本生成模型** 的名称，如 `ark-code-latest`，详细模型名称参见 [支持模型及 Harness](#)。

JSON

```
{
  "env": {
    "ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>",
    "ANTHROPIC_BASE_URL": "https://ark.cn-beijing.volces.com/api/plan",
    "ANTHROPIC_MODEL": "<MODEL_NAME>",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "<MODEL_NAME>",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "<MODEL_NAME>",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "<MODEL_NAME>",
    "CLAUDE_CODE_SUBAGENT_MODEL": "<MODEL_NAME>"
  }
}
```

说明

- 要开启 1M 上下文窗口，配置参见[扩展上下文](#)。
- 推荐使用完整模型配置，并按任务复杂度选择模型：Haiku（轻量）、Sonnet（日常）、Opus（复杂）。
- `CLAUDE_CODE_SUBAGENT_MODEL` 建议与主模型保持一致。
- `ANTHROPIC_DEFAULT_HAIKU_MODEL` 建议设置为小尺寸模型，通常不会影响整体使用效果。

3. 编辑或新增 `.claude.json` 文件，修改或新增 `hasCompletedOnboarding` 字段值为 `true`，设置状态为完成入门引导。

macOS/Linux

a. 创建并打开配置文件。

Bash

```
nano ~/.claude.json
```

ii. 修改或新增的字段信息如下：

JSON

```
{  
  "hasCompletedOnboarding": true  
}
```

Windows

a. 创建并打开配置文件。

Bash

```
notepad "%USERPROFILE%\claude.json"
```

ii. 修改或新增的字段信息如下：

JSON

```
{  
  "hasCompletedOnboarding": true  
}
```

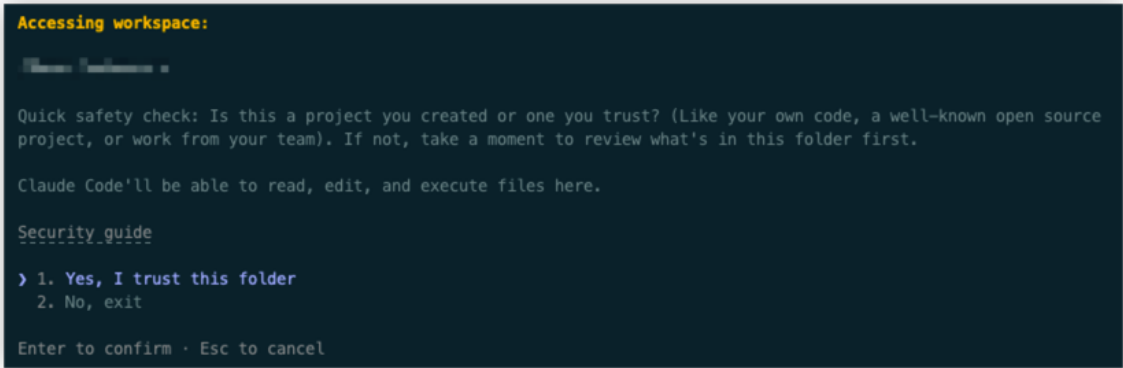
3. 启动并使用 Claude Code

1. 启动Claude Code：进入项目目录，执行 `claude` 命令，即可开始使用 Claude Code。

Bash

```
cd my-project  
claude
```

启动后，选择信任此文件夹，允许 Claude Code 访问该文件夹中的文件。



2. 模型状态验证：输入 `/model` 确认使用的模型指定正确。
3. 在 Claude Code 中对话。

更多使用教程请参见 [Claude Code 官网](#)。

其他操作

添加图片生成/视频生成能力

方舟 Agent Plan Medium 及以上套餐提供图片生成、视频生成模型配额。方舟提供 Skill 帮助 AI 工具快速接入并使用。具体配置方法请参见：

- [方式 2：手动配置](#)
- [配置图片生成模型](#)

注意

- 不同的套餐支持的视觉模型有所不同，具体参见[支持模型及 Harness](#)。
- 视觉模型不支持通过 Auto 及控制台切换使用。

切换模型

你可以随时根据需求切换 **文本生成模型**。方舟支持多种方式切换模型。

配置方式	通过控制台	通过命令参数	修改配置文件
切换模型操作	1. 参考2. 配置 Claude Code配置模型名称为 <code>ark-code-latest</code>。 2. 在开通管理页面选择要使用的模型，切换后3-5分钟即可生效。	· 启动时：执行 <code>claude --model <MODEL_NAME></code>，可指定本次对话使用的文本生成模型。 · 对话期间：执行 <code>/model</code>	参考 2. 配置 Claude Code 配置模型名称为您需要的模型。

配置方式	通过控制台	通过命令参数	修改配置文件
	3. 在命令行窗口，启动 Claude Code 后，输入 <code>/model</code> 确认模型状态。	<code><MODEL_NAME></code> 切换模型。	

4. 接入 AI 工具

4.1 Codex

本文介绍如何配置方舟 Agent Plan 到 Codex 中使用。

前提条件

您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。

Codex CLI

一、安装 Codex CLI

说明

Agent Plan 支持 Responses API，可以使用最新版 Codex CLI。

前提条件：安装 [Node.js 18 或更新版本](#)。
在命令行界面，执行以下命令安装 Codex CLI。

Bash

```
npm i -g @openai/codex
```

安装结束后，执行以下命令检查版本。

Bash

```
codex --version
```

二、配置 Codex CLI

1. 创建并打开 Codex 的配置文件。文件路径因系统而异，具体操作如下：

macOS/Linux

macOS/Linux 系统 Codex 配置文件路径：`~/codex/config.toml`。

1. 如果在主目录下没有`.codex`目录，执行以下命令创建目录。

Bash

```
mkdir -p ~/.codex
```

2. 创建并打开配置文件。

Bash

```
nano ~/.codex/config.toml
```

Windows

Windows 系统 Codex 配置文件路径：`%USERPROFILE%\codex\config.toml`。以 CMD 命令行方式为例，操作如下。

1. 如果当前用户目录下没有`.codex`目录，执行以下命令创建目录。

Bash

```
if not exist "%USERPROFILE%\codex" mkdir "%USERPROFILE%\codex"
```

2. 创建并打开配置文件。

Bash

```
notepad "%USERPROFILE%\codex\config.toml"
```

2. 编辑 `config.toml`，需关注的配置信息如下：

- `model`：需配置文本生成模型。支持配置 `ark-code-latest`、配置 Model Name。模型信息见[支持模型及 Harness](#)。
- `env_key`：设置的是环境变量名称，请不要直接修改 `ARK_API_KEY`，您需要在下一步设置该环境变量的值。

```
model = "<Model_Name>"
model_provider = "volcengine-agent-plan"
```

```
model_supports_reasoning_summaries = true

[model_providers.volcengine-agent-plan]
name = "volcengine-agent-plan"
base_url = "https://ark.cn-beijing.volces.com/api/plan/v3"
env_key = "ARK_API_KEY"
wire_api = "responses"
```

注意

- `model_supports_reasoning_summaries = true` : 开启推理能力。
- `model_reasoning_effort` : 控制思考长度, 可以设置为 `low` 、 `medium` 、 `high` 。
- `minimax-m2.7`、`kimi-k2.6`、`kimi-k2.7-code` 不支持设置 `model_supports_reasoning_summaries = true` 。

3. 配置环境变量, 需要将 `ARK_API_KEY` 环境变量设置为 [专属 API Key](#)。

macOS / Linux

a. 查看 Shell 类型。

Bash

```
echo $SHELL
```

ii. 将环境变量写入 Shell 配置文件, 使其在新开终端时自动生效。

■ Zsh

Bash

```
# 需要将 YOUR_API_KEY 替换为专属 API Key。
echo 'export ARK_API_KEY="YOUR_API_KEY"' >> ~/.zshrc
source ~/.zshrc
```

* Bash

Bash

```
# 需要将 YOUR_API_KEY 替换为专属 API Key。
echo 'export ARK_API_KEY="YOUR_API_KEY"' >> ~/.bashrc
source ~/.bashrc
```

Windows

- CMD：环境变量设置完成后，新开 CMD 窗口生效。

Bash

需要将 YOUR_API_KEY 替换为专属 API Key。

```
setx ARK_API_KEY "YOUR_API_KEY"
```

新开窗口，检查环境变量是否生效

```
echo %ARK_API_KEY%
```

- * PowerShell：环境变量设置完成后，新开 PowerShell 窗口生效。

Bash

需要将 YOUR_API_KEY 替换为专属 API Key。

```
[Environment]::SetEnvironmentVariable("ARK_API_KEY", "YOUR_API_KEY",  
[EnvironmentVariableTarget]::User)
```

新开窗口，检查环境变量是否生效

```
echo $env:ARK_API_KEY
```

三、开始使用

执行以下命令启动 Codex CLI。

Bash

```
codex
```

Codex 桌面客户端

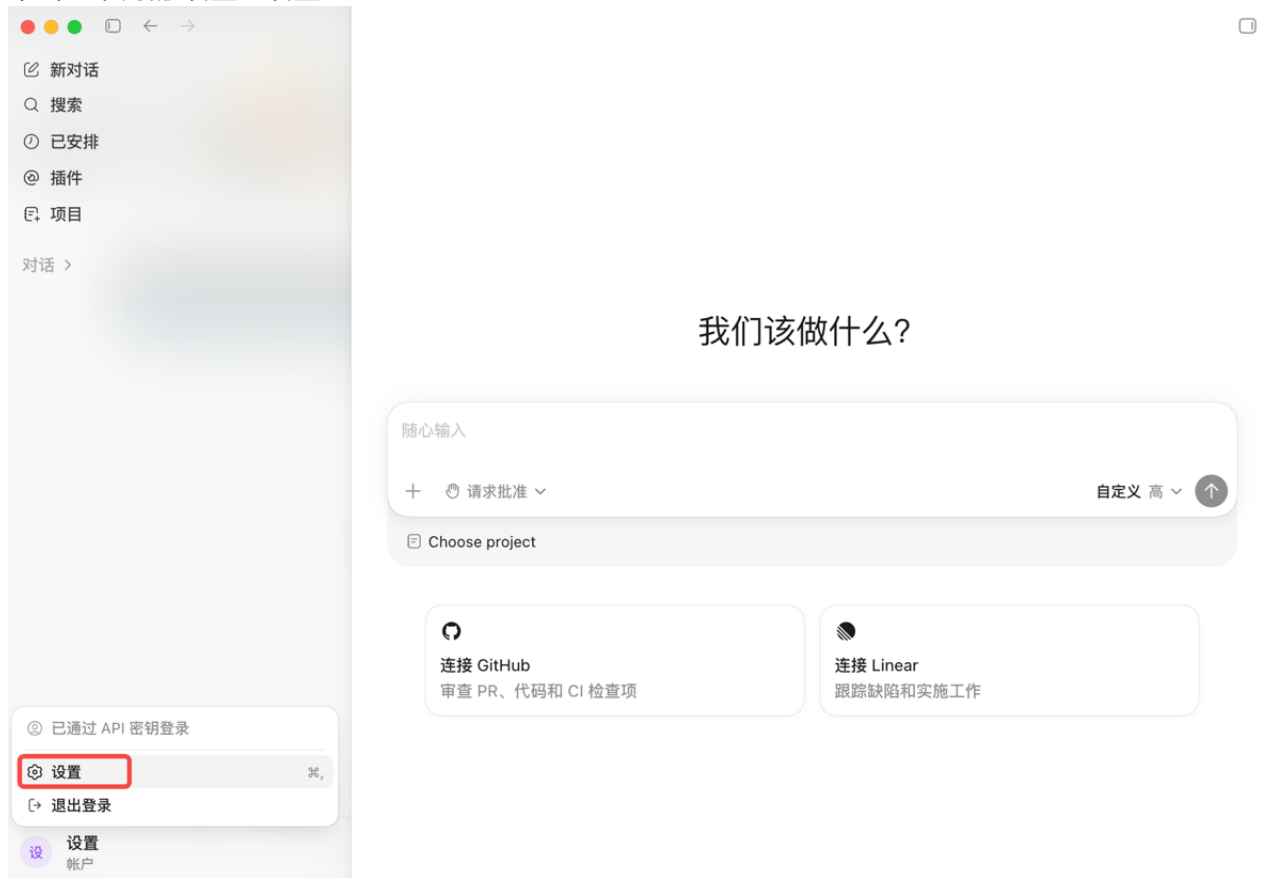
一、安装 Codex 桌面客户端

从 [Codex app](#) 下载并安装 Codex 桌面客户端。

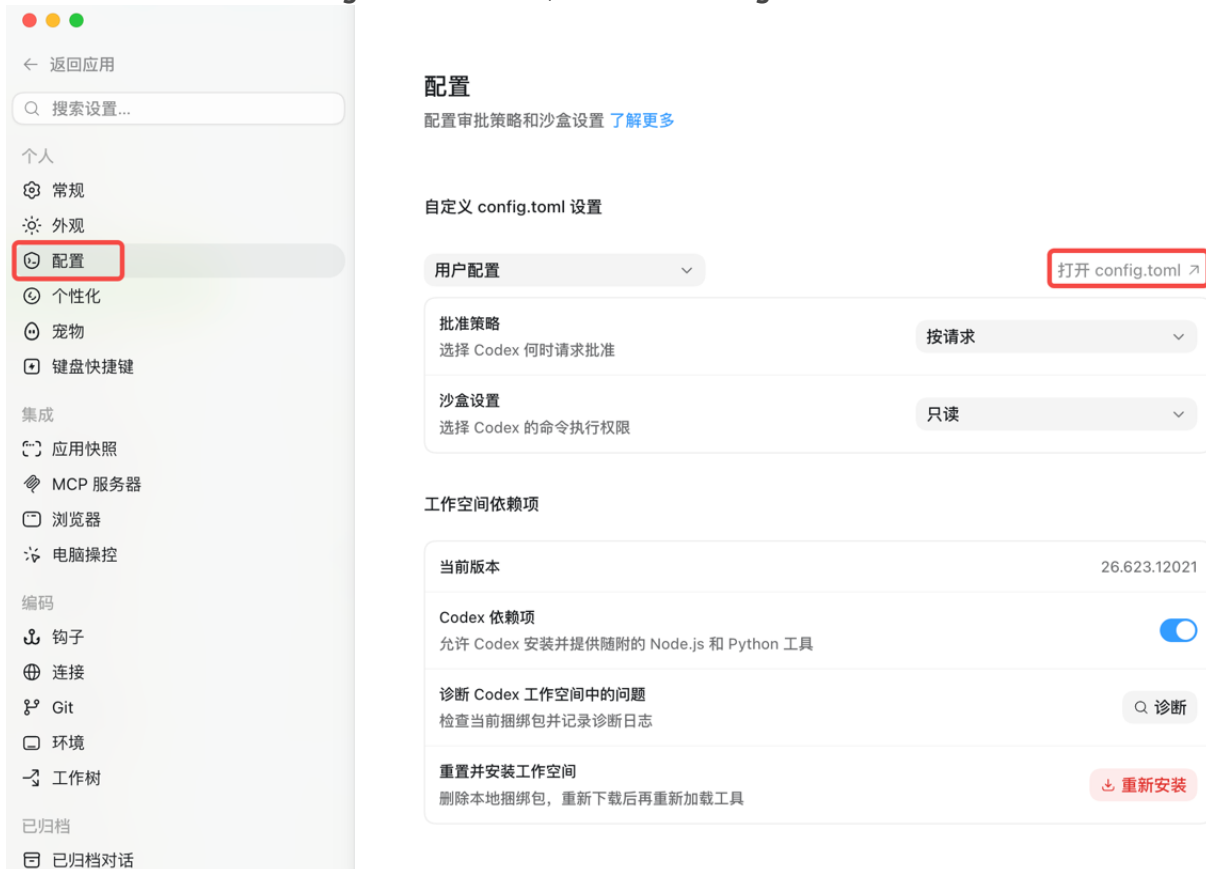
二、配置 Codex 桌面客户端

登录 Codex 桌面客户端后，根据以下步骤进行配置。

1. 单击左下角的 设置 - 设置。



2. 在 配置 页面的 自定义 config.toml 设置 区域，单击 打开 config.toml 。



3. 编辑 config.toml ，需关注的配置信息如下。

- **model** ：需配置文本生成模型。支持配置 `ark-code-latest` 、配置 Model Name。模型信息见[支持模型及 Harness](#)。
- **env_key** ：设置的是环境变量名称，请不要直接修改 `ARK_API_KEY` ，您需要在下一步设置该环境变量的值。

Plain

```
model = "<Model_Name>"
model_provider = "volcengine-agent-plan"
model_supports_reasoning_summaries = true

[model_providers.volcengine-agent-plan]
name = "volcengine-agent-plan"
base_url = "https://ark.cn-beijing.volces.com/api/plan/v3"
env_key = "ARK_API_KEY"
wire_api = "responses"
```

注意

- `model_supports_reasoning_summaries = true` ：开启推理能力。
- `model_reasoning_effort` ：控制思考长度，可以设置为 `low` 、 `medium` 、 `high` 。

■ minimax-m2.7、kimi-k2.6、kimi-k2.7-code 不支持设置
`model_supports_reasoning_summaries = true`。

4. 配置环境变量，需要将 `ARK_API_KEY` 环境变量设置为 [专属 API Key](#)，具体操作可参见[配置 Codex CLI](#)。

三、开始使用

配置完成后，建议退出并重新启动 Codex 桌面客户端以使配置生效。重新启动后，即可在对话界面中使用。

4.2 Claude Code

本文介绍如何配置方舟 Agent Plan 到 Claude Code 中使用。

一、前提条件

- 您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。
- 您已正确安装 Claude Code，如未，请参考 [官方文档的安装章节](#) 完成软件安装。

二、配置 Claude Code

需配置的信息

如您对 Claude Code 熟悉 / 使用 cc-switch 等配置工具 / 本地已有配置 / 已有本地 Agent等，可获取下列信息直接配置/让本地 Agent 帮您配置。

- **ANTHROPIC_BASE_URL** : `https://ark.cn-beijing.volces.com/api/plan`，为专属 Base URL，其他 Base URL 无法在 Agent Plan 中使用。
- **ANTHROPIC_AUTH_TOKEN** : [获取 Agent Plan API Key](#)，为专属 API Key，其他方舟 API Key 如Coding Plan API Key 无法在 Agent Plan 中使用。



- **ANTHROPIC_MODEL** : 见 [支持模型及 Harness](#)，不同 Agent Plan 套餐支持的模型不同。请先配置文本生成模型，后续图片生成模型、视频生成模型等可在 Claude Code 中使用自然语言/Skill 等方式完成配置。

注意

配置前注意清除以下环境变量，避免与旧配置冲突：

```
Bash
```

```
unset ANTHROPIC_AUTH_TOKEN
unset ANTHROPIC_BASE_URL
```

⚠️若变量在 `~/.bashrc` / `~/.zshrc` 中，可以备份并同步删除对应变量的配置。

```
<span id="9e5412a5"></span>
```


方式 1：自动化助手（推荐）

ArkCLI Helper 是一个编码工具助手，支持快速配置选择的工具接入 Agent Plan。安装并运行该助手，根据界面提示操作可自动完成工具配置，降低手动配置的时间成本和出错风险。

```
:::warning
```

* ArkCLI Helper 是基于 Ark Helper 升级演进的工具体配置服务，相关能力将持续在 ArkCLI Helper 上迭代更新。建议优先使用 ArkCLI Helper 完成工具配置。

* ArkCLI Helper 支持 macOS、Linux、Windows 系统。

```
...
```

1. 安装 Ark CLI。

```
```Bash
```

```
1. 通过 NPM 安装
```

```
npm install -g @volcengine/ark-cli@latest
```

```
2. 验证安装
```

```
arkcli --version
```

## 2. 登录 Ark CLI。

```
Bash
```

```
arkcli auth login
```

```
bytedance@NXMPVVL4VL ~$ arkcli auth login
请选择登录方式
> 火山 SSO (浏览器登录) [推荐]
火山 SSO (无浏览器模式)
```

根据界面提示选择登录方式。首次登录时，按提示完成以下配置：

a. 选择项目（Project）：可按需选择，或默认选择 账号全部资源。

b. 选择消费模式（Type）：选择 `agent-plan`。

```
步骤 2 · 选择消费模式 (Type)
cn-beijing > 账号全部资源 > 模式
> agent-plan Agent Plan 个人版 (max) ✓ 已检测到订阅
 coding-plan Coding Plan 个人版 (pro) ✓ 已检测到订阅
 platform 控制台按量
↑ up · ↓ down · / filter · enter submit
```

## 3. 运行 ArkCLI Helper，根据界面提示完成 Claude Code 的配置。

a. 在命令行界面输入 `arkcli helper` 命令，启动 ArkCLI Helper。

b. 选择要配置的 Plan profile： `agent-plan_cn-beijing_personal` (Agent Plan)。

```
选择要配置的 Plan profile
> agent-plan_cn-beijing_personal (Agent Plan) [default]
 coding-plan_cn-beijing_personal (Coding Plan)
```

c. 选择默认 model：按需选择。

d. 选择要配置的 AI Agent：`Claude Code`。

当前支持的 Harness 包括：豆包搜索、专业数据集、Agent 记忆（OpenViking）、AI Native 应用开发底座，根据界面提示按需选择 `配置` 或者 `跳过`。

```
选择要配置的 AI Agent(每次只配置一个)
Harness 工具集 = 豆包搜索 + 专业数据集 + Agent 记忆 (OpenViking) + 火山引擎 Supabase
> Claude Code [已安装] [已配置] [模型 + Harness 工具集]
 OpenCode [已安装] [已配置] [模型 + Harness 工具集]
 OpenClaw [未安装] [未配置] [模型 + Harness 工具集]
 Hermes Agent [未安装] [未配置] [仅模型]
```

完成上述操作后，将自动完成 Claude Code 的基础配置，并按选择结果自动安装对应的 Harness 工具，无需手动安装。

说明

Claude Code 的原生搜索工具仅支持 Claude 系列模型，第三方模型（如 Doubao、GPT 等）无法直接调用。但通过 [豆包搜索](#) 可在第三方模型上同样可以正常使用搜索能力。

方式 2：手动配置

下面详细介绍如何手动完成上述信息配置。

- 1. 创建并打开 `settings.json` 配置文件。文件路径因系统而异，具体操作如下：

macOS/Linux

macOS/Linux 系统 Claude Code 配置文件路径：`~/claude/settings.json`。

- 1. 如果在主目录下没有 `~/.claude` 目录，执行以下命令创建目录。

Bash

```
mkdir -p ~/.claude
```

- 2. 创建并打开配置文件。

Bash

```
nano ~/.claude/settings.json
```

Windows

Windows 系统 Claude Code 配置文件路径：`C:\Users\<用户名>\.claude\settings.json`。以 CMD 命令行方式为例，操作如下。

1. 如果当前用户目录下没有 `.claude` 目录，执行以下命令创建目录。

Bash

```
if not exist "%USERPROFILE%\.claude" mkdir "%USERPROFILE%\.claude"
```

2. 创建并打开配置文件。

Bash

```
notepad "%USERPROFILE%\.claude\settings.json"
```

2. 编辑 `settings.json`，需修改的配置信息如下：

- `<ARK_API_KEY>`：替换为您的 [ARK Agent Plan API Key](#)
- `<MODEL_NAME>`：结合套餐及您的需求，更新 **文本生成模型** 的名称，如 `ark-code-latest`，详细模型名称参见 [支持模型及 Harness](#)。

JSON

```
{
 "env": {
 "ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>",
 "ANTHROPIC_BASE_URL": "https://ark.cn-beijing.volces.com/api/plan",
 "ANTHROPIC_MODEL": "<MODEL_NAME>",
 "ANTHROPIC_DEFAULT_HAIKU_MODEL": "<MODEL_NAME>",
 "ANTHROPIC_DEFAULT_SONNET_MODEL": "<MODEL_NAME>",
 "ANTHROPIC_DEFAULT_OPUS_MODEL": "<MODEL_NAME>",
 "CLAUDE_CODE_SUBAGENT_MODEL": "<MODEL_NAME>"
 }
}
```

#### 说明

- 要开启 1M 上下文窗口，配置参见[扩展上下文](#)。
- 推荐使用完整模型配置，并按任务复杂度选择模型：Haiku（轻量）、Sonnet（日常）、Opus（复杂）。

- `CLAUDE_CODE_SUBAGENT_MODEL` 建议与主模型保持一致。
- `ANTHROPIC_DEFAULT_HAIKU_MODEL` 建议设置为小尺寸模型，通常不会影响整体使用效果。

3. 编辑或新增 `.claude.json` 文件，修改或新增 `hasCompletedOnboarding` 字段值为 `true`，设置状态为完成入门引导。

### macOS/Linux

a. 创建并打开配置文件。

Bash

```
nano ~/.claude.json
```

ii. 修改或新增的字段信息如下：

JSON

```
{
 "hasCompletedOnboarding": true
}
```

### Windows

a. 创建并打开配置文件。

Bash

```
notepad "%USERPROFILE%\claude.json"
```

ii. 修改或新增的字段信息如下：

JSON

```
{
 "hasCompletedOnboarding": true
}
```

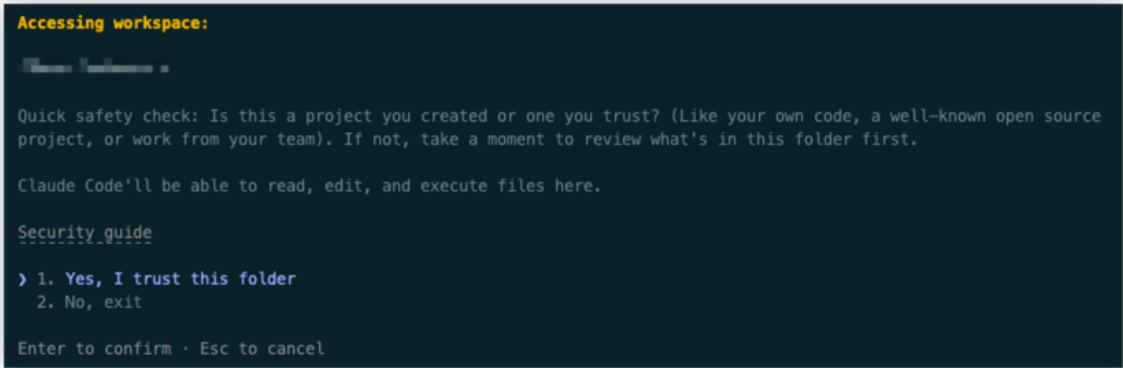
## 三、启动并使用 Claude Code

1. 启动 Claude Code：进入项目目录，执行 `claude` 命令，即可开始使用 Claude Code。

Bash

```
cd my-project
claude
```

启动后，选择信任此文件夹，允许 Claude Code 访问该文件夹中的文件。



- 2. 模型状态验证：输入 `/status` 确认模型状态。
- 3. 在 Claude Code 中对话。

更多使用教程请参见 [Claude Code 官网](#)。

## 其他操作

### 添加图片生成/视频生成能力

方舟 Agent Plan Medium 及以上套餐提供图片生成、视频生成模型配额。方舟提供 Skill 帮助 AI 工具快速接入并使用。具体配置方法请参见：[配置视觉模型](#)。

- 注意
- 不同的套餐支持的视觉模型有所不同，具体参见[支持模型及 Harness](#)。
  - 视觉模型不支持通过 Auto 及控制台切换使用。

### 切换模型

你可以随时根据需求切换 **文本生成模型**。方舟支持多种方式切换模型。

配置方式	通过控制台	通过命令参数	修改配置文件
切换模型操作	1. 参考<a href="#">二、配置 Claude Code</a>配置模型名称为 <code>ark-code-latest</code>。   2. 在<a href="#">开通管理页面</a> 选择要使用的模型，切换后3-5分钟即可生效。   3. 在命令行窗口，启动 Claude Code	· 启动时：执行 <code>claude --model &lt;MODEL_NAME&gt;</code>，可指定本次对话使用的文本生成模型。   · 对话期间：执行 <code>/model &lt;MODEL_NAME&gt;</code> 切换模型。	参考<a href="#">二、配置 Claude Code</a>配置模型名称为您需要的模型。

配置方式	通过控制台	通过命令参数	修改配置文件
	后，输入 <code>/status</code> 确认模型状态。		

### 扩展上下文

`glm-5.2`、`deepseek-v4-flash`、`deepseek-v4-pro` 支持 1M 上下文窗口，可用于包含大型代码库的长会话。以 `glm-5.2` 模型为例，配置信息如下：

```
JSON
{
 "env": {
 "ANTHROPIC_AUTH_TOKEN": "<ARK_API_KEY>",
 "ANTHROPIC_BASE_URL": "https://ark.cn-beijing.volces.com/api/plan",
 "CLAUDE_CODE_AUTO_COMPACT_WINDOW": "1000000",
 "ANTHROPIC_MODEL": "glm-5.2[1m]",
 "ANTHROPIC_DEFAULT_HAIKU_MODEL": "glm-5.2[1m]",
 "ANTHROPIC_DEFAULT_SONNET_MODEL": "glm-5.2[1m]",
 "ANTHROPIC_DEFAULT_OPUS_MODEL": "glm-5.2[1m]"
 }
}
```

- 注意
- 开启 1M 上下文时，需要在模型名称后增加 `[1m]` 后缀，如 `glm-5.2[1m]`，同时配置压缩窗口大小参数 `"CLAUDE_CODE_AUTO_COMPACT_WINDOW": "1000000"`。
  - 若模型参数加上 `[1m]` 后缀后，Claude Code 识别模型不存在，需升级 Claude Code 到最新版本后重试。

### 使用 CC Switch

CC Switch 是一款跨平台桌面应用，专为使用 AI 编程工具的开发者设计。它帮助你统一管理 Claude Code、Claude Desktop、Codex、Gemini CLI、OpenCode、OpenClaw 和 Hermes 等受管应用的配置。

### 支持的平台

- Windows 10 及以上
- macOS 12 (Monterey) 及以上
- Linux Ubuntu 22.04+ / Debian 11+ / Fedora 34+ ( x64 / ARM64 )

### 安装

更多安装方式和详细说明请参考 [CC Switch 官方文档](#)。

## macOS

- Homebrew（推荐）：

Bash

```
brew tap farion1231/ccswitch
brew install --cask cc-switch
```

- 手动下载：从 [Releases 页面](#) 下载 `.dmg`。

## Linux

- Arch 发行版：

Bash

```
paru -S cc-switch-bin
```

- 其他发行版：根据架构（Debian / Ubuntu）从 [Releases 页面](#) 下载 `.deb` / `.rpm`，或者下载 `.ApplImage`（通用）。

## Windows

- 从 [Releases 页面](#) 下载 `.msi` 安装包或 `.zip` 绿色版。

## 添加供应商

1. 在 CC Switch 中添加供应商：
  - a. 在主界面顶部图标栏选中 Claude Code 图标，点击右上角 + 进入添加新供应商。
  - b. 选择 **火山 Agentplan** 供应商，填写以下信息：
    - **API Key**：替换为您的 [ARK Agent Plan API Key](#)
    - **请求地址**：已自动填写为 `https://ark.cn-beijing.volces.com/api/plan`
  - c. 展开高级选项，根据当前套餐支持的模型范围，分别配置 Sonnet、Opus、Fable、Haiku 模型，可配置的模型见[支持模型及 Harness](#)。

### 注意

建议 Haiku 设置为小尺寸模型。

- d. 选择右下角 **添加**，完成供应商配置。
2. 回到首页，点击右侧 **启用** 按钮，新开一个 Claude Code 会话使配置生效。

## 安装使用 Claude Code IDE 插件

1. 安装 Claude Code 并配置好环境变量，具体参考[Claude Code](#)。  
Claude Code IDE 插件依赖 Claude Code CLI 工具，需先完成 Claude Code 的安装及配置。
2. 安装并使用 IDE 插件。

因 IDE 插件会迭代演进，以下内容仅供参考，具体的安装及使用可参考 [Visual Studio Code](#)、[JetBrains IDEs](#)。

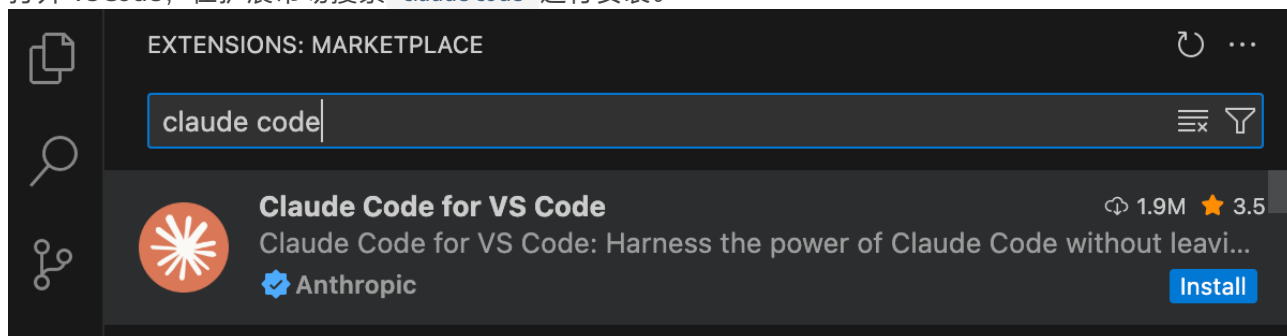
## Claude Code VSCode 插件

### 说明

Claude Code VSCode 插件支持在 VSCode 及基于 VSCode 的 IDE（如 Cursor、Trae 等）中使用。

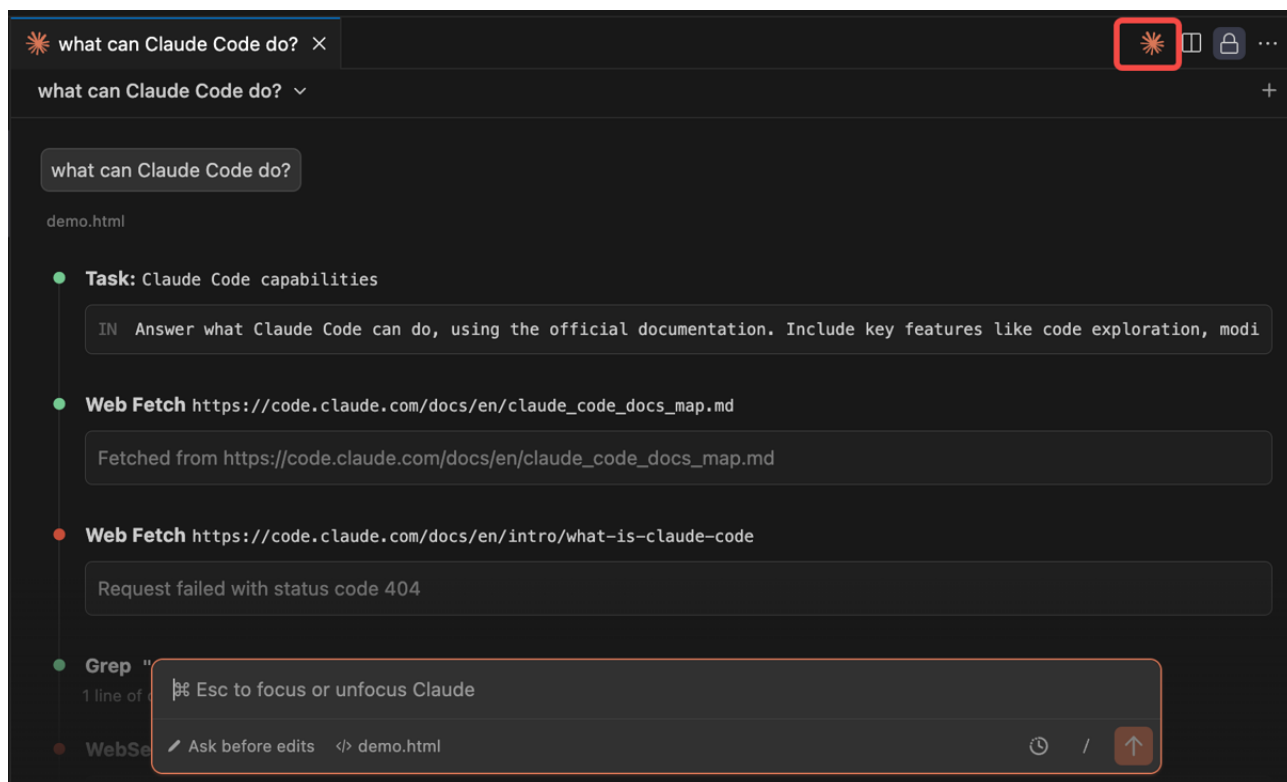
## 安装插件

打开 VSCode，在扩展市场搜索 `claude code` 进行安装。



## 配置环境变量

1. 安装完成后，重启 VSCode，点击右上角的 Claude Code 图标，进入 Claude Code 页面。



2. 在对话框中输入 `/config` 回车进入 settings 配置页面。



### 3. 在 **Claude Code: Environment Variables** 区域单击 **Edit in settings.json**，修改

`claudeCode.environmentVariables`、`claudeCode.selectedModel`。

#### 说明

需要修改的配置信息如下：

- `<ARK_API_KEY>`：替换为您自己的 [Agent Plan API Key](#) 
- `<Model_Name>`：更新为需要使用的模型信息，如 `ark-code-latest`。支持的模型信息参见[支持模型及Harness中文本生成模型](#)。

#### JSON

```
...
"claudeCode.environmentVariables": [
 {
 "name": "ANTHROPIC_BASE_URL",
 "value": "https://ark.cn-beijing.volces.com/api/plan"
 },
 {
 "name": "ANTHROPIC_AUTH_TOKEN",
 "value": "<ARK_API_KEY>"
 },
 {
 "name": "ANTHROPIC_MODEL",
 "value": "<MODEL_NAME>"
 }
],
"claudeCode.selectedModel": "<MODEL_NAME>",
...
```

保存配置后，新开一个窗口开始使用 Claude Code。

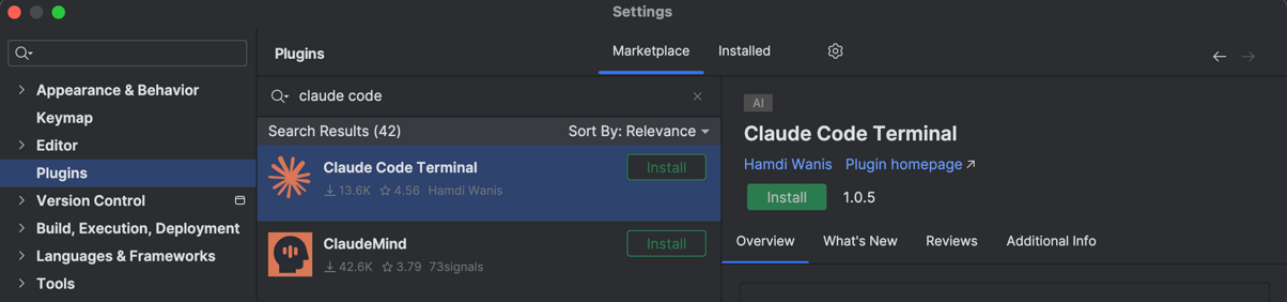
## Claude Code JetBrains 插件

#### 说明

Claude Code JetBrains 插件支持 JetBrains 的系列 IDE 如 IntelliJ IDEA、PyCharm、WebStorm 等。

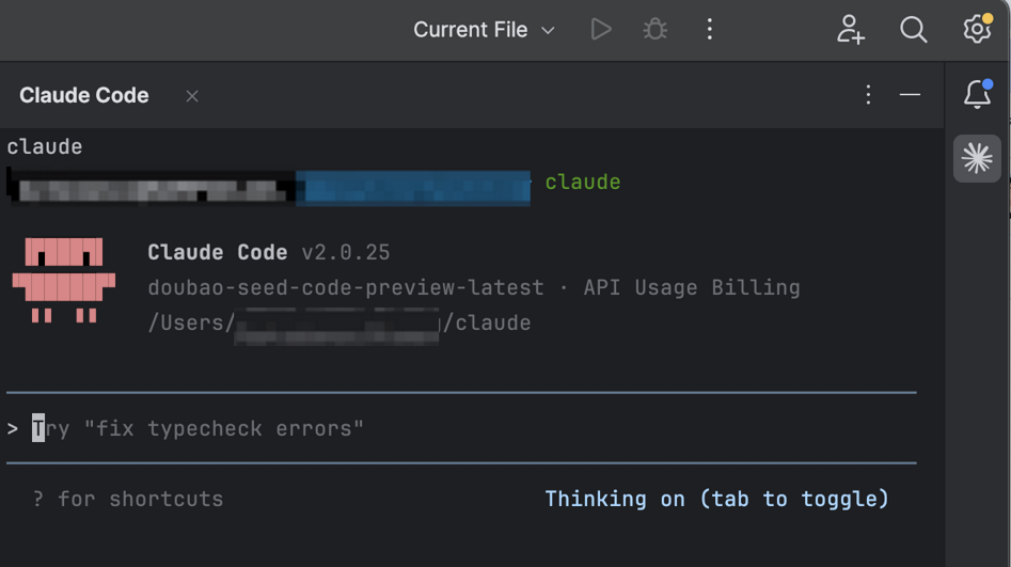
## 安装插件

打开 JetBrains IDE，在插件市场搜索 `claude code` 进行安装。



开始使用

安装完成后，重启IDE后，单击Claude Code 图标，进入 Claude Code 页面开始使用。



## 4.3 OpenCode

本文介绍如何配置方舟 Agent Plan 到 OpenCode 中使用。

### 一、前提条件

- 您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。
- 您已正确安装 OpenCode，如未，请参见以下操作完成软件安装。  
前提条件：安装 [Node.js 18 或更新版本](#)。  
在命令行界面，执行以下命令安装 OpenCode。

Bash

```
npm install -g opencode-ai
```

安装结束后，执行以下命令查看安装结果，若显示版本号则安装成功。

Bash

```
opencode --version
```

### 二、配置 OpenCode

#### 方式 1：自动化助手（推荐）

ArkCLI Helper 是一个编码工具助手，支持快速配置选择的工具接入 Agent Plan。安装并运行该助手，根据界面提示操作可自动完成工具配置，降低手动配置的时间成本和出错风险。

#### 注意

- ArkCLI Helper 是基于 Ark Helper 升级演进的工具配置服务，相关能力将持续在 ArkCLI Helper 上迭代更新。建议优先使用 ArkCLI Helper 完成工具配置。
- ArkCLI Helper 支持 macOS、Linux、Windows 系统。

#### 1. 安装 Ark CLI。

Bash

```
1. 通过 NPM 安装
npm install -g @volcengine/ark-cli@latest

2. 验证安装
arkcli --version
```

#### 2. 登录 Ark CLI。

Bash

```
arkcli auth login
```

```
bytedance@NXMPVVL4VL ~$ arkcli auth login
请选择登录方式
> 火山 SSO (浏览器登录) [推荐]
火山 SSO (无浏览器模式)
```

根据界面提示选择登录方式。首次登录时，按提示完成以下配置：

- 选择项目（Project）：可按需选择，或默认选择 账号全部资源。
- 选择消费模式（Type）：选择 `agent-plan`。

```
步骤 2 · 选择消费模式 (Type)
cn-beijing > 账号全部资源 > 模式
> agent-plan Agent Plan 个人版 (max) ✓ 已检测到订阅
coding-plan Coding Plan 个人版 (pro) ✓ 已检测到订阅
platform 控制台按量

↑ up · ↓ down · / filter · enter submit
```

3. 运行 ArkCLI Helper，根据界面提示完成 OpenCode 的配置。

- 在命令行界面输入 `arkcli helper` 命令，启动 ArkCLI Helper。
- 选择要配置的 Plan profile： `agent-plan_cn-beijing_personal` (Agent Plan)。

```
选择要配置的 Plan profile
> agent-plan_cn-beijing_personal (Agent Plan) [default]
coding-plan_cn-beijing_personal (Coding Plan)
```

- 选择默认 model：按需选择。
- 选择要配置的 AI Agent： `OpenCode`。

当前支持的 Harness 包括：豆包搜索、专业数据集、Agent 记忆（OpenViking）、AI Native 应用开发底座，根据界面提示按需选择 配置 或者 跳过。

```
选择要配置的 AI Agent (每次只配置一个)
Harness 工具集 = 豆包搜索 + 专业数据集 + Agent 记忆 (OpenViking) + 火山引擎 Supabase
> OpenCode [已安装] [已配置] [模型 + Harness 工具集]
OpenClaw [未安装] [未配置] [模型 + Harness 工具集]
Hermes Agent [未安装] [未配置] [仅模型]
Trae [已安装] [已配置] [仅 Harness 工具集]
```

完成上述操作后，将自动完成 OpenCode 的基础配置，并按选择结果自动安装对应的 Harness 工具，无需手动安装。

## 方式 2：手动配置

1. 创建并打开 OpenCode 的配置文件。文件路径因系统而异，具体操作如下：

macOS/Linux

macOS/Linux 系统 OpenCode 配置文件路径：`~/config/opencode/opencode.json`。

1. 如果在主目录下没有`.config/opencode`目录，执行以下命令创建目录。

Bash

```
mkdir -p ~/.config/opencode
```

2. 创建并打开配置文件。

Bash

```
nano ~/.config/opencode/opencode.json
```

## Windows

Windows 系统 OpenCode 配置文件路径：`%USERPROFILE%\config\opencode\opencode.json`。以 CMD 命令行方式为例，操作如下。

1. 如果当前用户目录下没有`.config\opencode`目录，执行以下命令创建目录。

Bash

```
if not exist "%USERPROFILE%\config\opencode" mkdir "%USERPROFILE%\config\opencode"
```

2. 创建并打开配置文件。

Bash

```
notepad "%USERPROFILE%\config\opencode\opencode.json"
```

2. 编辑 `opencode.json` ，需关注的配置信息如下：

- `<ARK_API_KEY>` ：替换为您自己的**专属 API Key** [🔗](#)。

开通管理

模型 | 应用组件库 | 云产品 | 高级创作权益包 | **Agent Plan** | Coding Plan

管理素材资产

套餐信息

套餐类型  
套餐限额  
订阅状态  
开始时间  
结束时间  
剩余时间

Small 套餐

2000 AFP/5小时  
7000 AFP/周  
20000 AFP/月  
计算规则详见 [说明文档](#)

生效中

2026-05-07 20:43  
2026-06-07 23:59  
31天

使用配置 | 用量统计

配置模型及Base URL | 配置Harness (可选) | **配置专属API Key** | 接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key	创建时间	操作
ark-ef	2026-05-07 20:43	更新API KEY

- `model` ：见[支持模型及 Harness](#)，需配置文本生成模型。  
Agent Plan 兼容 OpenAI 接口协议，支持 Responses API、Chat API。推荐使用 Responses API 的配置，推理效果更好。  
支持的两类 API 仅在 `npm` 配置上不同：
- Responses API 配置： `"npm": "@ai-sdk/openai"`
- Chat API 配置： `"npm": "@ai-sdk/openai-compatible"`  
以配置 `ark-code-latest` 为例，具体信息如下。

注意

- `glm-5.2` 、 `deepseek-v4-flash` 、 `deepseek-v4-pro` 支持 1M 上下文窗口用于包含大型代码库的长会话，可以通过 `limit.context` 字段显式指定窗口大小。
- `provider.volcengine-agent-plan.models` 节点下有 **两处**（对象键、`name` 字段）需 **替换为同一 Model Name**，切勿遗漏。
- 若需开启模型图片理解能力，需在模型配置节点下新增如下配置项：`"modalities":{"input":["text","image"],"output":["text"]}`

Responses API

说明

`npm` 必须使用 `@ai-sdk/openai` ，模型才会使用 Responses API，在 `baseUrl` 的基础上自动拼接 `/responses` 。

```
{
 "$schema": "https://opencode.ai/config.json",
 "model": "volcengine-agent-plan/ark-code-latest",
 "provider": {
 "volcengine-agent-plan": {
 "npm": "@ai-sdk/openai",

```

版权所有©北京火山引擎科技有限公司

53/215

```
"name": "Volcano Engine (Responses API) ",
"options": {
 "baseURL": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>"
},
"models": {
 "ark-code-latest": {
 "name": "ark-code-latest",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
 },
 "glm-5.2": {
 "name": "glm-5.2",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
 },
 "glm-latest": {
 "name": "glm-latest",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
```

```
 "output": [
 "text"
]
 },
 "deepseek-v4-flash": {
 "name": "deepseek-v4-flash",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
 },
 "deepseek-v4-pro": {
 "name": "deepseek-v4-pro",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
 },
 "doubao-seed-2.0-code": {
 "name": "doubao-seed-2.0-code",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
```



```
 "text"
]
 }
},
"doubao-seed-2.0-pro": {
 "name": "doubao-seed-2.0-pro",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
"doubao-seed-2.0-lite": {
 "name": "doubao-seed-2.0-lite",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
"doubao-seed-2.0-mini": {
 "name": "doubao-seed-2.0-mini",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
```

```
 "output": [
 "text"
]
 },
 "minimax-m2.7": {
 "name": "minimax-m2.7",
 "limit": {
 "context": 200000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
 },
 "minimax-m3": {
 "name": "minimax-m3",
 "limit": {
 "context": 512000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
 },
 "kimi-k2.6": {
 "name": "kimi-k2.6",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
```

```
JSON "output": [
 "text"
]
 },
 "kimi-k2.7-code": {
 "name": "kimi-k2.7-code",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
 }
}
```

## Chat API

### 说明

npm 必须使用 `@ai-sdk/openai-compatible`，模型才会使用 Chat API，在 `baseUrl` 的基础上自动拼接 `/chat/completions`。

```
{
 "$schema": "https://opencode.ai/config.json",
 "model": "volcengine-agent-plan/ark-code-latest",
 "provider": {
 "volcengine-agent-plan": {
 "npm": "@ai-sdk/openai-compatible",
 "name": "Volcano Engine (Chat API)",
 "options": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>"
 }
 }
 }
}
```

```
"models": {
 "ark-code-latest": {
 "name": "ark-code-latest",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
 },
 "glm-5.2": {
 "name": "glm-5.2",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
 },
 "glm-latest": {
 "name": "glm-latest",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
 },
}
```

```
"deepseek-v4-flash": {
 "name": "deepseek-v4-flash",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
},
"deepseek-v4-pro": {
 "name": "deepseek-v4-pro",
 "limit": {
 "context": 1024000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
},
"doubao-seed-2.0-code": {
 "name": "doubao-seed-2.0-code",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
"doubao-seed-2.0-pro": {
```

```
"name": "doubao-seed-2.0-pro",
"limit": {
 "context": 256000,
 "output": 4096
},
"modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
},
"doubao-seed-2.0-lite": {
 "name": "doubao-seed-2.0-lite",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
"doubao-seed-2.0-mini": {
 "name": "doubao-seed-2.0-mini",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
```

```
"minimax-m2.7": {
 "name": "minimax-m2.7",
 "limit": {
 "context": 200000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text"
],
 "output": [
 "text"
]
 }
},
"minimax-m3": {
 "name": "minimax-m3",
 "limit": {
 "context": 512000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
"kimi-k2.6": {
 "name": "kimi-k2.6",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
},
```

```
JSON {"kimi-k2.7-code": {
 "name": "kimi-k2.7-code",
 "limit": {
 "context": 256000,
 "output": 4096
 },
 "modalities": {
 "input": [
 "text",
 "image"
],
 "output": [
 "text"
]
 }
}
}
```

### 三、启动并使用 OpenCode

#### 1. 启动OpenCode:

Bash

```
opencode
```

#### 2. 输入 `/models`，选择配置的模型并在 OpenCode 中使用。

### 其他操作

#### 添加图片生成/视频生成能力

方舟 Agent Plan Medium 及以上套餐提供图片生成、视频生成模型配额。方舟提供 Skill 帮助 AI 工具快速接入并使用。具体配置方法请参见：[配置视觉模型](#)。

##### 注意

- 不同的套餐支持的视觉模型有所不同，具体参见[支持模型及 Harness](#)。
- 视觉模型不支持通过 Auto 及控制台切换使用。



## 切换模型

你可以随时根据需求切换 **文本生成模型** 。方舟支持多种方式切换模型。

配置方式	通过控制台	通过命令参数	修改配置文件
切换模型操作	1. 参考 <a href="#">二、配置 OpenCode</a>配置模型名称为 <code>ark-code-latest</code> 。   2. 在<a href="#">开通管理页面</a> <a href="#">↗</a> 选择要使用的模型，切换后3-5分钟即可生效。	启动 OpenCode后，执行 <code>/models</code> 切换模型。	参考 <a href="#">二、配置 OpenCode</a>配置模型名称为您需要的模型。

## 4.4 OpenClaw

本文介绍如何配置方舟 Agent Plan 到 OpenClaw 中使用。

### 一、前提条件

· 您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。

### 二、安装 OpenClaw

前提条件：安装 [Node.js 22 或更新版本](#)。

1. 执行以下命令安装 OpenClaw。

macOS / Linux

Bash

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Windows

Windows PowerShell 环境下安装命令如下：

Bash

```
iwr -useb https://openclaw.ai/install.ps1 | iex
```

2. 根据提示信息完成 OpenClaw 配置，配置信息如下。

**注意**

OpenClaw 在不断迭代，如果实际使用与以下配置存在差异，可以选择 "Skip" 或默认选项完成配置流程，后续可以参考文档配置并使用 Agent Plan。

提示信息	配置
I understand this is personal-by-default and shared/multi-user use requires lock-down. Continue?	选择 "Yes"
Setup mode	选择 “QuickStart”
Model/auth provider	选择 "Skip for now"，后续可以配置。
Default model	选择 "Keep current"
Select channel (QuickStart)	选择 “Skip for now” ，后续可以配置。

提示信息	配置
Search provider	选择 “Skip for now”
Configure skills now? (recommended)	选择 “No” ，后续可以配置。
Enable hooks?	选择 "Skip for now"。选择方式：按空格键选中选项，按回车键进入下一步。
How do you want to hatch your bot?	选择 "Hatch in Terminal"。

### 三、配置 OpenClaw

#### 方式 1：自动化助手（推荐）

ArkCLI Helper 是一个编码工具助手，支持快速配置选择的工具接入 Agent Plan。安装并运行该助手，根据界面提示操作可自动完成工具配置，降低手动配置的时间成本和出错风险。

- 注意
- ArkCLI Helper 是基于 Ark Helper 升级演进的工具配置服务，相关能力将持续在 ArkCLI Helper 上迭代更新。建议优先使用 ArkCLI Helper 完成工具配置。
  - ArkCLI Helper 支持 macOS、Linux、Windows 系统。

1. 安装 Ark CLI。

Bash

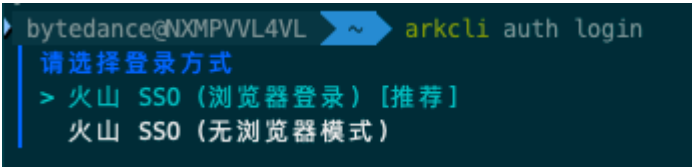
```
1. 通过 NPM 安装
npm install -g @volcengine/ark-cli@latest

2. 验证安装
arkcli --version
```

2. 登录 Ark CLI。

Bash

```
arkcli auth login
```



根据界面提示选择登录方式。首次登录时，按提示完成以下配置：

a. 选择项目（Project）：可按需选择，或默认选择 账号全部资源。

- b. 选择消费模式 (Type) : 选择 `agent-plan` 。

```
步骤 2 · 选择消费模式 (Type)
cn-beijing > 账号全部资源 > 模式
> agent-plan Agent Plan 个人版 (max) ✓ 已检测到订阅
 coding-plan Coding Plan 个人版 (pro) ✓ 已检测到订阅
 platform 控制台按量

↑ up · ↓ down · / filter · enter submit
```

3. 运行 ArkCLI Helper, 根据界面提示完成 OpenClaw 的配置。

- a. 在命令行界面输入 `arkcli helper` 命令, 启动 ArkCLI Helper。
- b. 选择要配置的 Plan profile : `agent-plan_cn-beijing_personal (Agent Plan)` 。
- c. 选择默认 model : 按需选择。
- d. 选择要配置的 AI Agent : `OpenClaw` 。

当前支持的 Harness 包括: 豆包搜索、专业数据集、Agent 记忆 (OpenViking)、AI Native 应用开发底座, 根据界面提示按需选择 `配置` 或者 `跳过` 。

```
选择要配置的 AI Agent (每次只配置一个)
Harness 工具集 = 豆包搜索 + 专业数据集 + Agent 记忆 (OpenViking) + 火山引擎 Supabase
> OpenClaw [未安装] [未配置] [模型 + Harness 工具集]
 Hermes Agent [未安装] [未配置] [仅模型]
 Trae [已安装] [已配置] [仅 Harness 工具集]
退出
```

完成上述操作后, 将自动完成 OpenClaw 的基础配置, 并按选择结果自动安装对应的 Harness 工具, 无需手动安装。

## 方式 2: 手动配置

Agent Plan 兼容 OpenAI 接口协议, 支持 Responses API、Chat API。推荐使用 Responses API 的配置, 推理效果更好。

两类 API 的配置方式基本一致, 只需在 `models.providers` 中指定对应的 API 适配器:

- Responses API 配置: `"api": "openai-responses"`
- Chat API 配置: `"api": "openai-completions"`

### 注意

`glm-5.2`、`deepseek-v4-flash`、`deepseek-v4-pro` 支持 1M 上下文窗口用于包含大型代码库的长会话, 可以通过 `contextWindow` 字段显式指定窗口大小。

## Web UI 方式

1. 执行以下命令打开 Web UI。

```
Bash
```

```
openclaw dashboard
```

2. 在左侧菜单栏选择配置 - **Settings - Advanced**, 单击 **Open** 查看并修改配置信息。具体配置信息如下, 其中需要将 `<ARK_API_KEY>` 替换为您自己的 [专属 API Key](#)。

注意

如果已经配置过 OpenClaw，请勿直接覆盖原有配置，建议根据提供的配置更新 `models`、`agents` 和 `gateway` 节点信息。其中 `models` 节点包含支持的模型列表。

开通管理

模型 | 应用组件库 | 云产品 | 高级创作权益包 | **Agent Plan** | Coding Plan

管理素材资产

套餐信息

套餐类型  
套餐限额  
订阅状态  
开始时间  
结束时间  
剩余时间

Small 套餐

2000 AFP/5小时  
7000 AFP/周  
20000 AFP/月  
计算规则详见 [说明文档](#)

生效中

2026-05-07 20:43  
2026-06-07 23:59  
31天

2 使用配置 | 用量统计

1 配置模型及Base URL | 2 配置Harness (可选) | **3 配置专属API Key** | 4 接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key	创建时间	操作
ark-ef	2026-05-07 20:43	更新API KEY

以下是 Responses API 的配置示例。

```
{
 "models": {
 "providers": {
 "volcengine-agent-plan": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>",
 "api": "openai-responses",
 "models": [
 {
 "id": "ark-code-latest",
 "name": "ark-code-latest",
 "contextWindow": 256000,
 "maxTokens": 32000,
 "input": [
 "text",
 "image"
]
 },
 {
 "id": "glm-5.2",
 "name": "glm-5.2",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
 }
]
 }
 }
 }
}
```

```
]
},
{
 "id": "glm-latest",
 "name": "glm-latest",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
},
{
 "id": "deepseek-v4-flash",
 "name": "deepseek-v4-flash",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
},
{
 "id": "deepseek-v4-pro",
 "name": "deepseek-v4-pro",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
},
{
 "id": "doubao-seed-2.0-code",
 "name": "doubao-seed-2.0-code",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "doubao-seed-2.0-pro",
 "name": "doubao-seed-2.0-pro",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
}
```

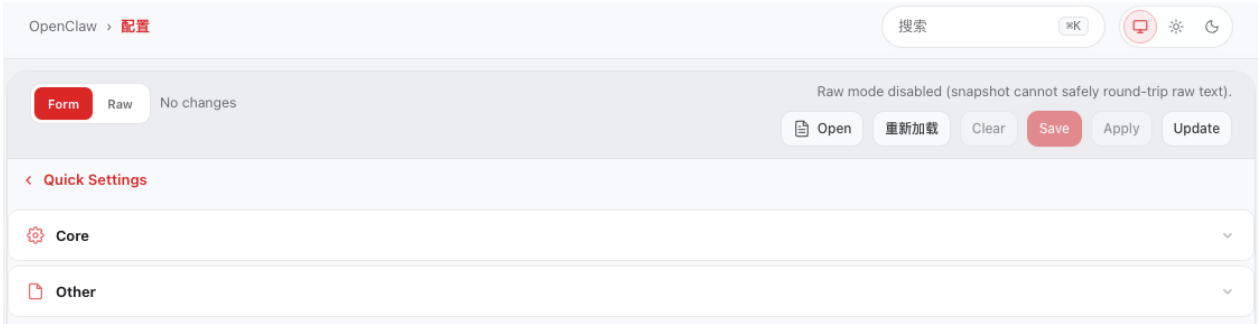
```
]
},
{
 "id": "doubao-seed-2.0-lite",
 "name": "doubao-seed-2.0-lite",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "doubao-seed-2.0-mini",
 "name": "doubao-seed-2.0-mini",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "minimax-m2.7",
 "name": "minimax-m2.7",
 "contextWindow": 200000,
 "maxTokens": 65536,
 "input": [
 "text"
]
},
{
 "id": "minimax-m3",
 "name": "minimax-m3",
 "contextWindow": 512000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "kimi-k2.6",
 "name": "kimi-k2.6",
 "contextWindow": 256000,
 "maxTokens": 32000,
 "input": [
```

```
 "text",
 "image"
]
},
{
 "id": "kimi-k2.7-code",
 "name": "kimi-k2.7-code",
 "contextWindow": 256000,
 "maxTokens": 32000,
 "input": [
 "text",
 "image"
]
}
]
}
}
},
"agents": {
 "defaults": {
 "model": {
 "primary": "volcengine-agent-plan/ark-code-latest"
 },
 },
 "models": {
 "volcengine-agent-plan/ark-code-latest": {},
 "volcengine-agent-plan/doubao-seed-2.0-code": {},
 "volcengine-agent-plan/doubao-seed-2.0-pro": {},
 "volcengine-agent-plan/doubao-seed-2.0-lite": {},
 "volcengine-agent-plan/doubao-seed-2.0-mini": {},
 "volcengine-agent-plan/glm-5.2": {},
 "volcengine-agent-plan/glm-latest": {},
 "volcengine-agent-plan/minimax-m2.7": {},
 "volcengine-agent-plan/minimax-m3": {},
 "volcengine-agent-plan/kimi-k2.6": {},
 "volcengine-agent-plan/kimi-k2.7-code": {},
 "volcengine-agent-plan/deepseek-v4-flash": {},
 "volcengine-agent-plan/deepseek-v4-pro": {}
 }
}
},
"gateway": {
 "mode": "local"
```



```
JSON
}
```

3. 配置完成后，先保存配置文件，然后单击 **Update** 更新配置，配置更新完成后需要重新连接服务进入 Web UI。



终端方式

1. 在终端执行以下命令打开 OpenClaw 配置文件。

```
Bash
nano ~/.openclaw/openclaw.json
```

2. 修改配置信息，完成 Agent Plan 对接。将 `<ARK_API_KEY>` 替换为您自己的 [专属 API Key](#)。

注意

如果已经配置过 OpenClaw，请勿直接覆盖原有配置，建议根据提供的配置更新 `models`、`agents` 和 `gateway` 节点信息。其中 `models` 节点包含支持的模型列表。

开通管理

模型 | 应用组件库 | 云产品 | 高级创作权益包 | **Agent Plan** | Coding Plan | 管理素材资产

套餐信息

套餐类型: Small 套餐

套餐限额: 2000 AFP/5小时, 7000 AFP/周, 20000 AFP/月, 计算规则详见 说明文档

订阅状态: 生效中

开始时间: 2026-05-07 20:43

结束时间: 2026-06-07 23:59

剩余时间: 31天

2 使用配置 | 用量统计

配置文档

1 配置模型及Base URL | 2 配置Harness (可选) | **3 配置专属API Key** | 4 接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key	创建时间	操作
ark-ef	2026-05-07 20:43	4  更新API KEY

以下是 Responses API 的配置示例。

```
{
 "models": {
 "providers": {
 "volcengine-agent-plan": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>",
 "api": "openai-responses",
 "models": [
 {
 "id": "ark-code-latest",
 "name": "ark-code-latest",
 "contextWindow": 256000,
 "maxTokens": 32000,
 "input": [
 "text",
 "image"
]
 },
 {
 "id": "glm-5.2",
 "name": "glm-5.2",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
 },
 {
 "id": "glm-latest",
 "name": "glm-latest",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
 },
 {
 "id": "deepseek-v4-flash",
 "name": "deepseek-v4-flash",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
 }
]
 }
 }
 }
}
```

```
{
 "id": "deepseek-v4-pro",
 "name": "deepseek-v4-pro",
 "contextWindow": 1024000,
 "maxTokens": 65536,
 "input": [
 "text"
]
},
{
 "id": "doubao-seed-2.0-code",
 "name": "doubao-seed-2.0-code",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "doubao-seed-2.0-pro",
 "name": "doubao-seed-2.0-pro",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "doubao-seed-2.0-lite",
 "name": "doubao-seed-2.0-lite",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "doubao-seed-2.0-mini",
 "name": "doubao-seed-2.0-mini",
 "contextWindow": 256000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
}
```

```
]
},
{
 "id": "minimax-m2.7",
 "name": "minimax-m2.7",
 "contextWindow": 200000,
 "maxTokens": 65536,
 "input": [
 "text"
]
},
{
 "id": "minimax-m3",
 "name": "minimax-m3",
 "contextWindow": 512000,
 "maxTokens": 65536,
 "input": [
 "text",
 "image"
]
},
{
 "id": "kimi-k2.6",
 "name": "kimi-k2.6",
 "contextWindow": 256000,
 "maxTokens": 32000,
 "input": [
 "text",
 "image"
]
},
{
 "id": "kimi-k2.7-code",
 "name": "kimi-k2.7-code",
 "contextWindow": 256000,
 "maxTokens": 32000,
 "input": [
 "text",
 "image"
]
}
}
}
},
"agents": {
 "defaults": {
```

```
JSON {
 "model": {
 "primary": "volcengine-agent-plan/ark-code-latest"
 },
 "models": {
 "volcengine-agent-plan/ark-code-latest": {},
 "volcengine-agent-plan/doubao-seed-2.0-code": {},
 "volcengine-agent-plan/doubao-seed-2.0-pro": {},
 "volcengine-agent-plan/doubao-seed-2.0-lite": {},
 "volcengine-agent-plan/doubao-seed-2.0-mini": {},
 "volcengine-agent-plan/glm-5.2": {},
 "volcengine-agent-plan/glm-latest": {},
 "volcengine-agent-plan/minimax-m2.7": {},
 "volcengine-agent-plan/minimax-m3": {},
 "volcengine-agent-plan/kimi-k2.6": {},
 "volcengine-agent-plan/kimi-k2.7-code": {},
 "volcengine-agent-plan/deepseek-v4-flash": {},
 "volcengine-agent-plan/deepseek-v4-pro": {}
 }
},
 "gateway": {
 "mode": "local"
 }
}
```

3. 配置完成后保存文件，并在终端执行以下命令重启服务使更改生效。

Bash

```
openclaw gateway restart
```

## 四、使用 OpenClaw

· 打开 Web UI，在 Chat 页面进行交互。

Bash

```
openclaw dashboard
```

· 打开 TUI，并查看 Gateway 状态。

Bash

```
openclaw tui
/status
```

## 其他操作

### 配置向量模型

方舟 Agent Plan 提供的 `doubao-embedding-vision` 模型（向量模型）可以为 OpenClaw 提供语义向量能力，将对话、文档与任务信息转换为高维向量表示，实现基于语义相似度的高效检索。相比传统关键词匹配，Embedding 检索在信息量大、对话长、资料分散的场景下，能够更准确地召回相关内容，显著降低遗漏与误判。同时，Embedding 为记忆检索、知识库问答与工具调用提供统一的语义索引层，帮助 OpenClaw 在不增加主模型推理成本的前提下，提升跨轮次上下文追踪能力与整体效果。选择以下任一方式配置向量模型。

### Web UI 方式

1. 执行以下命令打开 Web UI。

```
Bash
```

```
openclaw dashboard
```

2. 在左侧菜单栏选择配置 – **Settings – Advanced**，单击 **Open** 查看并修改配置信息，在 `agents.defaults` 节点下增加 `memorySearch` 相关配置信息，增加的配置信息如下。其中需要将 `<ARK_API_KEY>` 替换为您自己的**专属 API Key**。

```
JSON
```

```
{
 ...
 "agents": {
 "defaults": {
 ...
 "memorySearch": {
 "provider": "openai",
 "model": "doubao-embedding-vision",
 "remote": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>"
 }
 }
 }
 ...
 }
 ...
}
```

3. 配置完成后，先保存配置文件，然后单击 **Update** 更新配置，配置更新完成后需要重新连接服务进入 Web UI。

## 终端方式

1. 在终端执行以下命令打开 OpenClaw 配置文件。

Bash

```
nano ~/.openclaw/openclaw.json
```

2. 在 `agents.defaults` 节点下增加 `memorySearch` 相关配置信息，增加的配置信息如下。其中需要将 `<ARK_API_KEY>` 替换为您自己的[专属 API Key](#)。

JSON

```
{
 ...
 "agents": {
 "defaults": {
 ...
 "memorySearch": {
 "provider": "openai",
 "model": "doubao-embedding-vision",
 "remote": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>"
 }
 }
 }
 ...
 }
 ...
}
```

3. 配置完成后，重启网关使配置生效。

Bash

```
openclaw gateway restart
```

## 配置视觉模型

方舟 Agent Plan 提供生图、生视频模型，支持通过 AI 工具的 Skill 接入并使用。具体配置方法请参见[配置视觉模型](#)。

### 注意

- 不同的套餐支持的视觉模型有所不同，具体参见[支持模型及 Harness](#)。

- 视觉模型不支持通过 Auto 及控制台切换使用。

## 切换模型

请根据语言模型配置方式，选择对应的语言模型切换方案。

- 模型配置：Model Name，支持以下两种方式切换模型。  
前提条件：根据 [三、配置 OpenClaw](#) 步骤完成文件配置，其中 `agents.defaults.models` 包含支持的模型列表。
  - 方式 1：全局配置（永久有效）  
在配置文件中直接修改 `agents.defaults.model.primary` 为目标模型，修改完成后需要重新启动服务。

JSON

```
{
 "agents": {
 "defaults": {
 "model": {
 "primary": "volcengine-agent-plan/doubao-seed-2.0-code"
 }
 }
 }
}
```

- 方式 2：在当前会话切换模型（临时有效）  
在终端执行 `openclaw tui` 后，通过 `/models` 切换模型，当界面返回 "model set to <Model\_Name>" 时，表示配置生效。
- 模型配置：`volcengine-agent-plan/ark-code-latest`，通过控制台切换模型。

配置文件中 `agents.defaults.model.primary` 为 `volcengine-agent-plan/ark-code-latest`。  
模型切换操作：在 [开通管理页面](#) 选择要使用的模型，切换后3-5分钟即可生效。

## 接入飞书机器人

### 前置准备

安装并配置 OpenClaw，参见 [二、安装 OpenClaw](#)、[三、配置 OpenClaw](#)。

#### 注意

飞书插件处于快速迭代阶段，本文使用的插件版本为 **2026.3.10**，如果实际使用与以下步骤存在差异，可参见 [OpenClaw 飞书官网插件](#)。

### 安装飞书插件

1. 执行以下命令安装飞书插件。



Bash

```
npx -y @larksuite/openclaw-lark-tools install
```

如果执行命令时报错，可在命令行前增加 `sudo` 重新执行。

2. 根据界面提示信息，选择 **新建机器人**。

```
Installed plugin: feishu-openclaw-plugin
Restart the gateway to load plugins.
? What would you like to do (请选择操作): (Use arrow keys)
> Create a new bot (新建机器人)
 Use an existing bot linked to OpenClaw (使用 OpenClaw 已关联的机器人)
```

3. 通过飞书客户端扫描二维码，选择 **一键创建飞书机器人**。

```
Restart the gateway to load plugins.
? What would you like to do (请选择操作): Create a new bot (新建机器人)

Scan with Feishu to create your bot (请使用飞书扫码，创建机器人):
```



```
✓ Success! Bot created. (机器人创建成功!)
```

4. 在飞书中向机器人发送 `/feishu start`，若返回版本号信息，代表安装成功。

5. 在飞书中向机器人发送消息，开始对话。

## 4.5 Hermes Agent

本文介绍如何配置方舟 Agent Plan 到 Hermes Agent 中使用。

### 一、前提条件

- 您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。

### 二、安装 Hermes Agent

前提条件：支持在 macOS、Linux 和 WSL2 系统中安装 Hermes Agent。

#### 说明

Hermes Agent 不支持原生 Windows 环境，需要先安装 WSL2，安装命令 `wsl --install`。

1. 执行以下命令安装 Hermes Agent，安装过程中会检测并安装 uv、Python、Node.js、ripgrep、ffmpeg 等依赖。更多安装说明参见 [Hermes Agent 官方文档](#)。

#### Bash

```
curl -fsSL https://raw.githubusercontent.com/NousResearch/hermes-agent/main/scripts/
install.sh | bash
```

2. 安装完成后重新加载 Shell。

#### Bash

```
source ~/.bashrc # or: source ~/.zshrc
```

### 三、配置 Hermes Agent

#### 方式 1：自动化助手（推荐）

ArkCLI Helper 是一个编码工具助手，支持快速配置选择的工具接入 Agent Plan。安装并运行该助手，根据界面提示操作可自动完成工具配置，降低手动配置的时间成本和出错风险。

#### 注意

- ArkCLI Helper 是基于 Ark Helper 升级演进的工​​具配置服务，相关能力将持续在 ArkCLI Helper 上迭代更新。建议优先使用 ArkCLI Helper 完成工具配置。
- ArkCLI Helper 支持 macOS、Linux、Windows 系统。

1. 安装 Ark CLI。

Bash

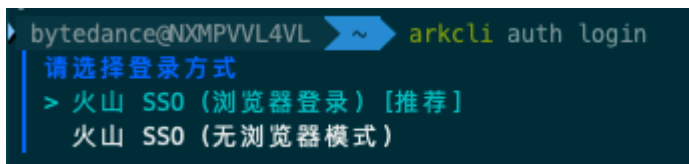
```
1. 通过 NPM 安装
npm install -g @volcengine/ark-cli@latest

2. 验证安装
arkcli --version
```

## 2. 登录 Ark CLI。

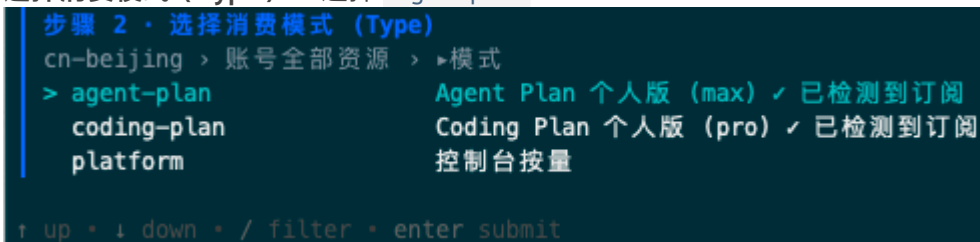
Bash

```
arkcli auth login
```



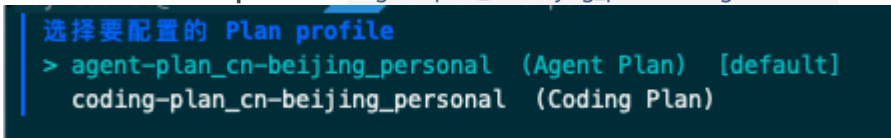
根据界面提示选择登录方式。首次登录时，按提示完成以下配置：

- 选择项目（Project）：可按需选择，或默认选择 账号全部资源。
- 选择消费模式（Type）：选择 `agent-plan`。



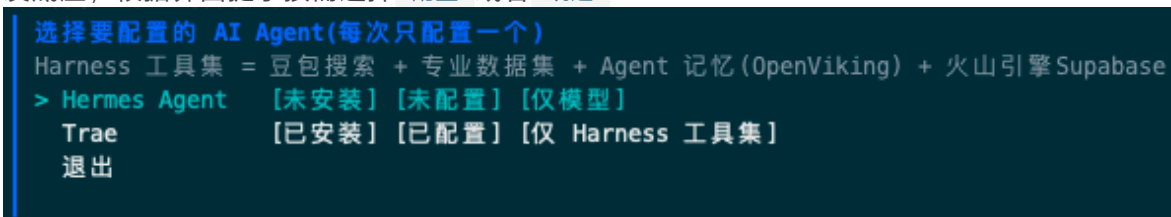
## 3. 运行 ArkCLI Helper，根据界面提示完成 Hermes Agent 的配置。

- 在命令行界面输入 `arkcli helper` 命令，启动 ArkCLI Helper。
- 选择要配置的 Plan profile： `agent-plan_cn-beijing_personal` (Agent Plan)。



- 选择默认 model：按需选择。
- 选择要配置的 AI Agent： `Hermes Agent`。

当前支持的 Harness 包括：豆包搜索、专业数据集、Agent 记忆（OpenViking）、AI Native 应用开发底座，根据界面提示按需选择 配置 或者 跳过。



完成上述操作后，将自动完成 Hermes Agent 的基础配置，并按选择结果自动安装对应的 Harness 工具，无需手动安装。

## 方式 2：手动配置

Agent Plan 兼容 OpenAI 接口协议，支持 Responses API、Chat API。推荐使用 Responses API 的配置，推理效果更好。

### Responses API

#### 注意

使用 Responses API，需要配置 `api_mode: codex_responses`。

执行以下命令配置 Hermes Agent，其中需要将 `<ARK_API_KEY>` 替换为您自己的 [专属 API Key](#)，可用的模型参见[支持模型及 Harness](#)。

开通管理



#### Bash

```
hermes config set model.provider custom
hermes config set model.base_url https://ark.cn-beijing.volces.com/api/plan/v3
hermes config set model.api_key <ARK_API_KEY>
hermes config set model.default ark-code-latest
hermes config set model.api_mode codex_responses
```

配置完成后，配置文件 `~/hermes/config.yaml` 内容如下。您也可以直接编辑该配置文件。

#### JSON

```
model:
 default: ark-code-latest
 provider: custom
 base_url: https://ark.cn-beijing.volces.com/api/plan/v3
 api_key: <ARK_API_KEY>
 api_mode: codex_responses
```

### Chat API

### 注意

使用 Chat API，需要配置 `api_mode: chat_completions`。

执行以下命令配置 Hermes Agent，其中需要将 `<ARK_API_KEY>` 替换为您自己的 [专属 API Key](#)，可用的模型参见[支持模型及 Harness](#)。

开通管理



### Bash

```
hermes config set model.provider custom
hermes config set model.base_url https://ark.cn-beijing.volces.com/api/plan/v3
hermes config set model.api_key <ARK_API_KEY>
hermes config set model.default ark-code-latest
hermes config set model.api_mode chat_completions
```

配置完成后，配置文件 `~/.hermes/config.yaml` 内容如下。您也可以直接编辑该配置文件。

### JSON

```
model:
 default: ark-code-latest
 provider: custom
 base_url: https://ark.cn-beijing.volces.com/api/plan/v3
 api_key: <ARK_API_KEY>
 api_mode: chat_completions
```

## 四、启动并使用 Hermes Agent

1. 执行以下命令开启 Hermes Agent。

### Bash

```
hermes
```

2. 开始使用 Hermes Agent，支持的 CLI Interface 参见[官方文档](#)。

## 其他操作

### 添加图片生成/视频生成能力

方舟 Agent Plan Medium 及以上套餐提供图片生成、视频生成模型配额。方舟提供 Skill 帮助 AI 工具快速接入并使用。具体配置方法请参见：[配置视觉模型](#)。

- 注意
- 不同的套餐支持的视觉模型有所不同，具体参见[支持模型及 Harness](#)。
  - 视觉模型不支持通过 Auto 及控制台切换使用。

### 切换模型

你可以随时根据需求切换 **文本生成模型** 。

配置方式	通过控制台	修改配置文件
切换模型操作	1. 参考<a href="#">三、配置 Hermes Agent</a>配置模型名称为 <code>ark-code-latest</code> 。  2. 在<a href="#">开通管理页面</a> <a href="#">↗</a>选择要使用的模型，切换后3-5分钟即可生效。	参考 <a href="#">三、配置 Hermes Agent</a> 配置模型名称为您需要的模型。

### 接入飞书机器人

- 注意
- 本文档基于 Hermes Agent v0.8.0（2026.4.8）编写，工具迭代演进过程配置步骤可能会有差异，可参考以下内容配置关键信息。

1. 单击[链接](#) [↗](#)快速创建飞书机器人。

2. 在终端执行以下命令配置信息平台。

Bash

hermes gateway setup

- **Select a platform to configure**：

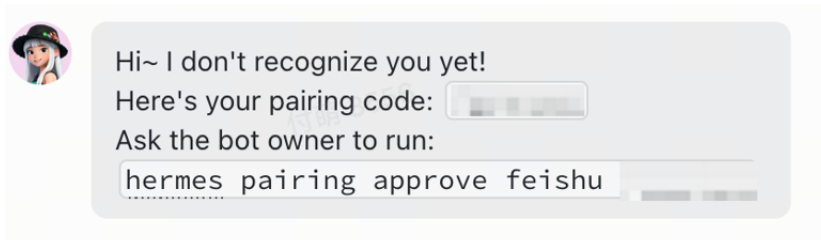
选择 `Feishu / Lark`

- **How would you like to set up Feishu / Lark :** 选择 `Scan QR code to create a new bot automatically (recommended)` , 通过飞书客户端扫描二维码后, 创建飞书机器人。



- **How should direct messages be authorized :** 选择 `Use DM pairing approval`
- **How should group chats be handled :** 选择 `Respond only when @mentioned in groups`
- **Home chat ID :** 暂不配置
- 选择 **done** 后, **Install the gateway as a launchd service :** 选择 `Y`
- **Start the service now :** 选择 `Y`

3. 在飞书打开刚创建的飞书机器人, 发送任意消息获取配对码。



4. 在终端执行以下命令对飞书进行配置, 其中 `<pairing-code>` 需替换为获取的配对码。

Bash

```
hermes pairing approve feishu <pairing-code>
```

```
bytedance@NXMPVVL4VL ~/$ hermes pairing approve feishu [redacted]

Approved! User ou_7b1aab78[redacted] on feishu can now use the bot~
They'll be recognized automatically on their next message.
```

5. 在飞书中发送 `/sethome` 命令将飞书会话设置为主会话，然后即可正常在飞书中使用 Hermes agent。





## 4.6 TRAE

本文介绍如何配置方舟 Agent Plan 到 TRAE 中使用。

### 一、前提条件

- 您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。
- 您已正确安装 TRAE，如未，请访问 [TRAE 官网](#) 下载并安装对应操作系统的版本。

### 二、配置 TRAE

#### 说明

请先将 TRAE IDE 升级至 3.3.57 及以上版本，再进行 Agent Plan 配置。

1. 根据个人用户入口登录后，进入设置中心添加模型。
  - a. 选择 **个人用户** 入口登录后，点击界面右上角的 **设置** 图标，进入设置中心。
  - b. 在左侧导航栏中，选择 **模型**，在模型管理页面进行配置。
  - c. 点击 **+ 添加模型** 按钮，界面上显示 **添加模型** 窗口。
2. 在 **添加模型** 窗口中，配置模型信息。
  - a. 选择服务商：**火山引擎 Agent Plan**
  - b. 选择模型。
    - 直接从列表中选择提供的模型。
    - 若希望使用 Agent Plan 支持的其他模型，单击 **使用其他模型**，**模型 ID** 需使用 **文本生成模型** 的名称，详细模型名称参见 [支持模型及 Harness](#)。
  - c. 填写 API 密钥：[获取专属 API Key](#)

开通管理



- d. 单击 **添加模型**，完成模型的配置。

### 三、使用 TRAE

在 AI 对话输入框的右下角，单击当前模型名称，在模型列表中，选择配置的模型。  
选定模型后，即可使用 TRAE 进行开发任务了。

## 其他操作

### 添加图片生成/视频生成能力

方舟 Agent Plan Medium 及以上套餐提供图片生成、视频生成模型配额。方舟提供 Skill 帮助 AI 工具快速接入并使用。具体配置方法请参见：[配置视觉模型](#)。

- 注意
- 不同的套餐支持的视觉模型有所不同，具体参见[支持模型及 Harness](#)。
  - 视觉模型不支持通过 Auto 及控制台切换使用。

### 切换模型

你可以随时根据需求切换 **文本生成模型** 。

配置方式	通过控制台	通过TRAE
切换模型操作	1. 参考<a href="#">二、配置 TRAE</a>配置模型名称为 <code>ark-code-latest</code> 。  2. 在<a href="#">开通管理页面</a> <a href="#">↗</a>选择要使用的模型，切换后3-5分钟即可生效。	1. 参考<a href="#">二、配置 TRAE</a>添加模型信息。  2. 在 AI 对话输入框的右下角，单击当前模型名称，在模型列表中，选择配置的模型。

## 4.7 OpenViking

**OpenViking** 是专为 AI 智能体（Agent）打造的上下文数据库，主要适用于多 Agent 协同、长期记忆管理、复杂任务执行等场景。该数据库采用 `viking://` 文件系统范式，将智能体的**记忆、资源与技能**进行统一组织管理，依托分层加载机制按需供给上下文，在减少 Token 消耗的同时提升运行效率，结合目录式检索优化信息召回效果。功能上，OpenViking 支持**多租户、多用户及多 Agent 隔离**，且能在会话结束后自动沉淀长期记忆，实现智能体能力的持续积累与自我优化，既有效降低开发者的上下文管理成本，也能显著提升 OpenClaw 等智能体的整体运行表现。

方舟 Agent Plan 支持在 OpenViking 中使用，可参考本文进行配置及使用。

#### 注意

OpenViking 工具处于快速迭代阶段，本文使用的版本为 **0.2.13**，如果实际使用与以下步骤存在差异，可参见 [OpenViking: AI 智能体的上下文数据库](#)。

## 一、前提条件

- 您已按需订阅 Agent Plan 套餐。如未，请访问 [开通管理-Agent Plan](#) 页面订阅套餐。

## 二、安装 OpenViking

### 前提条件

- 已安装 [Python 3.10 或更高版本](#)。
- 已安装 [Go 1.22 或更高版本](#)。
- 已安装 C++ 编译器，且 GCC 9+ 或 Clang 11+。

#### 说明

通过命令 `clang++ --version` 和 `clang++ -std=c++17 -E -x c++ - < /dev/null` 检查 C++编译器支持情况。

### 安装步骤

执行以下命令安装 OpenViking。

Bash

```
pip install openviking --upgrade --force-reinstall
```

安装结束后，执行以下命令查看安装结果，若显示版本号则安装成功。

Bash

```
openviking --version
```

### 三、配置 OpenViking

1. 创建配置文件 `~/openviking/ov.conf`，配置信息如下。需要修改的配置信息如下：

- `storage` ``workspace`：替换为实际的工作空间路径。
- `embedding`：配置向量化模型，用于向量化和语义检索。
  - `embedding.dense.model`：支持的 Embedding 模型 `doubao-embedding-vision`。
  - `embedding.dense.api_key`：替换为您自己的**专属 API Key**。



- `vlm`：配置语言模型，用于图像和内容理解。
  - `vlm.model`：支持的语言模型，具体参见**支持模型及 Harness**。
  - `vlm.api_key`：替换为您自己的**专属 API Key**。

```
{
 "storage": {
 "workspace": "/home/your-name/openviking_workspace"
 },
 "log": {
 "level": "INFO",
 "output": "stdout"
 },
 "embedding": {
 "dense": {
 "backend": "volcengine",
 "api_key": "<ARK_API_KEY>",
 "model": "doubao-embedding-vision",
 "api_base": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "dimension": 1024,
 "input": "multimodal"
 }
 },
 "vlm": {
 "backend": "volcengine",
 "api_key": "<ARK_API_KEY>",
 "model": "doubao-seed-2.0-pro",
 "api_base": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "temperature": 0.1,
 "max_retries": 3
 }
}
```

```
JSON
}
```

2. 设置服务器配置环境变量。

根据系统环境选择对应的方式设置环境变量。

#### 说明

如果配置文件放在其他位置，需要在环境变量中指定对应的文件路径。

## macOS/Linux

执行以下命令，设置环境变量指向对应的配置文件。

Bash

```
export OPENVIKING_CONFIG_FILE=~/.openviking/ov.conf
```

## Windows

选择以下任一方式，设置环境变量指向对应的配置文件。

### · PowerShell

Bash

```
$env:OPENVIKING_CONFIG_FILE = "$HOME/.openviking/ov.conf"
```

### · CMD

Bash

```
set "OPENVIKING_CONFIG_FILE=%USERPROFILE%\openviking\ov.conf"
```

## 四、开始使用 OpenViking

1. 创建 Python 文件 `example.py`，代码如下。

```
import openviking as ov

Initialize OpenViking client with data directory
client = ov.OpenViking(path="./data")
```

```
try:
 # Initialize the client
 client.initialize()

 # Add resource (supports URL, file, or directory)
 add_result = client.add_resource(
 path="https://raw.githubusercontent.com/volcengine/OpenViking/refs/heads/main/
README.md"
)
 root_uri = add_result['root_uri']

 # Explore the resource tree structure
 ls_result = client.ls(root_uri)
 print(f"Directory structure:\n{ls_result}\n")

 # Use glob to find markdown files
 glob_result = client.glob(pattern="**/*.md", uri=root_uri)
 if glob_result['matches']:
 content = client.read(glob_result['matches'][0])
 print(f"Content preview: {content[:200]}...\n")

 # Wait for semantic processing to complete
 print("Wait for semantic processing...")
 client.wait_processed()

 # Get abstract and overview of the resource
 abstract = client.abstract(root_uri)
 overview = client.overview(root_uri)
 print(f"Abstract:\n{abstract}\n\nOverview:\n{overview}\n")

 # Perform semantic search
 results = client.find("what is openviking", target_uri=root_uri)
 print("Search results:")
 for r in results.resources:
 print(f" {r.uri} (score: {r.score:.4f})")

 # Close the client
 client.close()

except Exception as e:
 print(f"Error: {e}")
```

2. 执行命令 `python example.py`，输出如下内容则说明 OpenViking 已成功运行。

Plain

Directory structure:

...

Content preview: ...

Wait for semantic processing...

Abstract:

...

Overview:

...

Search results:

viking://resources/... (score: 0.8523)

...

## 其他操作

### CLI 服务端配置并使用 OpenViking

如果需要将 OpenViking 作为独立的 HTTP 服务运行，然后从任意客户端连接，请参考以下步骤进行配置。

#### 前提条件

已安装并完成配置 OpenViking，具体参见[安装 OpenViking](#)、[配置 OpenViking](#)。

#### 启动服务

执行以下命令启动服务。

Bash

```
openviking-server
```

启动完成后，在新的命令行界面执行以下命令验证服务启动是否成功，若返回 `"status":"ok"` 则表示启动成功。

Bash

```
curl http://localhost:1933/health
```

### 使用 CLI 连接

1. 创建 CLI 连接配置文件 `~/openviking/ovcli.conf`，配置信息如下。其中 `<ARK_API_KEY>` 需替换为您自己的 API Key [🔗](#)。



JSON

```
{
 "url": "http://localhost:1933",
 "api_key": "<ARK_API_KEY>",
 "output": "table"
}
```

2. 在新的命令行界面执行以下 CLI 命令。

Bash

```
Check system health
openviking observer system

Add a resource to memory
openviking add-resource https://raw.githubusercontent.com/volcengine/OpenViking/refs/heads/main/README.md

List all synchronized resources
openviking ls viking://resources

Query
openviking find "what is openviking"
```

## 使用 curl 连接

启动服务后，也可以在命令行界面使用 curl 命令连接 OpenViking。

Bash

```
Add a resource
curl -X POST http://localhost:1933/api/v1/resources \
 -H "Content-Type: application/json" \
 -d '{"path": "https://raw.githubusercontent.com/volcengine/OpenViking/refs/heads/main/README.md"}'

List resources
curl "http://localhost:1933/api/v1/fs/ls?uri=viking://resources/"

Semantic search
curl -X POST http://localhost:1933/api/v1/search/find \
 -H "Content-Type: application/json" \
 -d '{"query": "what is openviking"}'
```

## 在 OpenClaw 配置 OpenViking 记忆功能

通过 OpenViking 为 OpenClaw 提供长效记忆能力，配置完成后，OpenClaw 将自动记住对话中的重要信息，并在回复前回忆相关内容。

### 前提条件

已完成 OpenClaw 的安装与配置，具体参见[安装 OpenClaw](#)、[配置 OpenClaw](#)。

### 安装及配置

具体步骤参见[为 OpenClaw 安装 OpenViking 记忆功能](#) [↗](#)。

安装完成后，编辑配置文件 `~/openviking/ov.conf`，使用 Agent Plan 支持的模型。配置信息可参见[配置 OpenViking](#)。

## 4.8 其他工具

方舟 Agent Plan 已适配 Anthropic、OpenAI 接口协议，可兼容对应工具生态。开发者通过配置 Agent Plan 专用的 Base URL并指定支持模型，可快速接入并使用相关工具。

注意

获取专属 API Key [🔗](#)：Agent Plan 专属 API Key 与火山方舟平台的 API Key 不同，请勿混用。

开通管理

模型

应用组件库

云产品

高级创作权益包

Agent Plan

Agent Plan 企业版

Coding Plan

Coding Plan 企业版

管理素材资产

套餐信息

套餐类型

套餐限制

订阅状态

开始时间

结束时间

25000 AFP/5小时

125000 AFP/周

250000 AFP/月

计算规则详见 运营文档

生效中

2026-04-29 03:33

2026-07-29 23:59

使用配置

用量统计

配置文档

1 配置模型及Base URL

2 配置Harness (可选)

3 配置专属API Key

4 接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key

ark-

创建时间

操作

2026-04-29 03:33

更新API KEY

## 适用工具

接口协议	Base URL	适用工具
兼容 Anthropic 接口协议	<code>https://ark.cn-beijing.volces.com/api/plan</code>	Claude Code
兼容 OpenAI 接口协议	<code>https://ark.cn-beijing.volces.com/api/plan/v3</code>	OpenCode、OpenClaw、TRAE、Hermes Agent、Codex CLI、Cline、Cursor、Kilo Code、Roo Code 等

## 接入Cline

### 安装步骤

打开 VSCode，在扩展市场搜索 `Cline` 安装。

### 配置工具

Cline插件安装完成后，您需要配置以下信息。

- **API Provider**：`OpenAI Compatible`（Agent Plan 接口兼容 OpenAI 标准）
- **Base URL**：`https://ark.cn-beijing.volces.com/api/plan/v3`
- **API Key**：[获取API Key 🔗](#)
- **Model ID**：支持配置 `ark-code-latest`、配置 Model Name 两种方式。模型信息见[支持模型及 Harness](#)，需配置文本生成模型。

配置完成后，就可以在输入框中输入需求，与模型进行交互。

## 接入Cursor

### 安装步骤

官网下载安装包：通过[Cursor官网](#) 下载并安装Cursor。

### 配置工具

#### 说明

由于 Cursor 的限制，只有订阅了 Cursor Pro 及以上套餐的用户才支持自定义配置模型。

Cursor 安装完成后，Models 模块的具体配置如下：

- OpenAI API Key: [获取API Key](#)
- Override OpenAI Base URL: `https://ark.cn-beijing.volces.com/api/plan/v3`
- Add Custom Model: 支持配置 `ark-code-latest` 、配置 Model Name 两种方式。模型信息见[支持模型及Harness](#)，需配置文本生成模型。

#### 注意

在 Cursor 中配置 `minimax-m2.7` 、 `glm-5.2` 、 `kimi-k2.6` 模型时若遇到名称冲突问题，建议改用 `minimax-m2-7` 、 `glm-5-2` 、 `kimi-k2-6` 配置。

配置完成后，即可在聊天面板中选择配置的模型进行交互。

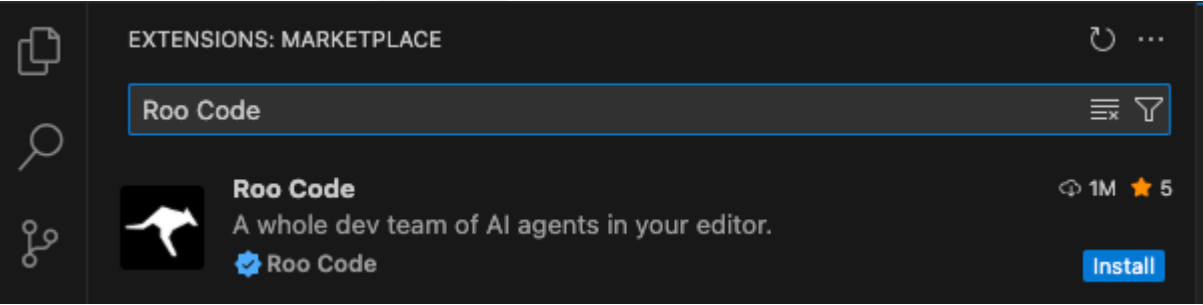
## 接入Roo Code

### 安装步骤

#### 说明

若模型表现异常，建议将插件更新到最新版本（3.43.0 及后续版本）。

打开 VSCode，在扩展市场搜索 `Roo Code` 进行安装，安装完成后选择信任发布者。



### 配置工具

安装完成后，配置以下信息。

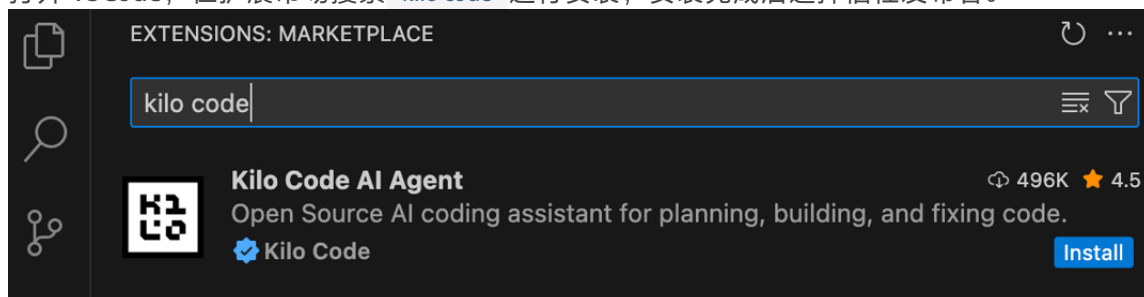
- **API Provider** : `OpenAI Compatible` ( Agent Plan 接口兼容 OpenAI 标准 )
- **Base URL** : `https://ark.cn-beijing.volces.com/api/plan/v3`
- **API Key** : [获取API Key](#)
- **Model** : 支持配置 `ark-code-latest` 、配置 Model Name 两种方式。模型信息见[支持模型及 Harness](#)，需配置文本生成模型。

配置完成后，就可以在输入框中输入需求，与模型进行交互。

## 接入 Kilo Code

### 安装步骤

打开 VSCode，在扩展市场搜索 `kilo code` 进行安装，安装完成后选择信任发布者。



### 配置工具

选择 **Settings – Providers – Custom provider**，然后配置以下信息。

- **API Provider** : `OpenAI Compatible` ( Agent Plan 接口兼容 OpenAI 标准 )
- **Base URL** : `https://ark.cn-beijing.volces.com/api/plan/v3`
- **API Key** : [获取API Key](#)
- **Model** : 支持配置 `ark-code-latest` 、配置 Model Name 两种方式。模型信息见[支持模型及 Harness](#)，需配置文本生成模型。

配置完成后，就可以在输入框中输入需求，与模型进行交互。

## 5. 接入多模态生成模型

### 5.1 接入向量化模型

方舟 Agent Plan 提供向量化模型，可以为工具提供语义向量能力，将对话、文档与任务信息等转换为高维向量表示，实现基于语义相似度的高效检索。

相比传统关键词匹配，Embedding 检索在信息量大、对话长、资料分散的场景下，能够更准确地召回相关内容，显著降低遗漏与误判。同时，Embedding 为记忆检索、知识库问答与工具调用提供统一的语义检索层，帮助工具在不增加主模型推理成本的前提下，提升跨轮次上下文追踪能力与整体效果。

#### 核心配置信息

在 Agent Plan 中使用向量模型时，需要的核心配置如下：

- **专属 Base URL**：https://ark.cn-beijing.volces.com/api/plan/v3，Agent Plan 专属 Base URL 中包含 /plan，请勿混用其他 Base URL。
- **专属 API Key**：获取专属 API Key，其他方舟 API Key 如 Coding Plan API Key 无法在 Agent Plan 中使用。

开通管理

模型应用组件库云产品高级创作权益包Agent PlanCoding Plan管理素材资产

套餐信息套餐类型套餐限额订阅状态开始时间结束时间剩余时间

Small 套餐2000 AFP/5小时7000 AFP/周20000 AFP/月计算规则详见 说明文档生效中2026-05-07 20:432026-06-07 23:5931天

使用配置用量统计配置文档

1配置模型及Base URL2配置Harness（可选）3配置专属API Key4接入使用

专属API KEYAgent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key	创建时间	操作
ark-ef	2026-05-07 20:43	更新API KEY

- **支持的模型**：doubao-embedding-vision（对应模型 doubao-embedding-vision-251215）

注意

- 建议固定使用同一版本的向量化模型，请勿混用不同版本。
- 向量化模型只支持在配置文件中指定 Model Name，不支持通过 Auto 及控制台切换使用。

#### 接入向量化模型

##### OpenClaw

OpenClaw 支持配置向量模型，解决大模型原生记忆有限、易产生幻觉、无法适配私有数据的问题，从而提升模型在处理复杂任务时的准确性和可靠性。

1. 前提条件：已完成 OpenClaw 的安装与配置，步骤参见二、安装 OpenClaw、三、配置 OpenClaw。

版权所有©北京火山引擎科技有限公司

101/215

2. 选择以下任一方式配置向量模型。

## Web UI 方式

1. 执行以下命令打开 Web UI。

Bash

```
openclaw dashboard
```

2. 在左侧菜单栏选择配置 – **Settings – Advanced**，单击 **Open** 查看并修改配置信息，在 `agents.defaults` 节点下增加 `memorySearch` 相关配置信息，增加的配置信息如下。其中需要将 `<ARK_API_KEY>` 替换为您自己的**专属 API Key** [🔗](#)。

JSON

```
{
 ...
 "agents": {
 "defaults": {
 ...
 "memorySearch": {
 "provider": "openai",
 "model": "doubao-embedding-vision",
 "remote": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>"
 }
 }
 }
 ...
 }
 ...
}
```

3. 配置完成后，先保存配置文件，然后单击 **Update** 更新配置，配置更新完成后需要重新连接服务进入 Web UI。

## 终端方式

1. 在终端执行以下命令打开 OpenClaw 配置文件。

Bash

```
nano ~/.openclaw/openclaw.json
```

2. 在 `agents.defaults` 节点下增加 `memorySearch` 相关配置信息，增加的配置信息如下。其中需要将 `<ARK_API_KEY>` 替换为您自己的**专属API Key** [🔗](#)。

JSON

```
{
 ...
 "agents": {
 "defaults": {
 ...
 "memorySearch": {
 "provider": "openai",
 "model": "doubao-embedding-vision",
 "remote": {
 "baseUrl": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "apiKey": "<ARK_API_KEY>"
 }
 }
 }
 ...
 }
 ...
}
```

3. 配置完成后，重启网关使配置生效。

Bash

```
openclaw gateway restart
```

## OpenViking

OpenViking 是专为 AI 智能体设计的开源上下文数据库，支持多模态 Embedding 模型的集成与接入，实现非结构化数据的向量化转换，并提供高效的语义相似度检索能力。

1. 前提条件：已完成 OpenViking 的安装与基础配置，步骤参见 [安装 OpenViking](#)、[配置 OpenViking](#)。
2. 在配置文件 `~/openviking/ov.conf` 中新增 `embedding` 相关配置。其中需要将 `<ARK_API_KEY>` 替换为您自己的[专属 API Key](#)。

```
{
 ...
 "embedding": {
 "dense": {
 "api_base": "https://ark.cn-beijing.volces.com/api/plan/v3",
 "api_key": "<ARK_API_KEY>",
 "provider": "volcengine",
 "dimension": 1024,
```



```
JSON "model" : "doubao-embedding-vision"
 },
 "max_concurrent": 10
 }
 ...
}
```

## 5.2 接入视觉模型

方舟 Agent Plan 提供图片生成、视频生成模型，支持通过 Skill、API 接口调用两种方式接入并使用生图、生视频能力。

### 通过 Skill 接入

在 AI 工具中使用图片生成、视频生成能力，建议使用 Skill 方式接入。

#### 方式 1：自动化助手（推荐）

支持的 AI 工具：Claude Code、OpenCode、OpenClaw。

Ark Helper 是一个编码工具助手，支持快速配置选择的工具接入 Agent Plan。安装并运行该助手，根据界面提示操作可自动完成工具配置，其中包括自动完成视频生成 Skill（[byted-ark-seedance-skill](#) ）、图片生成 Skill（[byted-ark-seedream-skill](#) ）的安装配置，有效降低手动配置的时间成本和出错风险。

#### 注意

- Ark Helper 仅支持 macOS、Linux 系统，暂不支持 Windows 系统。
- 以下配置步骤及截图为 Ark Helper 首次使用指引；非首次使用请按界面提示完成套餐配置和工具配置。

1. 执行以下命令安装 Ark Helper。

Bash

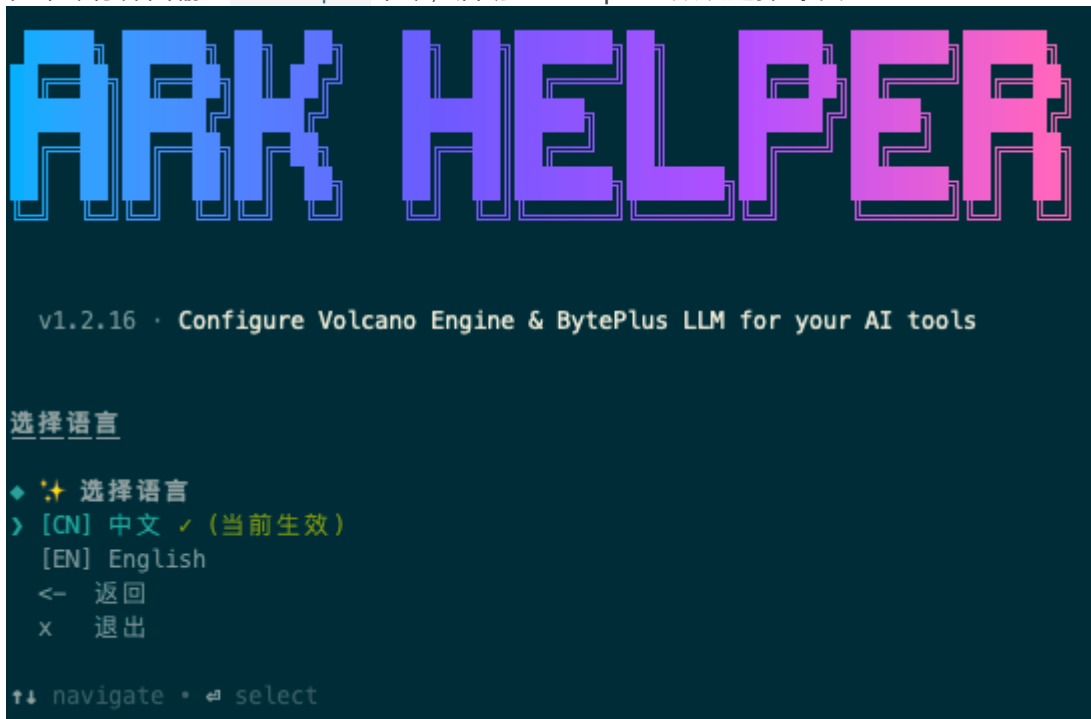
```
curl -fsSL https://lf3-static.bytednsdoc.com/obj/eden-cn/ylwslo-yrh/ljhWZthlaukjlkulzlp/install.sh
| sh
```

安装完成后，执行以下命令查看安装的版本号。

Bash

```
ark-helper --version
```

2. 在命令行界面输入 `ark-helper` 命令，启动 Ark Helper。默认选择 **中文**。

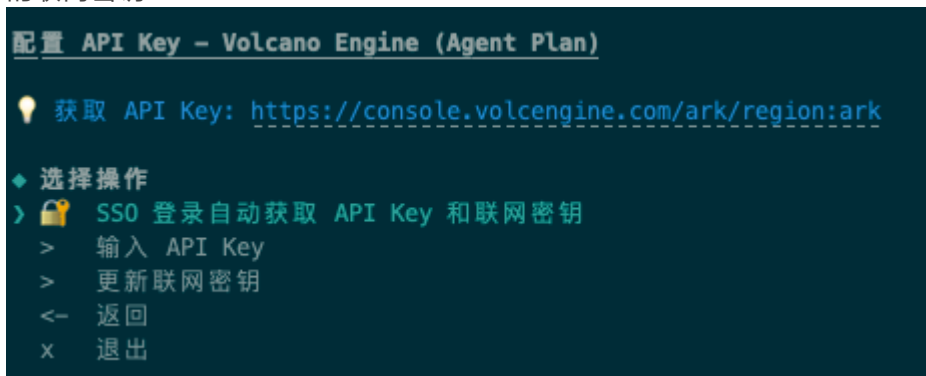


3. 根据界面提示完成套餐配置。

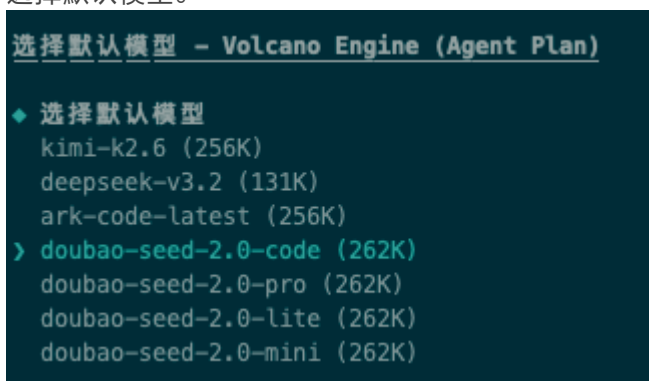
a. 选择要配置的套餐： `[Volcano] Volcano Engine (国内)` - `Agent Plan`。



- b. 配置 API Key: 推荐选择 `SSO 登录自动获取 API Key 和联网密钥`。根据界面提示完成登录并选择要使用的联网密钥。



- c. 选择默认模型。



4. 根据界面提示点击返回，完成工具配置。以 Claude Code 配置为例，具体步骤如下。

- a. 选择编码工具: `Claude Code`。



- b. 选择操作： 设置 Volcano Agent Plan (API Key + Skills + MCP) 到 Claude Code ，配置完成后，选择 退出 。如果需要重新配置工具，可先选择 卸载 Claude Code 配置 ，再重新进行配置流程。



完成上述操作后，将自动完成视觉模型 Skill、豆包搜索 Skill / MCP 的安装，无需手动安装。

5. 启动配置好的工具，发送视频生成/图片生成提示词，将会触发视觉模型 Skill 生成视频/图片。

## 方式 2：手动安装 Skill

### 安装视频生成 Skill

通过 Skill 方式，可以快速为 AI 工具安装并接入视频生成能力，目前支持的 AI 工具：OpenClaw、Hermes Agent、Claude Code。视频生成 Skill 信息参见 [byted-ark-seedance-skill](#)。

#### OpenClaw

1. 已完成 OpenClaw 的安装及语言模型的配置，具体步骤见 [OpenClaw](#)。
2. 在终端执行以下命令为 OpenClaw 安装并接入视频生成 Skill `byted-ark-seedance-skill`。

Bash

```
npx skills add https://skills.volces.com/skills/volcengine/agentplan -s byted-ark-seedance-skill --agent openclaw
```

3. 安装完成后，可以在 OpenClaw 发送视频生成的提示词，触发并使用生成视频能力。

#### Hermes Agent

1. 已完成 Hermes Agent 的安装及语言模型的配置，具体步骤见 [Hermes Agent](#)。
2. 进入 [byted-ark-seedance-skill](#) 页面下载 ZIP 压缩包，将解压后的文件粘贴至 `~/hermes/skills/` 目录下。
3. 配置完成后，可以在 Hermes Agent 发送视频生成的提示词，触发并使用生成视频能力。

#### Claude Code

1. 已完成 Claude Code 的安装及语言模型的配置，具体步骤见 [Claude Code](#)。

2. 在终端执行以下命令为 Claude Code 安装并接入视频生成 Skill `byted-ark-seedance-skill`。

Bash

```
npx skills add https://skills.volces.com/skills/volcengine/agentplan -s byted-ark-seedance-skill --agent claude-code
```

3. 配置完成后，可以在 Claude Code 发送视频生成的提示词，触发并使用生成视频能力。

## TRAE

1. 已完成 TRAE 的安装及语言模型的配置，具体步骤见 [TRAE](#)。
2. 在终端执行以下命令下载 `byted-ark-seedance-skill`。

Bash

```
npx skills add https://skills.volces.com/skills/volcengine/agentplan -s byted-ark-seedance-skill --agent trae
```

3. 安装完成后，可以在 TRAE 发送视频生成的提示词，触发并使用生成视频能力。

## 安装图片生成 Skill

通过 Skill 方式，可以快速为 AI 工具安装并接入图片生成能力，目前支持的 AI 工具：OpenClaw、Hermes Agent、Claude Code。图片生成 Skill 信息参见 [byted-ark-seedream-skill](#)。

### OpenClaw

1. 已完成 OpenClaw 的安装及语言模型的配置，具体步骤见 [OpenClaw](#)。
2. 在终端执行以下命令为 OpenClaw 安装并接入图片生成 Skill `byted-ark-seedream-skill`。

Bash

```
npx skills add https://skills.volces.com/skills/volcengine/agentplan -s byted-ark-seedream-skill --agent openclaw
```

3. 安装完成后，可以在 OpenClaw 发送图片生成的提示词，触发并使用生成图片能力。

### Hermes Agent

1. 已完成 Hermes Agent 的安装及语言模型的配置，具体步骤见 [Hermes Agent](#)。
2. 进入 [byted-ark-seedream-skill](#) 页面下载 ZIP 压缩包，将解压后的文件粘贴至 `~/hermes/skills/` 目录下。
3. 安装完成后，可以在 Hermes Agent 发送图片生成的提示词，触发并使用生成图片能力。

### Claude Code

1. 已完成 Claude Code 的安装及语言模型的配置，具体步骤见 [Claude Code](#)。
2. 在终端执行以下命令为 Claude Code 安装并接入图片生成 Skill `byted-ark-seedream-skill`。

Bash

```
npx skills add https://skills.volces.com/skills/volcengine/agentplan -s byted-ark-seedream-skill --agent claude-code
```

3. 安装完成后，可以在 Claude Code 发送图片生成的提示词，触发并使用生成图片能力。

TRAE

- 1. 已完成 TRAE 的安装及语言模型的配置，具体步骤见 [TRAE](#)。
- 2. 在终端执行以下命令下载 `byted-ark-seedream-skill` 。

Bash

```
npx skills add https://skills.volces.com/skills/volcengine/agentplan -s byted-ark-seedream-skill --agent trae
```

3. 安装完成后，可以在 TRAE 发送图片生成的提示词，触发并使用生成图片能力。

通过 API 接入

如果使用的 AI 工具不支持接入 Skill，或者无需在 AI 工具中使用，可通过 API 接入的方式使用图片生成、视频生成能力。

核心配置信息

在使用 Agent Plan 时，需要使用专属 API Key、专属 Base URL、支持的模型来调用视频生成、图片生成 API，否则可能会调用失败或产生额外费用。

- 专属 API Key： [获取专属 API Key](#)，其他方舟 API Key 如 Coding Plan API Key 无法在 Agent Plan 中使用。



- 支持的模型： 见[支持模型及 Harness](#)。
- 专属 Base URL： Agent Plan 对应的 API 接口信息中包含 `/plan`，请勿混用其他 API 接口。
  - Curl 方式：支持的 API 参数可查看对应的 API 文档。

视频生成 API	图片生成 API
■ 创建视频生成任务 <a href="#">🔗</a>: <code>https://ark.cn-beijing.volces.com/api/plan/v3/contents/generations/tasks</code>  ■ 查询视频生成任务 <a href="#">🔗</a>: <code>https://ark.cn-beijing.volces.com/api/plan/v3/contents/generations/tasks/{id}</code>  ■ 查询视频生成任务列表 <a href="#">🔗</a>: <code>https://ark.cn-beijing.volces.com/api/plan/v3/contents/generations/tasks?page_num={page_num}&amp;page_size={page_size}&amp;filter.status={filter.status}&amp;filter.task_ids={filter.task_ids}&amp;filter.model={filter.model}</code>  ■ 取消或删除视频生成任务 <a href="#">🔗</a>: <code>https://ark.cn-beijing.volces.com/api/plan/v3/contents/generations/tasks/{id}</code>	图片生成 <a href="#">🔗</a>: <code>https://ark.cn-beijing.volces.com/api/plan/v3/images/generations</code>

- SDK 方式:
- 专属 Base URL: `https://ark.cn-beijing.volces.com/api/plan/v3`

### 调用视频生成 API

参见[视频生成教程](#) [🔗](#)、[Doubao Seedance 2.0 系列教程](#) [🔗](#)完成视频生成任务，需注意必须使用 Agent Plan 专属 API Key、专属 Base URL 及支持的模型，否则可能会调用失败或产生额外费用。具体参见[核心配置信息](#)。

通过 Agent Plan 创建视频生成任务的示例如下：

#### Curl

```
curl https://ark.cn-beijing.volces.com/api/plan/v3/contents/generations/tasks \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $AGENT_API_KEY" \
-d '{
 "model": "doubao-seedance-2.0",
 "content": [
 {
 "type": "text",
 "text": "女孩抱着狐狸，女孩睁开眼，温柔地看向镜头，狐狸友善地抱着，镜头缓缓拉出，女孩的头发被风吹动，可以听到风声"
 },
 {
 "type": "image_url",
 "image_url": {
 "url": "https://ark-project.tos-cn-beijing.volces.com/doc_image/i2v_foxrgirl.png"
 }
 }
]
}
```



```
Bash }
 }
],
 "generate_audio": true,
 "ratio": "adaptive",
 "duration": 5,
 "watermark": false
}'
```

## Python

```
import os
from volcenginesdkarkruntime import Ark

client = Ark(
 # Agent Plan 专属 Base URL
 base_url="https://ark.cn-beijing.volces.com/api/plan/v3",
 # Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/
 openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 api_key=os.environ.get("AGENT_API_KEY")
)

if __name__ == "__main__":
 print("----- create request -----")
 resp = client.content_generation.tasks.create(
 model="doubao-seedance-2.0",
 content=[
 {
 "text": (
 "女孩抱着狐狸，女孩睁开眼，温柔地看向镜头，狐狸友善地抱着，镜头缓缓拉出，女孩
的头发被风吹动"
),
 "type": "text"
 },
 {
 "image_url": {
 "url": (
 "https://ark-project.tos-cn-beijing.volces.com/doc_image/i2v_foxrgirl.png"
)
 },
 "type": "image_url"
 }
],
 generate_audio=True,
```

```
Pythonratio="adaptive",
 duration=5,
 watermark=False,
)

 print(resp)
```

## Java

```
package com.ark.sample;

import com.volcengine.ark.runtime.model.content.generation.*;
import com.volcengine.ark.runtime.model.content.generation.CreateContentGenerationTaskRequest.Content;
import com.volcengine.ark.runtime.service.ArkService;
import okhttp3.ConnectionPool;
import okhttp3.Dispatcher;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.TimeUnit;

public class ContentGenerationTaskExample {
 // Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 static String apiKey = System.getenv("AGENT_API_KEY");
 static ConnectionPool connectionPool = new ConnectionPool(5, 1, TimeUnit.SECONDS);
 static Dispatcher dispatcher = new Dispatcher();
 static ArkService service = ArkService.builder()
 .baseUrl("https://ark.cn-beijing.volces.com/api/plan/v3") // Agent Plan 专属 Base URL
 .dispatcher(dispatcher)
 .connectionPool(connectionPool)
 .apiKey(apiKey)
 .build();

 public static void main(String[] args) {
 String model = "doubao-seedance-2.0";
 Boolean generateAudio = true;
 String ratio = "adaptive";
 Long duration = 5L;
 Boolean watermark = false;
 System.out.println("----- create request -----");
 List<Content> contents = new ArrayList<>();
```

```
Java
// Combination of text prompt and parameters
contents.add(Content.builder()
 .type("text")
 .text("女孩抱着狐狸，女孩睁开眼，温柔地看向镜头，狐狸友善地抱着，镜头缓缓拉出，
女孩的头发被风吹动，可以听到风声")
 .build());
// The URL of the first frame image
contents.add(Content.builder()
 .type("image_url")
 .imageUrl(CreateContentGenerationTaskRequest.ImageUrl.builder()
 .url("https://ark-project.tos-cn-beijing.volces.com/doc_image/i2v_foxrgirl.png")
 .build())
 .build());

// Create a video generation task
CreateContentGenerationTaskRequest createRequest =
CreateContentGenerationTaskRequest.builder()
 .model(model)
 .content(contents)
 .generateAudio(generateAudio)
 .ratio(ratio)
 .duration(duration)
 .watermark(watermark)
 .build();

CreateContentGenerationTaskResult createResult =
service.createContentGenerationTask(createRequest);
System.out.println(createResult);

service.shutdownExecutor();
}
}
```

Go

```
package main
```

```
import (
 "context"
 "fmt"
 "os"
 "time"
```

```

"github.com/volcengine/volcengine-go-sdk/service/arkruntime"
"github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"
"github.com/volcengine/volcengine-go-sdk/volcengine"
)

func main() {
 client := arkruntime.NewClientWithApiKey(
 // Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/
 openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 os.Getenv("AGENT_API_KEY"),
 // Agent Plan 专属 Base URL
 arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/plan/v3"),
)
 ctx := context.Background()
 modelEp := "doubao-seedance-2.0"

 // Generate a task
 fmt.Println("----- create request -----")
 createReq := model.CreateContentGenerationTaskRequest{
 Model: modelEp,
 GenerateAudio: volcengine.Bool(true),
 Ratio: volcengine.String("adaptive"),
 Duration: volcengine.Int64(5),
 Watermark: volcengine.Bool(false),
 Content: []*model.CreateContentGenerationContentItem{
 {
 // Combination of text prompt and parameters
 Type: model.ContentGenerationContentItemTypeText,
 Text: volcengine.String("女孩抱着狐狸，女孩睁开眼，温柔地看向镜头，狐狸友善地抱着，
 镜头缓缓拉出，女孩的头发被风吹动，可以听到风声"),
 },
 {
 // The URL of the first frame image
 Type: model.ContentGenerationContentItemTypeImage,
 ImageURL: &model.ImageURL{
 URL: "https://ark-project.tos-cn-beijing.volces.com/doc_image/i2v_foxrgirl.png",
 },
 },
 },
 }
 createResp, err := client.CreateContentGenerationTask(ctx, createReq)
 if err != nil {
 fmt.Printf("create content generation error: %v", err)
 return
 }
 taskID := createResp.ID

```

```
Go fmt.Printf("Task Created with ID: %s", taskID)
}
```

## 调用图片生成 API

参见[Seedream 4.0-5.0 教程](#) 完成图片生成任务，需注意必须使用 Agent Plan 专属 API Key、专属 Base URL 及支持的模型，否则可能会调用失败或产生额外费用。具体参见[核心配置信息](#)。  
通过 Agent Plan 实现文生图的示例如下：

### Curl

Bash

```
curl https://ark.cn-beijing.volces.com/api/plan/v3/images/generations \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $AGENT_API_KEY" \
-d '{
 "model": "doubao-seedream-5.0-lite",
 "prompt": "充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
 "size": "2K",
 "output_format": "png",
 "watermark": false
}'
```

### Python

```
import os
Install SDK: pip install 'volcengine-python-sdk[ark]' .
from volcenginesdkarkruntime import Ark

client = Ark(
 # Agent Plan 专属 Base URL
 base_url="https://ark.cn-beijing.volces.com/api/plan/v3",
 # Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 api_key=os.getenv('AGENT_API_KEY'),
)

imagesResponse = client.images.generate(
 model="doubao-seedream-5.0-lite",
 prompt="充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
 size="2K",
```

```

Python
output_format="png",
response_format="url",
watermark=False
)

print(imagesResponse.data[0].url)

```

## Java

```

package com.ark.sample;
import com.volcengine.ark.runtime.model.images.generation.*;
import com.volcengine.ark.runtime.service.ArkService;
import okhttp3.ConnectionPool;
import okhttp3.Dispatcher;

import java.util.Arrays;
import java.util.List;
import java.util.concurrent.TimeUnit;

public class ImageGenerationsExample {
 public static void main(String[] args) {
 // Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/
 // openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 String apiKey = System.getenv("AGENT_API_KEY");
 ConnectionPool connectionPool = new ConnectionPool(5, 1, TimeUnit.SECONDS);
 Dispatcher dispatcher = new Dispatcher();
 ArkService service = ArkService.builder()
 .baseUrl("https://ark.cn-beijing.volces.com/api/plan/v3") // Agent Plan 专属 Base URL
 .dispatcher(dispatcher)
 .connectionPool(connectionPool)
 .apiKey(apiKey)
 .build();

 GenerateImagesRequest generateRequest = GenerateImagesRequest.builder()
 .model("doubao-seedream-5.0-lite")
 .prompt("充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。")
 .size("2K")
 .sequentialImageGeneration("disabled")
 .outputFormat("png")
 .responseFormat(ResponseFormat.Url)
 .stream(false)
 .watermark(false)
 }
}

```

```

Java .build();
 ImagesResponse imagesResponse = service.generateImages(generateRequest);
 System.out.println(imagesResponse.getData().get(0).getUrl());

 service.shutdownExecutor();
 }
 }
}

```

## Go

```

package main

import (
 "context"
 "fmt"
 "os"

 "github.com/volcengine/volcengine-go-sdk/service/arkruntime"
 "github.com/volcengine/volcengine-go-sdk/service/arkruntime/model"
 "github.com/volcengine/volcengine-go-sdk/volcengine"
)

func main() {
 client := arkruntime.NewClientWithApiKey(
 // Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/
 // openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 os.Getenv("AGENT_API_KEY"),
 // Agent Plan 专属 Base URL
 arkruntime.WithBaseUrl("https://ark.cn-beijing.volces.com/api/plan/v3"),
)
 ctx := context.Background()
 outputFormat := model.OutputFormatPNG

 generateReq := model.GenerateImagesRequest{
 Model: "doubao-seedream-5.0-lite",
 Prompt: "充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
 Size: volcengine.String("2K"),
 OutputFormat: &outputFormat,
 ResponseFormat: volcengine.String("url"),
 Watermark: volcengine.Bool(false),
 }
}

```

```
Go
imagesResponse, err := client.GenerateImages(ctx, generateReq)
if err != nil {
 fmt.Printf("generate images error: %v\n", err)
 return
}

fmt.Printf("%s\n", *imagesResponse.Data[0].Url)
}
```

## OpenAI

```
Python

import os
from openai import OpenAI

client = OpenAI(
 # Agent Plan 专属 Base URL
 base_url="https://ark.cn-beijing.volces.com/api/plan/v3",
 # Agent Plan 专属 API Key: https://console.volcengine.com/ark/region:ark+cn-beijing/
 openManagement?LLM=%7B%7D&OpenModelVisible=false&advancedActiveKey=agentPlan
 api_key=os.getenv('AGENT_API_KEY'),
)

imagesResponse = client.images.generate(
 model="doubao-seedream-5.0-lite",
 prompt="充满活力的特写编辑肖像，模特眼神犀利，头戴雕塑感帽子，色彩拼接丰富，眼部焦点锐利，景深较浅，具有Vogue杂志封面的美学风格，采用中画幅拍摄，工作室灯光效果强烈。",
 size="2K",
 output_format="png",
 response_format="url",
 extra_body={
 "watermark": False,
 },
)

print(imagesResponse.data[0].url)
```

## 支持模型及能力

### 注意

Agent Plan 的套餐支持的模型不同，具体参见[支持模型及 Harness](#)。



生视频模型

模型		doubao-seedance-2.0	doubao-seedance-2.0-fast	doubao-seedance-2.0-mini【 New 】	doubao-seedance-1.5-pro
文生视频		✓	✓	✓	✓
图生视频-首帧		✓	✓	✓	✓
图生视频-首尾帧		✓	✓	✓	✓
多模态参考	图片参考	✓	✓	✓	-
	视频参考	✓	✓	✓	-
	组合参考				
	· 图片 + 音频  · 图片 + 视频  · 视频 + 音频  · 图片 + 视频 + 音频	✓	✓	✓	·
编辑视频		✓	✓	✓	-
延长视频		✓	✓	✓	-
生成有声视频		✓	✓	✓	✓
联网搜索工具		✓	✓	✓	-
样片模式		-	-	-	✓
返回视频产物对应的尾帧图		✓	✓	✓	✓
输出视频规格	输出分辨率	480p, 720p, 1080p, 4k【 New 】	480p, 720p	480p, 720p	480p, 720p, 1080p

模型		doubao-seedance-2.0	doubao-seedance-2.0-fast	doubao-seedance-2.0-mini【 New 】	doubao-seedance-1.5-pro
	输出宽高比	21:9, 16:9, 4:3, 1:1, 3:4, 9:16	21:9, 16:9, 4:3, 1:1, 3:4, 9:16	21:9, 16:9, 4:3, 1:1, 3:4, 9:16	21:9, 16:9, 4:3, 1:1, 3:4, 9:16
	输出时长	4~15 秒	4~15 秒	4~15 秒	4~12 秒
	输出视频格式	mp4	mp4	mp4	mp4
抵扣系数		· 输出视频分辨率为 480p, 720p  ◦ 输入包含视频: 140  ◦ 输入不含视频: 230  · 输出视频分辨率为 1080p  ◦ 输入包含视频: 155  ◦ 输入不含视频: 255  · 输出视频分辨率为 4k  ◦ 输入包含视频: 80  ◦ 输入不含视频: 130	· 输入包含视频: 110  · 输入不含视频: 185	· 输入包含视频: 70  · 输入不含视频: 115	· 无声视频: 36  · 有声视频: 72

生图模型

模型		doubao-seedream-5.0-lite
文生图		✓
文生组图		✓
单 / 多图生图		✓
单 / 多图生组图		✓
流式输出		✓
联网搜索		✓
模型参数	分辨率	2K, 3K, 4K
	输出格式	png, jpeg
	提示词优化模式	标准模式
	生成数量	输入的参考图数量 + 最终生成的图片数量 ≤ 15张
抵扣系数		1 张成功生成的图片 = 99 AFP

## 5.3 接入语音模型

方舟 Agent Plan 提供的语音模型支持**语音合成**（TTS）和**语音识别**（ASR）两大能力，可以通过 WebSocket 和 HTTP 协议接入使用语音服务。

### 核心配置

在接入语音模型前，需要准备以下信息：

· 专属 API Key：[获取专属 API Key](#)

开通管理

1

Agent Plan

Coding Plan

管理素材资产

套餐信息

2

使用配置

用量统计

配置文档

套餐类型

套餐限额

订购状态

开始时间

结束时间

剩余时间

2000 AFP/5小时

7000 AFP/周

20000 AFP/月

计算规则详见 [说明文档](#)

生效中

2026-05-07 20:43

2026-06-07 23:59

31天

3

配置专属API Key

4

配置模型及Base URL

配置Harness（可选）

接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key

创建时间

操作

ark-ef

2026-05-07 20:43

更新API KEY

· 支持的模型：

◦ 豆包语音合成模型2.0（doubao-seed-tts-2.0）：接口中参数 `X-API-Resource-Id` 需配置为 `seed-tts-2.0`

◦ 豆包流式语音识别模型2.0（doubao-seed-asr-2.0）：接口中参数 `X-API-Resource-Id` 需配置为 `volc.seedasr.sauc.duration`

· 接口信息：[流式语音合成（Streaming TTS）](#)、[流式语音识别（ASR Streaming）](#)

### 流式语音合成（Streaming TTS）

流式语音合成支持将文本实时转换为语音，适用于实时文本流输入、音频内容生成场景。

#### 接口信息

根据具体场景选择合适的语音合成 API，对应的参数及细节可参见对应的 API 文档。

接口类型	接口协议	接口地址	推荐场景	API 文档
双流接口	WebSocket	wss:// openspeech.byteda nce.com/api/v3/ plan/tts/ bidirection	支持流式发送文 本、流式接收音 频，适合实时对 话等低延迟场 景。	<a href="#">WebSocket 双向 流式文档</a>
单流接口	WebSocket	wss:// openspeech.byteda	一次性发送全部 文本，流式接收	<a href="#">WebSocket 单向 流式文档</a>

接口类型	接口协议	接口地址	推荐场景	API 文档
		nce.com/api/v3/ plan/tts/ unidirectional/ stream	音频片段，适合 长文本播报。	
HTTP 接口	HTTP POST	https:// openspeech.byteda nce.com/api/v3/ plan/tts/ unidirectional	一次性发送文 本，一次性返回 完整音频，适合 简单场景或非实 时调用。	<a href="#">HTTP Chunked 单向流式文档</a> <a href="#">🔗</a>

## 代码示例

- 注意
- 代码示例中 `X-API-Key` 需要设置为[专属 API Key](#) [🔗](#)。
  - 在运行[双流接口](#)、[单流接口](#)的代码示例前，请先创建 `protocols.py` 文件。该文件封装了与语音合成服务端进行 WebSocket 通信的协议与消息处理逻辑，代码如下：  
[【附件下载】: protocols.py](#)，大小为

## 双流接口

```
import copy
import json
import logging
import uuid
import asyncio

import websockets

from protocols import (
 EventType,
 MessageType,
 finish_connection,
 finish_session,
 receive_message,
 start_connection,
 start_session,
 task_request,
 wait_for_event,
)

logging.basicConfig(level=logging.INFO)
```

```
logger = logging.getLogger(__name__)
logging.getLogger('protocols').setLevel(logging.WARNING)

URL = "wss://openspeech.bytedance.com/api/v3/plan/tts/bidirection"

async def main():
 # 要合成的文本
 text = "你好，欢迎使用语音合成服务。"

 # 连接服务器
 headers = {
 "X-API-Key": "your_api_key",
 "X-API-Resource-Id": "seed-tts-2.0",
 "X-API-Connect-Id": str(uuid.uuid4()),
 "X-Control-Require-Usage-Tokens-Return": "*"
 }

 logger.info(f"Connecting to {URL}")
 websocket = await websockets.connect(
 URL, additional_headers=headers, max_size=10 * 1024 * 1024
)
 logger.info(
 f"Connected to WebSocket server, Logid: {websocket.response.headers['x-tt-logid']}"
)

 try:
 # 开始连接
 await start_connection(websocket)
 await wait_for_event(
 websocket, MsgType.FullServerResponse, EventType.ConnectionStarted
)

 # 按句子分割处理
 sentences = text.split("。")
 audio_received = False

 for i, sentence in enumerate(sentences):
 if not sentence:
 continue

 # 每次会话可以有不同参数
 base_request = {
 "req_params": {
 "text": sentence,
 "speaker": "zh_female_gaolengyujie_uranus_bigtts",
 "audio_params": {
 "format": "mp3",
```

```
 "sample_rate": 24000,
 "enable_timestamp": False,
 }
},
}

开始会话
start_session_request = copy.deepcopy(base_request)
start_session_request["event"] = EventType.StartSession
session_id = str(uuid.uuid4())
await start_session(
 websocket, json.dumps(start_session_request).encode(), session_id
)
await wait_for_event(
 websocket, MsgType.FullServerResponse, EventType.SessionStarted
)

逐字发送
async def send_chars():
 for char in sentence:
 synthesis_request = copy.deepcopy(base_request)
 synthesis_request["event"] = EventType.TaskRequest
 synthesis_request["req_params"]["text"] = char
 await task_request(
 websocket, json.dumps(synthesis_request).encode(), session_id
)
 await asyncio.sleep(0.005) # 每个字符间隔5ms

 await finish_session(websocket, session_id)

在后台开始发送字符
send_task = asyncio.create_task(send_chars())

接收音频数据
audio_data = bytearray()
while True:
 msg = await receive_message(websocket)

 if msg.type == MsgType.FullServerResponse:
 if msg.event == EventType.SessionFinished:
 break
 if msg.payload:
 payload = json.loads(msg.payload)
 logger.info(f"Server response: {payload}")
 elif msg.type == MsgType.AudioOnlyServer:
 if not audio_received and len(audio_data) > 0:
 audio_received = True
```

```
Python audio_data.extend(msg.payload)
 else:
 raise RuntimeError(f"TTS conversion failed: {msg}")

 # 等待 send_chars 完成
 await send_task

 # 保存音频文件
 if audio_data:
 filename = f"bidirectional_session_{i}.mp3"
 with open(filename, "wb") as f:
 f.write(audio_data)
 logger.info(f"Audio received: {len(audio_data)}, saved to {filename}")

 if not audio_received:
 raise RuntimeError("No audio data received")

 finally:
 # 结束连接
 await finish_connection(websocket)
 msg = await wait_for_event(
 websocket, MsgType.FullServerResponse, EventType.ConnectionFinished
)
 if msg.payload:
 payload = json.loads(msg.payload)
 if "usage" in payload:
 logger.info(f"Usage: {payload['usage']}")
 await websocket.close()
 logger.info("Connection closed")

if __name__ == "__main__":
 asyncio.run(main())
```

## 单流接口

```
import asyncio
import json
import logging
import uuid

import websockets

from protocols import EventType, MsgType, full_client_request, receive_message
```



```
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def main() -> None:
 headers = {
 "X-API-Key": "your_api_key",
 "X-API-Resource-Id": "seed-tts-2.0",
 "X-Control-Require-Usage-Tokens-Return": "*"
 }

 body = {
 "req_params": {
 "speaker": "zh_female_gaolengyujie_uranus_bigtts",
 "text": "这是一段较长的文本内容，单向流式会一次发送全部文本，然后流式返回音频片
段。",
 "audio_params": {
 "format": "mp3",
 "sample_rate": 24000,
 }
 }
 }

 websocket = await websockets.connect(
 "wss://openspeech.bytedance.com/api/v3/plan/tts/unidirectional/stream",
 additional_headers=headers,
 max_size=10 * 1024 * 1024,
)
 logger.info(
 f"Connected to WebSocket server, Logid: {websocket.response.headers.get('x-tt-logid', '')}"
)

 try:
 await full_client_request(websocket, json.dumps(body).encode())

 audio_data = bytearray()
 while True:
 msg = await receive_message(websocket)

 if msg.type == MsgType.FullServerResponse and msg.event == EventType.SessionFinished:
 break
 if msg.type == MsgType.AudioOnlyServer and msg.payload:
 audio_data.extend(msg.payload)
 elif msg.type == MsgType.Error:
 raise RuntimeError(f"TTS conversion failed: {msg}")

 if not audio_data:
 raise RuntimeError("No audio data received")
```

Python

```
with open("unidirectional_stream.mp3", "wb") as f:
 f.write(audio_data)
 logger.info(f"Audio received: {len(audio_data)}, saved to unidirectional_stream.mp3")
finally:
 await websocket.close()
 logger.info("Connection closed")

if __name__ == "__main__":
 asyncio.run(main())
```

## HTTP 接口

```
import requests
import json
import base64
import os

url = "https://openspeech.bytedance.com/api/v3/plan/tts/unidirectional"

def tts_http_stream():
 headers = {
 "X-API-Key": "your_api_key",
 "X-API-Resource-Id": "seed-tts-2.0",
 "Content-Type": "application/json",
 "Connection": "keep-alive",
 "X-Control-Require-Usage-Tokens-Return": "*"
 }

 payload = {
 "req_params": {
 "text": "你好，这是通过 HTTP 接口合成的语音。",
 "speaker": "zh_female_vv_uranus_big tts",
 "audio_params": {
 "format": "mp3",
 "sample_rate": 24000,
 }
 }
 }
 session = requests.Session()
 response = None
 try:
 response = session.post(url, headers=headers, json=payload, stream=True)
```

```
Python
audio_data = bytearray()
total_audio_size = 0
for chunk in response.iter_lines(decode_unicode=True):
 if not chunk:
 continue
 data = json.loads(chunk)
 print(f"json data:{data}")
 if data.get("code", 0) == 0 and "data" in data and data["data"]:
 chunk_audio = base64.b64decode(data["data"])
 audio_size = len(chunk_audio)
 total_audio_size += audio_size
 audio_data.extend(chunk_audio)
 if data.get("code", 0) == 20000000:
 break
 if data.get("code", 0) > 0:
 print(f"error response:{data}")
 break

if audio_data:
 if not os.path.exists("tts"):
 os.makedirs("tts")
 output_file = os.path.join("tts/", f"tts_test.mp3")
 with open(output_file, "wb") as f:
 f.write(audio_data)
 print(f"file size: {len(audio_data) / 1024:.2f} KB")

 os.chmod(output_file, 0o644)

except Exception as e:
 print(f"request error: {e}")
finally:
 if response:
 response.close()
 session.close()

if __name__ == "__main__":
 tts_http_stream()
```

## 流式语音识别 ( ASR Streaming )

流式语音识别支持通过 WebSocket 将实时语音流转写为文本，适用于实时转写、语音助手等场景。

### 接口信息

根据推荐场景选择对应的模式，对应的参数及交互流程可参见对应的 API 文档。

接口类型	接口协议	接口地址	推荐场景	API 文档
双流接口	WebSocket	wss:// openspeech.byteda nce.com/api/v3/ plan/sauc/ bigmodel_async	边发送音频边实时返回识别结果，适合实时转写、语音助手等低延迟场景。	大模型流式语音识别 API <a href="#">↗</a>
单流接口	WebSocket	wss:// openspeech.byteda nce.com/api/v3/ plan/sauc/ bigmodel_nostream	流式发送音频，待全部发送完成或超过 15s 后统一返回高精度结果，适合准确率优先的场景。	大模型流式语音识别 API <a href="#">↗</a>

### 代码示例

**注意**  
代码示例中 `X-API-Key` 需要设置为[专属 API Key](#) [↗](#)。

#### 双流接口

```
import asyncio
import aiohttp
import json
import struct
import gzip
import uuid
import logging
import os
import subprocess
from typing import Optional, List, Dict, Any, Tuple, AsyncGenerator

配置日志
logging.basicConfig(
 level=logging.INFO,
 format='%(asctime)s - %(levelname)s - %(message)s',
 handlers=[
 logging.FileHandler('run.log'),
 logging.StreamHandler()
]
)
logger = logging.getLogger(__name__)
```

```
常量定义
DEFAULT_SAMPLE_RATE = 16000

class ProtocolVersion:
 V1 = 0b0001

class MessageType:
 CLIENT_FULL_REQUEST = 0b0001
 CLIENT_AUDIO_ONLY_REQUEST = 0b0010
 SERVER_FULL_RESPONSE = 0b1001
 SERVER_ERROR_RESPONSE = 0b1111

class MessageTypeSpecificFlags:
 NO_SEQUENCE = 0b0000
 POS_SEQUENCE = 0b0001
 NEG_SEQUENCE = 0b0010
 NEG_WITH_SEQUENCE = 0b0011

class SerializationType:
 NO_SERIALIZATION = 0b0000
 JSON = 0b0001

class CompressionType:
 GZIP = 0b0001

class Config:
 def __init__(self):
 # 填入新版控制台获取的 API Key 和 Resource ID
 self.api_key = "your_api_key"
 self.resource_id = "volc.seedasr.sauc.duration"

config = Config()

class CommonUtils:
 @staticmethod
 def gzip_compress(data: bytes) -> bytes:
 return gzip.compress(data)

 @staticmethod
 def gzip_decompress(data: bytes) -> bytes:
 return gzip.decompress(data)

 @staticmethod
 def judge_wav(data: bytes) -> bool:
 if len(data) < 44:
 return False
 return data[:4] == b'RIFF' and data[8:12] == b'WAVE'
```

```

@staticmethod
def convert_wav_with_path(audio_path: str, sample_rate: int = DEFAULT_SAMPLE_RATE) -> bytes:
 try:
 cmd = [
 "ffmpeg", "-v", "quiet", "-y", "-i", audio_path,
 "-acodec", "pcm_s16le", "-ac", "1", "-ar", str(sample_rate),
 "-f", "wav", "-"
]
 result = subprocess.run(cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

 return result.stdout
 except subprocess.CalledProcessError as e:
 logger.error(f"FFmpeg conversion failed: {e.stderr.decode()}")
 raise RuntimeError(f"Audio conversion failed: {e.stderr.decode()}")

@staticmethod
def read_wav_info(data: bytes) -> Tuple[int, int, int, int, bytes]:
 if len(data) < 44:
 raise ValueError("Invalid WAV file: too short")

 # 解析WAV头
 chunk_id = data[:4]
 if chunk_id != b'RIFF':
 raise ValueError("Invalid WAV file: not RIFF format")

 format_ = data[8:12]
 if format_ != b'WAVE':
 raise ValueError("Invalid WAV file: not WAVE format")

 # 解析fmt子块
 audio_format = struct.unpack('<H', data[20:22])[0]
 num_channels = struct.unpack('<H', data[22:24])[0]
 sample_rate = struct.unpack('<I', data[24:28])[0]
 bits_per_sample = struct.unpack('<H', data[34:36])[0]

 # 查找data子块
 pos = 36
 while pos < len(data) - 8:
 subchunk_id = data[pos:pos+4]
 subchunk_size = struct.unpack('<I', data[pos+4:pos+8])[0]
 if subchunk_id == b'data':
 wave_data = data[pos+8:pos+8+subchunk_size]
 return (
 num_channels,
 bits_per_sample // 8,
 sample_rate,
)
 pos += 4 + subchunk_size

```

```
 subchunk_size // (num_channels * (bits_per_sample // 8)),
 wave_data
)
 pos += 8 + subchunk_size
```

```
raise ValueError("Invalid WAV file: no data subchunk found")
```

```
class AsrRequestHeader:
```

```
 def __init__(self):
```

```
 self.message_type = MessageType.CLIENT_FULL_REQUEST
```

```
 self.message_type_specific_flags = MessageTypeSpecificFlags.POS_SEQUENCE
```

```
 self.serialization_type = SerializationType.JSON
```

```
 self.compression_type = CompressionType.GZIP
```

```
 self.reserved_data = bytes([0x00])
```

```
 def with_message_type(self, message_type: int) -> 'AsrRequestHeader':
```

```
 self.message_type = message_type
```

```
 return self
```

```
 def with_message_type_specific_flags(self, flags: int) -> 'AsrRequestHeader':
```

```
 self.message_type_specific_flags = flags
```

```
 return self
```

```
 def with_serialization_type(self, serialization_type: int) -> 'AsrRequestHeader':
```

```
 self.serialization_type = serialization_type
```

```
 return self
```

```
 def with_compression_type(self, compression_type: int) -> 'AsrRequestHeader':
```

```
 self.compression_type = compression_type
```

```
 return self
```

```
 def with_reserved_data(self, reserved_data: bytes) -> 'AsrRequestHeader':
```

```
 self.reserved_data = reserved_data
```

```
 return self
```

```
 def to_bytes(self) -> bytes:
```

```
 header = bytearray()
```

```
 header.append((ProtocolVersion.V1 << 4) | 1)
```

```
 header.append((self.message_type << 4) | self.message_type_specific_flags)
```

```
 header.append((self.serialization_type << 4) | self.compression_type)
```

```
 header.extend(self.reserved_data)
```

```
 return bytes(header)
```

```
@staticmethod
```

```
def default_header() -> 'AsrRequestHeader':
```

```
 return AsrRequestHeader()
```

```
class RequestBuilder:
 @staticmethod
 def new_auth_headers() -> Dict[str, str]:
 reqid = str(uuid.uuid4())
 return {
 "X-API-Key": config.api_key,
 "X-API-Resource-Id": config.resource_id,
 "X-API-Request-Id": reqid,
 "X-API-Connect-Id": reqid,
 "X-API-Sequence": "-1",
 }

 @staticmethod
 def new_full_client_request(seq: int) -> bytes: # 添加seq参数
 header = AsrRequestHeader.default_header() \
 .with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)

 payload = {
 "user": {
 "uid": "demo_uid"
 },
 "audio": {
 "format": "wav",
 "codec": "raw",
 "rate": 16000,
 "bits": 16,
 "channel": 1
 },
 "request": {
 "model_name": "bigmodel",
 "enable_itn": True,
 "enable_punc": True,
 "enable_ddc": True,
 "show_utterances": True,
 "enable_nonstream": False
 }
 }

 payload_bytes = json.dumps(payload).encode('utf-8')
 compressed_payload = CommonUtils.gzip_compress(payload_bytes)
 payload_size = len(compressed_payload)

 request = bytearray()
 request.extend(header.to_bytes())
 request.extend(struct.pack('>i', seq)) # 使用传入的seq
 request.extend(struct.pack('>I', payload_size))
 request.extend(compressed_payload)
```



```
 return bytes(request)

 @staticmethod
 def new_audio_only_request(seq: int, segment: bytes, is_last: bool = False) -> bytes:
 header = AsrRequestHeader.default_header()
 if is_last: # 最后一个包特殊处理
 header.with_message_type_specific_flags(MessageTypeSpecificFlags.NEG_WITH_SEQUENCE)
 seq = -seq # 设为负值
 else:
 header.with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)
 header.with_message_type(MessageType.CLIENT_AUDIO_ONLY_REQUEST)

 request = bytearray()
 request.extend(header.to_bytes())
 request.extend(struct.pack('>i', seq))

 compressed_segment = CommonUtils.gzip_compress(segment)
 request.extend(struct.pack('>I', len(compressed_segment)))
 request.extend(compressed_segment)

 return bytes(request)

class AsrResponse:
 def __init__(self):
 self.code = 0
 self.event = 0
 self.is_last_package = False
 self.payload_sequence = 0
 self.payload_size = 0
 self.payload_msg = None

 def to_dict(self) -> Dict[str, Any]:
 return {
 "code": self.code,
 "event": self.event,
 "is_last_package": self.is_last_package,
 "payload_sequence": self.payload_sequence,
 "payload_size": self.payload_size,
 "payload_msg": self.payload_msg
 }

class ResponseParser:
 @staticmethod
 def parse_response(msg: bytes) -> AsrResponse:
 response = AsrResponse()
```

```
header_size = msg[0] & 0x0f
message_type = msg[1] >> 4
message_type_specific_flags = msg[1] & 0x0f
serialization_method = msg[2] >> 4
message_compression = msg[2] & 0x0f

payload = msg[header_size*4:]

解析message_type_specific_flags
if message_type_specific_flags & 0x01:
 response.payload_sequence = struct.unpack('>i', payload[:4])[0]
 payload = payload[4:]
if message_type_specific_flags & 0x02:
 response.is_last_package = True
if message_type_specific_flags & 0x04:
 response.event = struct.unpack('>i', payload[:4])[0]
 payload = payload[4:]

解析message_type
if message_type == MessageType.SERVER_FULL_RESPONSE:
 response.payload_size = struct.unpack('>i', payload[:4])[0]
 payload = payload[4:]
elif message_type == MessageType.SERVER_ERROR_RESPONSE:
 response.code = struct.unpack('>i', payload[:4])[0]
 response.payload_size = struct.unpack('>i', payload[4:8])[0]
 payload = payload[8:]

if not payload:
 return response

解压缩
if message_compression == CompressionType.GZIP:
 try:
 payload = CommonUtils.gzip_decompress(payload)
 except Exception as e:
 logger.error(f"Failed to decompress payload: {e}")
 return response

解析payload
try:
 if serialization_method == SerializationType.JSON:
 response.payload_msg = json.loads(payload.decode('utf-8'))
except Exception as e:
 logger.error(f"Failed to parse payload: {e}")

return response
```

```
class AsrWsClient:
 def __init__(self, url: str, segment_duration: int = 200):
 self.seq = 1
 self.url = url
 self.segment_duration = segment_duration
 self.conn = None
 self.session = None # 添加session引用

 async def __aenter__(self):
 self.session = aiohttp.ClientSession()
 return self

 async def __aexit__(self, exc_type, exc, tb):
 if self.conn and not self.conn.closed:
 await self.conn.close()
 if self.session and not self.session.closed:
 await self.session.close()

 async def read_audio_data(self, file_path: str) -> bytes:
 try:
 with open(file_path, 'rb') as f:
 content = f.read()

 if not CommonUtils.judge_wav(content):
 logger.info("Converting audio to WAV format...")
 content = CommonUtils.convert_wav_with_path(file_path, DEFAULT_SAMPLE_RATE)

 return content
 except Exception as e:
 logger.error(f"Failed to read audio data: {e}")
 raise

 def get_segment_size(self, content: bytes) -> int:
 try:
 channel_num, samp_width, frame_rate, _, _ = CommonUtils.read_wav_info(content)[:5]
 size_per_sec = channel_num * samp_width * frame_rate
 segment_size = size_per_sec * self.segment_duration // 1000
 return segment_size
 except Exception as e:
 logger.error(f"Failed to calculate segment size: {e}")
 raise

 async def create_connection(self) -> None:
 headers = RequestBuilder.new_auth_headers()
 try:
 self.conn = await self.session.ws_connect(# 使用self.session
 self.url,
```

```

 headers=headers
)
 logger.info(f"Connected to {self.url}")
except Exception as e:
 logger.error(f"Failed to connect to WebSocket: {e}")
 raise

async def send_full_client_request(self) -> None:
 request = RequestBuilder.new_full_client_request(self.seq)
 self.seq += 1 # 发送后递增
 try:
 await self.conn.send_bytes(request)
 logger.info(f"Sent full client request with seq: {self.seq-1}")

 msg = await self.conn.receive()
 if msg.type == aiohttp.WSMsgType.BINARY:
 response = ResponseParser.parse_response(msg.data)
 logger.info(f"Received response: {response.to_dict()}")
 else:
 logger.error(f"Unexpected message type: {msg.type}")
 except Exception as e:
 logger.error(f"Failed to send full client request: {e}")
 raise

async def send_messages(self, segment_size: int, content: bytes) -> AsyncGenerator[None, None]:
 audio_segments = self.split_audio(content, segment_size)
 total_segments = len(audio_segments)

 for i, segment in enumerate(audio_segments):
 is_last = (i == total_segments - 1)
 request = RequestBuilder.new_audio_only_request(
 self.seq,
 segment,
 is_last=is_last
)
 await self.conn.send_bytes(request)
 logger.info(f"Sent audio segment with seq: {self.seq} (last: {is_last})")

 if not is_last:
 self.seq += 1

 await asyncio.sleep(self.segment_duration / 1000) # 逐个发送，间隔时间模拟实时流
 # 让出控制权，允许接受消息
 yield

async def recv_messages(self) -> AsyncGenerator[AsrResponse, None]:
 try:

```

```
async for msg in self.conn:
 if msg.type == aiohttp.WSMsgType.BINARY:
 response = ResponseParser.parse_response(msg.data)
 yield response

 if response.is_last_package or response.code != 0:
 break
 elif msg.type == aiohttp.WSMsgType.ERROR:
 logger.error(f"WebSocket error: {msg.data}")
 break
 elif msg.type == aiohttp.WSMsgType.CLOSED:
 logger.info("WebSocket connection closed")
 break
except Exception as e:
 logger.error(f"Error receiving messages: {e}")
 raise

async def start_audio_stream(self, segment_size: int, content: bytes) ->
AsyncGenerator[AsrResponse, None]:
 async def sender():
 async for _ in self.send_messages(segment_size, content):
 pass

 # 启动发送和接收任务
 sender_task = asyncio.create_task(sender())

 try:
 async for response in self.recv_messages():
 yield response
 finally:
 sender_task.cancel()
 try:
 await sender_task
 except asyncio.CancelledError:
 pass

 @staticmethod
 def split_audio(data: bytes, segment_size: int) -> List[bytes]:
 if segment_size <= 0:
 return []

 segments = []
 for i in range(0, len(data), segment_size):
 end = i + segment_size
 if end > len(data):
 end = len(data)
 segments.append(data[i:end])
```

```
return segments

async def execute(self, file_path: str) -> AsyncGenerator[AsrResponse, None]:
 if not file_path:
 raise ValueError("File path is empty")

 if not self.url:
 raise ValueError("URL is empty")

 self.seq = 1

 try:
 # 1. 读取音频文件
 content = await self.read_audio_data(file_path)

 # 2. 计算分段大小
 segment_size = self.get_segment_size(content)

 # 3. 创建WebSocket连接
 await self.create_connection()

 # 4. 发送完整客户端请求
 await self.send_full_client_request()

 # 5. 启动音频流处理
 async for response in self.start_audio_stream(segment_size, content):
 yield response

 except Exception as e:
 logger.error(f"Error in ASR execution: {e}")
 raise
 finally:
 if self.conn:
 await self.conn.close()

async def main():
 # 配置信息
 audio_file_path = "your_file_path" # 替换为你的音频文件路径
 ws_url = "wss://openspeech.bytedance.com/api/v3/plan/sauc/bigmodel_async"
 seg_duration = 200 # 每次发送的音频片段时长(ms)

 async with AsrWsClient(ws_url, seg_duration) as client:
 try:
 async for response in client.execute(audio_file_path):
 logger.info(f"Received response: {json.dumps(response.to_dict(), indent=2, ensure_ascii=False)}")
 except Exception as e:
```

```
Python logger.error(f"ASR processing failed: {e}")

if __name__ == "__main__":
 asyncio.run(main())
```

## 单流接口

```
import asyncio
import aiohttp
import json
import struct
import gzip
import uuid
import logging
import os
import subprocess
from typing import Optional, List, Dict, Any, Tuple, AsyncGenerator

配置日志
logging.basicConfig(
 level=logging.INFO,
 format='%(asctime)s - %(levelname)s - %(message)s',
 handlers=[
 logging.FileHandler('run.log'),
 logging.StreamHandler()
]
)
logger = logging.getLogger(__name__)

常量定义
DEFAULT_SAMPLE_RATE = 16000

class ProtocolVersion:
 V1 = 0b0001

class MessageType:
 CLIENT_FULL_REQUEST = 0b0001
 CLIENT_AUDIO_ONLY_REQUEST = 0b0010
 SERVER_FULL_RESPONSE = 0b1001
 SERVER_ERROR_RESPONSE = 0b1111

class MessageTypeSpecificFlags:
 NO_SEQUENCE = 0b0000
 POS_SEQUENCE = 0b0001
```

```
NEG_SEQUENCE = 0b0010
NEG_WITH_SEQUENCE = 0b0011
```

```
class SerializationType:
 NO_SERIALIZATION = 0b0000
 JSON = 0b0001
```

```
class CompressionType:
 GZIP = 0b0001
```

```
class Config:
 def __init__(self):
 # 填入新版控制台获取的 API Key 和 Resource ID
 self.api_key = "your_api_key"
 self.resource_id = "volc.seedasr.sauc.duration"
```

```
config = Config()
```

```
class CommonUtils:
 @staticmethod
 def gzip_compress(data: bytes) -> bytes:
 return gzip.compress(data)

 @staticmethod
 def gzip_decompress(data: bytes) -> bytes:
 return gzip.decompress(data)

 @staticmethod
 def judge_wav(data: bytes) -> bool:
 if len(data) < 44:
 return False
 return data[:4] == b'RIFF' and data[8:12] == b'WAVE'

 @staticmethod
 def convert_wav_with_path(audio_path: str, sample_rate: int = DEFAULT_SAMPLE_RATE) -> bytes:
 try:
 cmd = [
 "ffmpeg", "-v", "quiet", "-y", "-i", audio_path,
 "-acodec", "pcm_s16le", "-ac", "1", "-ar", str(sample_rate),
 "-f", "wav", "-"
]
 result = subprocess.run(cmd, check=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE)

 return result.stdout
 except subprocess.CalledProcessError as e:
 logger.error(f"FFmpeg conversion failed: {e.stderr.decode()}")
 raise RuntimeError(f"Audio conversion failed: {e.stderr.decode()}")
```



```
@staticmethod
def read_wav_info(data: bytes) -> Tuple[int, int, int, int, bytes]:
 if len(data) < 44:
 raise ValueError("Invalid WAV file: too short")

 # 解析WAV头
 chunk_id = data[:4]
 if chunk_id != b'RIFF':
 raise ValueError("Invalid WAV file: not RIFF format")

 format_ = data[8:12]
 if format_ != b'WAVE':
 raise ValueError("Invalid WAV file: not WAVE format")

 # 解析fmt子块
 audio_format = struct.unpack('<H', data[20:22])[0]
 num_channels = struct.unpack('<H', data[22:24])[0]
 sample_rate = struct.unpack('<I', data[24:28])[0]
 bits_per_sample = struct.unpack('<H', data[34:36])[0]

 # 查找data子块
 pos = 36
 while pos < len(data) - 8:
 subchunk_id = data[pos:pos+4]
 subchunk_size = struct.unpack('<I', data[pos+4:pos+8])[0]
 if subchunk_id == b'data':
 wave_data = data[pos+8:pos+8+subchunk_size]
 return (
 num_channels,
 bits_per_sample // 8,
 sample_rate,
 subchunk_size // (num_channels * (bits_per_sample // 8)),
 wave_data
)
 pos += 8 + subchunk_size

 raise ValueError("Invalid WAV file: no data subchunk found")

class AsrRequestHeader:
 def __init__(self):
 self.message_type = MessageType.CLIENT_FULL_REQUEST
 self.message_type_specific_flags = MessageTypeSpecificFlags.POS_SEQUENCE
 self.serialization_type = SerializationType.JSON
 self.compression_type = CompressionType.GZIP
 self.reserved_data = bytes([0x00])
```

```
def with_message_type(self, message_type: int) -> 'AsrRequestHeader':
 self.message_type = message_type
 return self

def with_message_type_specific_flags(self, flags: int) -> 'AsrRequestHeader':
 self.message_type_specific_flags = flags
 return self

def with_serialization_type(self, serialization_type: int) -> 'AsrRequestHeader':
 self.serialization_type = serialization_type
 return self

def with_compression_type(self, compression_type: int) -> 'AsrRequestHeader':
 self.compression_type = compression_type
 return self

def with_reserved_data(self, reserved_data: bytes) -> 'AsrRequestHeader':
 self.reserved_data = reserved_data
 return self

def to_bytes(self) -> bytes:
 header = bytearray()
 header.append((ProtocolVersion.V1 << 4) | 1)
 header.append((self.message_type << 4) | self.message_type_specific_flags)
 header.append((self.serialization_type << 4) | self.compression_type)
 header.extend(self.reserved_data)
 return bytes(header)

@staticmethod
def default_header() -> 'AsrRequestHeader':
 return AsrRequestHeader()

class RequestBuilder:
 @staticmethod
 def new_auth_headers() -> Dict[str, str]:
 reqid = str(uuid.uuid4())
 return {
 "X-API-Key": config.api_key,
 "X-API-Resource-Id": config.resource_id,
 "X-API-Request-Id": reqid,
 "X-API-Connect-Id": reqid,
 "X-API-Sequence": "-1",
 }

 @staticmethod
 def new_full_client_request(seq: int) -> bytes: # 添加seq参数
 header = AsrRequestHeader.default_header() \
```

```
.with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)

payload = {
 "user": {
 "uid": "demo_uid"
 },
 "audio": {
 "format": "wav",
 "codec": "raw",
 "rate": 16000,
 "bits": 16,
 "channel": 1
 },
 "request": {
 "model_name": "bigmodel",
 "enable_itn": True,
 "enable_punc": True,
 "enable_ddc": True,
 "show_utterances": True,
 "enable_nonstream": False
 }
}

payload_bytes = json.dumps(payload).encode('utf-8')
compressed_payload = CommonUtils.gzip_compress(payload_bytes)
payload_size = len(compressed_payload)

request = bytearray()
request.extend(header.to_bytes())
request.extend(struct.pack('>i', seq)) # 使用传入的seq
request.extend(struct.pack('>I', payload_size))
request.extend(compressed_payload)

return bytes(request)

@staticmethod
def new_audio_only_request(seq: int, segment: bytes, is_last: bool = False) -> bytes:
 header = AsrRequestHeader.default_header()
 if is_last: # 最后一个包特殊处理
 header.with_message_type_specific_flags(MessageTypeSpecificFlags.NEG_WITH_SEQUENCE)
 seq = -seq # 设为负值
 else:
 header.with_message_type_specific_flags(MessageTypeSpecificFlags.POS_SEQUENCE)
 header.with_message_type(MessageType.CLIENT_AUDIO_ONLY_REQUEST)

 request = bytearray()
 request.extend(header.to_bytes())
```

```
request.extend(struct.pack('>i', seq))

compressed_segment = CommonUtils.gzip_compress(segment)
request.extend(struct.pack('>I', len(compressed_segment)))
request.extend(compressed_segment)

return bytes(request)

class AsrResponse:
 def __init__(self):
 self.code = 0
 self.event = 0
 self.is_last_package = False
 self.payload_sequence = 0
 self.payload_size = 0
 self.payload_msg = None

 def to_dict(self) -> Dict[str, Any]:
 return {
 "code": self.code,
 "event": self.event,
 "is_last_package": self.is_last_package,
 "payload_sequence": self.payload_sequence,
 "payload_size": self.payload_size,
 "payload_msg": self.payload_msg
 }

class ResponseParser:
 @staticmethod
 def parse_response(msg: bytes) -> AsrResponse:
 response = AsrResponse()

 header_size = msg[0] & 0x0f
 message_type = msg[1] >> 4
 message_type_specific_flags = msg[1] & 0x0f
 serialization_method = msg[2] >> 4
 message_compression = msg[2] & 0x0f

 payload = msg[header_size*4:]

 # 解析message_type_specific_flags
 if message_type_specific_flags & 0x01:
 response.payload_sequence = struct.unpack('>i', payload[:4])[0]
 payload = payload[4:]
 if message_type_specific_flags & 0x02:
 response.is_last_package = True
 if message_type_specific_flags & 0x04:
```

```
response.event = struct.unpack('>i', payload[:4])[0]
payload = payload[4:]

解析message_type
if message_type == MessageType.SERVER_FULL_RESPONSE:
 response.payload_size = struct.unpack('>i', payload[:4])[0]
 payload = payload[4:]
elif message_type == MessageType.SERVER_ERROR_RESPONSE:
 response.code = struct.unpack('>i', payload[:4])[0]
 response.payload_size = struct.unpack('>i', payload[4:8])[0]
 payload = payload[8:]

if not payload:
 return response

解压缩
if message_compression == CompressionType.GZIP:
 try:
 payload = CommonUtils.gzip_decompress(payload)
 except Exception as e:
 logger.error(f"Failed to decompress payload: {e}")
 return response

解析payload
try:
 if serialization_method == SerializationType.JSON:
 response.payload_msg = json.loads(payload.decode('utf-8'))
except Exception as e:
 logger.error(f"Failed to parse payload: {e}")

return response

class AsrWsClient:
 def __init__(self, url: str, segment_duration: int = 200):
 self.seq = 1
 self.url = url
 self.segment_duration = segment_duration
 self.conn = None
 self.session = None # 添加session引用

 async def __aenter__(self):
 self.session = aiohttp.ClientSession()
 return self

 async def __aexit__(self, exc_type, exc, tb):
 if self.conn and not self.conn.closed:
 await self.conn.close()
```

```
if self.session and not self.session.closed:
 await self.session.close()

async def read_audio_data(self, file_path: str) -> bytes:
 try:
 with open(file_path, 'rb') as f:
 content = f.read()

 if not CommonUtils.judge_wav(content):
 logger.info("Converting audio to WAV format...")
 content = CommonUtils.convert_wav_with_path(file_path, DEFAULT_SAMPLE_RATE)

 return content
 except Exception as e:
 logger.error(f"Failed to read audio data: {e}")
 raise

def get_segment_size(self, content: bytes) -> int:
 try:
 channel_num, samp_width, frame_rate, _, _ = CommonUtils.read_wav_info(content)[:5]
 size_per_sec = channel_num * samp_width * frame_rate
 segment_size = size_per_sec * self.segment_duration // 1000
 return segment_size
 except Exception as e:
 logger.error(f"Failed to calculate segment size: {e}")
 raise

async def create_connection(self) -> None:
 headers = RequestBuilder.new_auth_headers()
 try:
 self.conn = await self.session.ws_connect(# 使用self.session
 self.url,
 headers=headers
)
 logger.info(f"Connected to {self.url}")
 except Exception as e:
 logger.error(f"Failed to connect to WebSocket: {e}")
 raise

async def send_full_client_request(self) -> None:
 request = RequestBuilder.new_full_client_request(self.seq)
 self.seq += 1 # 发送后递增
 try:
 await self.conn.send_bytes(request)
 logger.info(f"Sent full client request with seq: {self.seq-1}")

 msg = await self.conn.receive()
```

```

 if msg.type == aiohttp.WSMsgType.BINARY:
 response = ResponseParser.parse_response(msg.data)
 logger.info(f"Received response: {response.to_dict()}")
 else:
 logger.error(f"Unexpected message type: {msg.type}")
except Exception as e:
 logger.error(f"Failed to send full client request: {e}")
 raise

async def send_messages(self, segment_size: int, content: bytes) -> AsyncGenerator[None, None]:
 audio_segments = self.split_audio(content, segment_size)
 total_segments = len(audio_segments)

 for i, segment in enumerate(audio_segments):
 is_last = (i == total_segments - 1)
 request = RequestBuilder.new_audio_only_request(
 self.seq,
 segment,
 is_last=is_last
)
 await self.conn.send_bytes(request)
 logger.info(f"Sent audio segment with seq: {self.seq} (last: {is_last})")

 if not is_last:
 self.seq += 1

 await asyncio.sleep(self.segment_duration / 1000) # 逐个发送，间隔时间模拟实时流
 # 让出控制权，允许接受消息
 yield

async def rcv_messages(self) -> AsyncGenerator[AsrResponse, None]:
 try:
 async for msg in self.conn:
 if msg.type == aiohttp.WSMsgType.BINARY:
 response = ResponseParser.parse_response(msg.data)
 yield response

 if response.is_last_package or response.code != 0:
 break
 elif msg.type == aiohttp.WSMsgType.ERROR:
 logger.error(f"WebSocket error: {msg.data}")
 break
 elif msg.type == aiohttp.WSMsgType.CLOSED:
 logger.info("WebSocket connection closed")
 break
 except Exception as e:
 logger.error(f"Error receiving messages: {e}")

```

```
raise

async def start_audio_stream(self, segment_size: int, content: bytes) ->
AsyncGenerator[AsrResponse, None]:
 async def sender():
 async for _ in self.send_messages(segment_size, content):
 pass

 # 启动发送和接收任务
 sender_task = asyncio.create_task(sender())

 try:
 async for response in self.recv_messages():
 yield response
 finally:
 sender_task.cancel()
 try:
 await sender_task
 except asyncio.CancelledError:
 pass

 @staticmethod
 def split_audio(data: bytes, segment_size: int) -> List[bytes]:
 if segment_size <= 0:
 return []

 segments = []
 for i in range(0, len(data), segment_size):
 end = i + segment_size
 if end > len(data):
 end = len(data)
 segments.append(data[i:end])
 return segments

 async def execute(self, file_path: str) -> AsyncGenerator[AsrResponse, None]:
 if not file_path:
 raise ValueError("File path is empty")

 if not self.url:
 raise ValueError("URL is empty")

 self.seq = 1

 try:
 # 1. 读取音频文件
 content = await self.read_audio_data(file_path)
```



```
Python # 2. 计算分段大小
 segment_size = self.get_segment_size(content)

 # 3. 创建WebSocket连接
 await self.create_connection()

 # 4. 发送完整客户端请求
 await self.send_full_client_request()

 # 5. 启动音频流处理
 async for response in self.start_audio_stream(segment_size, content):
 yield response

 except Exception as e:
 logger.error(f"Error in ASR execution: {e}")
 raise
 finally:
 if self.conn:
 await self.conn.close()

async def main():
 # 配置信息
 audio_file_path = "your_file_path" # 替换为你的音频文件路径
 ws_url = "wss://openspeech.bytedance.com/api/v3/plan/sauc/bigmodel_nostream"
 seg_duration = 200 # 每次发送的音频片段时长(ms)

 async with AsrWsClient(ws_url, seg_duration) as client:
 try:
 async for response in client.execute(audio_file_path):
 logger.info(f"Received response: {json.dumps(response.to_dict(), indent=2,
ensure_ascii=False)}")
 except Exception as e:
 logger.error(f"ASR processing failed: {e}")

if __name__ == "__main__":
 asyncio.run(main())
```

## 接口接入建议

请求和响应参数：

- 始终记录 Response Header 中的 `X-Tt-Logid` ，便于问题排查。
- 为每次连接/请求生成唯一的 UUID 作为 `X-API-Connect-Id` 或 `X-API-Request-Id` 。

## 计费说明

- **未开启超额后付费：**语音模型调用产生的费用使用套餐内 AFP 进行抵扣，不消耗其他资源包或账户余额。抵扣规则参见[套餐内 AFP 抵扣规则](#)。
- **开启超额后付费：**在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息（如 Base URL、API Key、模型名称），系统将自动切换至超额后付费模式继续使用，并产生后付费账单。具体规则与操作参见[超额后付费规则](#)和[超额后付费管理](#)。

## 6. 配置 Harness

### 6.1 豆包搜索

豆包搜索是面向大模型构建的搜索服务，通过灵活 API 形式提供多垂类、多信源搜索能力，用于补充模型训练数据截止后的最新事实、核验信息出处、获取时效性内容，提升大模型回答的知识面和可信度。

#### 注意

2026年6月22日，Agent Plan 产品中的联网搜索（Beta版）服务正式升级为豆包搜索服务。建议已接入联网搜索（Beta版）的用户迁移至豆包搜索。

### 用户权益

#### 使用联网搜索（Beta版）的用户权益

2026年7月1日起，联网搜索（Beta版）的限时免费权益活动结束，每月免费额度发生变更。

- 变更前：根据下单的 Agent Plan 套餐类型发放每月免费额度，该额度专属于 Agent Plan 获取的搜索 API Key 调用。套餐到期时，下单附赠的免费搜索额度也将回收。
  - Small：每月赠送 50 次免费搜索额度。
  - Medium：每月赠送 150 次免费搜索额度。
  - Large：每月赠送 400 次免费搜索额度。
  - Max：每月赠送 800 次免费搜索额度。
- 变更后：以火山引擎主账号 ID 维度，每月发放 500 次免费额度（每月1日重置），免费额度不专属于 Agent Plan 获取的搜索 API Key 调用，在火山引擎的所有开通方式下都会被优先消耗。

#### 使用豆包搜索的用户权益

以火山引擎主账号 ID 维度，每月发放 500 次免费额度（每月1日重置），免费额度不专属于 Agent Plan 获取的 API Key 调用，在火山引擎的所有开通方式下都会被优先消耗。

### 能力介绍

豆包搜索支持的核心能力如下，详情参见 [豆包搜索 API 文档](#)。

- **网页搜索**：返回标题、来源站点、URL、摘要、正文等结构化字段，单次最多 50 条。
- **多模态搜索**：对文本 query 返回图片内容（包括图片标题、URL、尺寸与形状）。
- **权威来源过滤**：可限定返回可信度高的各类官方站点内容。
- **时间范围筛选**：支持近一天/周/月/年的快捷枚举，或自定义日期区间。
- **Query 改写**：对口语化长问题自动改写为搜索式 query，提升召回。

### 使用豆包搜索

注意

- 使用联网搜索（Beta版）的用户：可继续使用原 API Key 进行接入鉴权，但无法使用套餐内 AFP 进行抵扣。建议迁移至豆包搜索。
- 使用豆包搜索的用户：使用 Agent Plan 专属 API Key 调用搜索服务，支持使用套餐内 AFP 进行抵扣。

步骤一：订阅套餐并开通豆包搜索

- 在 [开通管理-Agent Plan](#) 页面购买 Agent Plan 个人版套餐。套餐说明参见[套餐概览](#)。
- 在控制台进入 [使用配置](#) 页签，单击 **配置 Harness**，开启豆包搜索的抵扣开关。

1 配置模型及Base URL

2 配置Harness（可选）

3 配置专属API Key

4 接入使用

专属 Harness开箱即用，解锁 Agent 场景全新体验，更多能力持续扩展中

 **专业数据集** 首次上线 Hot

金融股票 工商企业库 学术论文 MCP

面向AI agent场景的搜索基础设施，聚合多个垂直专业领域的数据源，提供高质量专业数据检索能力

抵扣说明：金融数据库、企业工商数据库、企业风险数据库 12AFP/次，科研学术数据搜索服务 24AFP/次

开启抵扣 ☒ 超额后付费 ☐ [能力介绍](#) [配置使用](#)

 **豆包搜索** 每月免费额度

Skill/MCP

全新升级联网搜索能力，适配 Agent Plan AFP 统一抵扣，专为Agent场景深度开发；扩充大模型实时信息，有效降低模型幻觉，提升回答事实可信度。

抵扣说明：单账号每月赠送500次免费额度，免费额度耗尽后套餐内调用5 AFP/次

开启抵扣 ☒ 超额后付费 ☐ [能力介绍](#) [配置使用](#) [查看免费额度](#)

- 获取 Agent Plan 的专属 API Key，用于在工具中完成 Skill 或 MCP 的配置。

使用配置 用量统计

配置文档

1 配置模型及Base URL

2 配置Harness（可选）

3 配置专属API Key

4 接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key	创建时间	操作
*****	2026-06-04 14:26	<a href="#">更新API KEY</a>

步骤二：在工具中使用豆包搜索

目前支持通过 MCP Server 或 Skill 两种方式在工具中使用豆包搜索。具体接入方式参见 [AI 工具接入指南](#)。

## Skill 接入方式

通过 Skill 方式为 AI 工具快速安装并接入豆包搜索能力。以下为常用 AI 工具的配置方案，其他工具可按需选择通过命令安装或通过对话安装方式完成豆包搜索 Skill 的安装并使用。

### Claude Code

- 安装并配置 Claude Code，具体步骤见 [Claude Code](#)。
- 在终端执行以下命令为 Claude Code 安装并接入豆包搜索 Skill `byted-web-search`。

Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples -s byted-web-search --agent claude-code
```

- 安装完成后，执行 `claude` 命令启动 Claude Code。
- 通过对话方式提供 [Agent Plan](#) 的 API Key 完成 Skill 的配置。
- 配置完成后，可以在 Claude Code 中使用豆包搜索能力。



### OpenCode

- 安装并配置 OpenCode，具体步骤见 [OpenCode](#)。
- 在终端执行以下命令为 OpenCode 安装并接入豆包搜索 Skill `byted-web-search`。

## Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples -s byted-web-search --agent opencode
```

3. 安装完成后，执行 `opencode` 命令启动 OpenCode。
4. 通过对话方式提供 [Agent Plan](#) 的 API Key 完成 Skill 的配置。
5. 配置完成后，可以在 OpenCode 中使用豆包搜索能力。

联网搜索杭州今天天气

*Thinking:* 用户现在需要搜索杭州今天的天气，我需要使用byted-web-search的脚本。根据skill文档，需要调用web\_search.py，传入搜索词，还有用户提供的API Key。首先确认路径是/Users/bytedance/.agents/skills/byted-web-search/scripts/web\_search.py，参数是"杭州今天天气"，还有--api-key参数传入用户给的。用bash工具执行这个命令。

# 搜索杭州今日天气

```
$ python3 /Users/bytedance/.agents/skills/byted-web-search/scripts/web_search.py "杭州今天天气" --api-key
```

结果数：10 耗时：180ms

[1] 浙江省杭州市今天天气 | 本周天气 | 下周天气 | 未来天气  
火山如意 | 非常权威  
# 杭州市天气报告

## 2026年5月7日星期四 当前天气 杭州市，温度：27°C 湿度：42%，北风 5级，气压：1005.0hPa，日出/日落：05:11/18:41，月出/月落：23:40/08:56。

空气质量：AQI指数：39（优），主要污染物：—，PM2.5：27.0 µg/m³，PM10：36.0 µg/m³，全国排名：407/785。

...

[Click to expand](#)

*Thinking:* 用户现在需要我整理杭州今天的天气信息。首先看搜索结果，多个来源显示今天（5月7日）杭州的天气：多云，气温16~30°C左右，下午到后期有阵雨或雷雨，空气质量优，穿衣适合T恤短外套，紫外线中等。需要把关键信息提炼出来，简洁明了。

首先整理核心信息：

- 今日（5月7日）天气：多云，下午起转阴有阵雨或雷雨
  - 气温：16~30°C
  - 空气质量：AQI 39（优）
  - 生活提示：适合穿T恤、轻薄外套，紫外线中等需防晒，今日后期有雨洗车注意
- 然后用简洁的方式输出，不要太多废话，符合要求的4行以内。

## OpenClaw

1. 安装并配置 OpenClaw，具体步骤见 [OpenClaw](#)。
2. 在终端执行以下命令为 OpenClaw 安装并接入豆包搜索 Skill `byted-web-search`。

## Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples -s byted-web-search --agent openclaw
```

3. 安装完成后，执行以下命令启动 OpenClaw。

```
Bash
openclaw tui
```

- 4. 在对话框中输入 [Agent Plan 的 API Key](#)，完成 Skill 配置。
- 5. Skill 配置完成后可以在 OpenClaw 中使用豆包搜索能力。



### MCP 接入方式

以下为常用 AI 工具的 MCP 接入方案，其他工具可通过对话方式让模型完成 MCP 的接入。

#### Claude Code

- 1. 安装并配置 Claude Code，具体步骤见 [Claude Code](#)。

2. 编辑配置文件 `~/.claude.json`，添加 MCP 配置。其中 `<AGENT_PLAN_API_KEY>` 需替换为 [Agent Plan 的 API Key](#)。

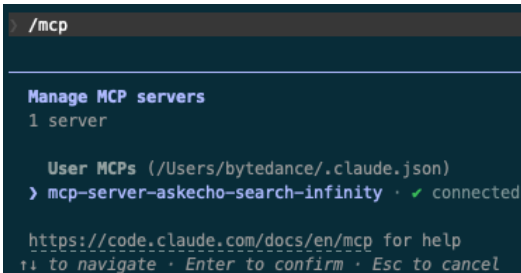
JSON

```
{
 ...
 "mcpServers": {
 "mcp-server-askecho-search-infinity": {
 "command": "uvx",
 "args": [
 "--from",
 "git+https://github.com/volcengine/mcp-server#subdirectory=server/mcp_server_askecho_search_infinity",
 "mcp-server-askecho-search-infinity"
],
 "env": {
 "ASK_ECHO_SEARCH_INFINITY_API_KEY": "<AGENT_PLAN_API_KEY>"
 }
 }
 }
 ...
}
```

### 说明

配置保存后，建议使用在线 JSON 验证工具或本地命令检查 JSON 格式是否合法。

3. 在终端执行 `claude` 命令启动 Claude 服务后，输入 `/mcp` 后看到 `mcp-server-askecho-search-infinity`，说明配置成功。



```
> /mcp

Manage MCP servers
1 server

User MCPs (/Users/bytedance/.claude.json)
> mcp-server-askecho-search-infinity · ✓ connected

https://code.claude.com/docs/en/mcp for help
↑↓ to navigate · Enter to confirm · Esc to cancel
```

4. MCP 安装完成后即可使用豆包搜索能力。

## OpenCode

1. 安装并配置 OpenCode，具体步骤见 [OpenCode](#)。
2. 编辑配置文件 `~/.config/opencode/opencode.json`，添加 MCP 配置。其中 `<AGENT_PLAN_API_KEY>` 需替换为 [Agent Plan 的 API Key](#)。



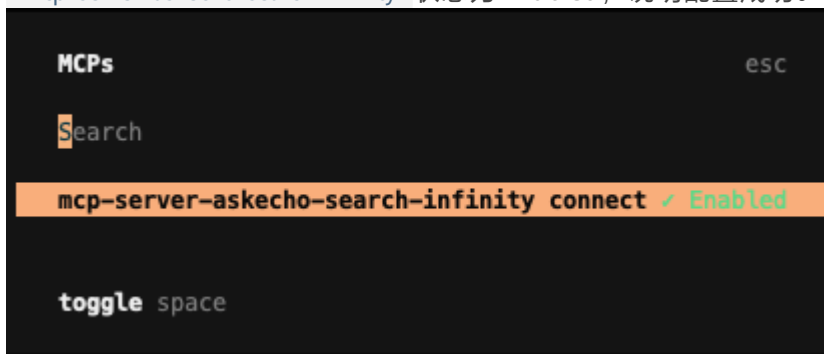
JSON

```
{
 ...
 "mcp": {
 "mcp-server-askecho-search-infinity": {
 "type": "local",
 "enabled": true,
 "command": ["uvx", "--from", "git+https://github.com/volcengine/mcp-
server#subdirectory=server/mcp_server_askecho_search_infinity", "mcp-server-askecho-search-
infinity"],
 "environment": {
 "ASK_ECHO_SEARCH_INFINITY_API_KEY": "<AGENT_PLAN_API_KEY>"
 }
 }
 }
 ...
}
```

### 说明

配置保存后，建议使用在线 JSON 验证工具或本地命令检查 JSON 格式是否合法。

- 在终端执行 `opencode` 命令启动 OpenCode 服务后，输入 `/mcp` 后看到 `mcp-server-askecho-search-infinity` 状态为 `Enabled`，说明配置成功。



- MCP 安装完成后即可使用豆包搜索能力。

## OpenClaw

- 安装并配置 OpenClaw，具体步骤见 [OpenClaw](#)。
- 在终端执行如下命令安装 MCPorater。

Bash

```
npm install -g mcporter
```

- 编辑配置文件 `~/.openclaw/openclaw.json`，添加 MCP 配置。其中 `<AGENT_PLAN_API_KEY>` 需替换为 [Agent Plan](#) 的 API Key。

JSON

```
{
 ...
 "mcp": {
 "servers": {
 "mcp-server-askecho-search-infinity": {
 "command": "uvx",
 "args": [
 "--from",
 "git+https://github.com/volcengine/mcp-server#subdirectory=server/mcp_server_askecho_search_infinity",
 "mcp-server-askecho-search-infinity"
],
 "env": {
 "ASK_ECHO_SEARCH_INFINITY_API_KEY": "<AGENT_PLAN_API_KEY>"
 }
 }
 },
 "skills": {
 "entries": {
 "mcporter": {
 "enabled": true
 }
 }
 }
 },
 ...
}
```

### 说明

配置保存后，建议使用在线 JSON 验证工具或本地命令检查 JSON 格式是否合法。

4. 在终端执行如下命令使配置生效。

Bash

```
openclaw gateway restart
```

5. 在项目目录下，执行如下命令查看已安装的 `mcp-server-askecho-search-infinity` MCP。

Bash

```
mcporter list
```

6. 启动 OpenClaw 后，即可使用豆包搜索能力。

## 实践教程

---

### 搭建个人投资助手网站

基于 Agent Plan + 豆包搜索 + 专业数据集，搭建一个面向普通投资者的个人投资助手网站。

### 搭建短视频网站

基于 Agent Plan + 豆包搜索，搭建一个抖音同款短视频网站。

### 搭建学习教育网站

基于 Agent Plan + 豆包搜索，搭建一个绘声绘色学古诗\–教育网站。

## 常见问题

---

- 如何关闭豆包搜索服务？  
豆包搜索是按量后付费型产品，当前官网无法主动关停服务。如有关停需求可提交[工单申请](#)。
- 如何查看豆包搜索付费用量统计？  
可以在[豆包搜索控制台数据管理](#)查看豆包搜索付费用量信息。
- 豆包搜索和联网内容插件有什么区别？  
豆包搜索和联网内容插件的区别请参见[豆包搜索产品简介](#)。

## 服务协议

---

- [豆包搜索专用条款](#)
- [豆包搜索服务等级协议](#)
- [Harness 能力说明和产品专用条款](#)

## 6.2 专业数据集

专业数据集是面向 AI Agent 场景的专业搜索基础设施，通过聚合多个垂直专业领域的数据库，为 Agent 提供结构化、高质量、高可信度的专业领域的数据检索能力，有效降低 Agent 执行过程中出现幻觉、数据来源不明确等风险。

本文将介绍购买 Agent Plan 套餐后，如何在 AI 工具中通过 MCP Server 接入并使用专业数据集。

### 能力介绍

专业数据集适用于 Agent 在专业垂类场景下进行数据查询与补充。服务提供了 Query 的解析与路由能力，可根据 Agent 传入的 Query 判断适用的专业数据类型并进行智能调用，减少用户自行处理与判断需要调用数据类型的操作行为，提高数据调用的准确性。

目前，专业数据集支持以下数据类型：

数据类型	能力说明	使用场景	调用限制
金融数据库	涵盖全球股票、期货，国内期权、债券、基金等各类上市品种基本资料、财务数据、日频行情信息、盈利预测、技术形态等各类分析指标等数据。	使用场景：Query 中包含股票代码或上市公司名称，以及金融指标相关描述时，专业数据集会主动路由调用金融数据库服务查询数据。 推荐用法：在 Query 中明确调用或提及金融数据库、明确具体股票信息与指标，避免使用模糊描述或条件描述式查询。	单次查询最多可查 3 只股票/标的/实体。
企业工商数据库	覆盖国内全量企业，可提供基本信息、经营状况、知识产权等维度的企业工商数据。	使用场景：Query 中包含明确的公司名称，以及基本信息/经营情况/知识产权等描述时，专业数据集会主动路由调用企业工商数据。 推荐用法：在 Query 中明确调用或提及企业工商数据，明确的公司名称或统一信用代码，避免使用简称查询。	单次查询最多可查 5 家公司的详细工商信息。
企业风险数据库	覆盖国内全量企业，可提供企业相关的司法诉讼、行政处罚等	使用场景：Query 中包含明确的公司名称，以及经营风险/司	单次查询最多可查 5 家公司的风险信息情况。

数据类型	能力说明	使用场景	调用限制
	企业异常与风险相关信息数据。	法案件/行政处罚等描述时，专业数据集会主动路由调用企业风险数据库。 推荐用法：在 Query 中明确调用或提及企业工商数据，明确的完整公司名称或统一社会信用代码，避免使用简称查询。	
科研学术数据搜索服务	提供海外英文学术论文的全网搜索服务，覆盖主流学术论文、科研期刊相关网站，基于关键词提供搜索命中的论文标题、论文所在链接、链接摘要、发表时间、作者等丰富的数据字段。	使用场景：Query 中包含论文、文献、期刊、学科等与学术载体有关名词时，专业数据集会主动路由调用科研学术数据搜索服务。 推荐用法：在 Query 中明确调用或提及科研学术数据搜索、想要搜索的学科或方向。	单次查询最多返回 50 篇相关论文或文章。
其他专业数据搜索服务	当 Query 解析后不属于平台内其他垂类专业数据集时，提供兜底的数据检索服务。	使用场景：当 Query 无明显意图可以匹配上述垂类专业数据集时，专业数据集才会路由到其他专业数据搜索服务。 推荐用法：若 Query 或目标不在垂类专业数据集列表中，建议优先调用其他 Harness 服务而非本专业数据集。	单次查询最多返回 20 条相关内容。

针对不同数据类型的 Query 及返回结果示例如下：

金融数据库

- Query示例：002594 股票 K 线形态分析  
返回结果：

JSON

```
{
 "code": 0,
 "msg": "success",
 "trace_id": "ac3e8789a5884d0e859ccd40134cdd8c",
 "tool": "dataPro_search",
 "query": "002594 股票 K 线形态分析",
 "total": 1,
 "items": [
 {
 "查询代码": "002594.SZ",
 "table": {
 "连涨天数": [2],
 "连跌天数": [0],
 "创历史新高": [false],
 "创历史新低": [false],
 ...
 }
 }
]
}
```

- Query示例：比亚迪 ROE 盈利水平  
返回结果：

JSON

```
{
 "code": 0,
 "msg": "success",
 "trace_id": "d4bd28f151f64e7f83d7ae4381c3b5a9",
 "tool": "dataPro_search",
 "query": "比亚迪 ROE 盈利水平",
 "total": 1,
 "items": [
 {
 "查询代码": "002594.SZ",
 "table": {
 "净资产收益率ROE(平均,同花顺计算)": [1.6464],
 "净资产收益率ROE(加权,公布值)": [1.65],
 "净资产收益率ROE(摊薄,公布值)": [1.6344],
 ...
 }
 }
]
}
```

- Query示例：五粮液季度业绩季报数据  
返回结果：

JSON

```
{
 "code": 0,
 "msg": "success",
 "trace_id": "16a4b22217814c4a8b9ac9ed5e225465",
 "tool": "dataPro_search",
 "query": "五粮液季度业绩季报数据",
 "total": 1,
 "items": [
 {
 "查询代码": "000858.SZ",
 "table": {
 "定期报告实际披露日期": ["20260430"],
 "定期报告预计披露日期": ["20260430"],
 "定期报告最新报告期": ["20260331"],
 ...
 }
 }
]
}
```

## 科研学术数据搜索服务

- Query示例：碳中和 课题  
返回结果：

JSON

```
{
 "code": 0,
 "msg": "success",
 "trace_id": "c4e123cc1fed406297b721aefba08b8d",
 "tool": "dataPro_search",
 "query": "碳中和 课题 学位论文 硕士 博士 研究资料 双碳 碳达峰 碳排放 绿色低碳 能源转型 综述",
 "total": 5,
 "items": [
 {
 "name": "Continuous extraction and continuous backfill for gaseous and mineralized dual CO2 sequestration and water-preserving coal mining",
 "url": "https://link.springer.com/content/pdf/10.1007/s40948-025-00957-z.pdf",
 "date_published": "2025-01-01T08:00:00.0000000",
 "snippet": "... carbon emissions. FA require less energy to process compared to cement. FA-based backfills exhibit 30 – 50% lower embodied energy than ordinary Portland cement systems (Zhang et ...",
 "abstract": "Abstract This study presents an innovative dual CO2 sequestration (DCS) method by ...",
 "extra_data": {
 "authors": "Y Xu, L Ma, L Wang, I Ngo, Y Wang",
 "cite_by": 6,
 ...
 }
 },
 ...
]
}
```

- Query示例：自动驾驶技术  
返回结果：

```
{
 "code": 0,
 "msg": "success",
 "trace_id": "7010deba04d04801a456499501eeec2",
 "tool": "dataPro_search",
 "query": "自动驾驶技术 发表文献 综述 研究论文 感知 规划 控制 多传感器融合 端到端驾驶 2024 2025",
}
```



```
JSON
{
 "total": 5,
 "items": [
 {
 "name": "Multimodal End-to-End Autonomous Driving",
 "url": "https://arxiv.org/pdf/1906.03199",
 "date_published": "2020-08-11T00:00:00.0000000",
 "snippet": "; J. Yu, J. Cai, and J. Luo, “End-to-end multi-modal multi-task vehicle control for self-driving cars with visual perceptions,” in Proc",
 "abstract": "A crucial component of an autonomous vehicle (AV) is the artificial intelligence (AI) is able to drive ...",
 "extra_data": {
 "authors": "Yi Xiao,Felipe Codevilla,Akhil Gurram,Onay Urfalioglu,Antonio M. Lopez,Yi Xiao,Felipe Codevilla,Akhil Gurram,Onay Urfalioglu,Antonio M. Lopez",
 "cite_by": 254,
 ...
 }
 },
 ...
]
}
```

使用专业数据集

步骤一：订阅套餐并开通专业数据集

- 1. 在 [开通管理-Agent Plan](#) 页面购买 Agent Plan 个人版套餐。套餐说明参见[套餐概览](#)。
- 2. 在控制台进入 [使用配置](#) 页签，单击 [配置Harness](#)，开启专业数据集的抵扣开关。

使用配置 用量统计

配置文档

1 配置模型及Base URL 2 配置Harness (可选) 3 配置专属API Key 4 接入使用

专属 Harness开箱即用，解锁 Agent 场景全新体验，更多能力持续扩展中

联网搜索 Beta版

限时免费额度 Skill/MCP

权益生效时间：2026/04/16–2028/03/27

剩余免费额度：9994/10000

超额后付费

查看联网密钥 查看文档

专业数据集

金融股票 工商企业库 学术论文 MCP

面向AI agent场景的搜索基础设施，聚合多个垂直专业领域的数据源，提供高质量专业数据检索能力

抵扣说明：金融数据库、企业工商数据库、企业风险数据库 12AFP/次，科研学术数据搜索服务 24AFP/次

开启抵扣

超额后付费

能力介绍 配置说明

版权所有©北京火山引擎科技有限公司

168/215

3. 获取用于接入专业数据集的专属 API Key，供步骤二配置使用。

使用配置

用量统计

配置文档

1 配置模型及Base URL

2 配置Harness (可选)

3 配置专属API Key

4 接入使用

专属API KEY

Agent Plan个人版专属API Key是访问火山方舟大模型服务的重要凭证，长期有效。请妥善保管并定期更换密钥，避免公开共享，以防安全风险和资金损失

API Key	创建时间	操作
***** *****	2026-06-04 14:26	更新API KEY

步骤二：在工具中使用专业数据集

目前支持通过 MCP Server 方式在工具中配置与使用专业数据集。

注意

如果需要避免调用某个类型数据集，可以进入[高质量数据集管理控制台](#)进行数据集的开关管理。

Claude Code

- 1. 安装并配置 Claude Code，具体参见 [Claude Code](#)。
- 2. 执行以下命令添加 MCP Server。其中 `<Agent-Plan-Key>` 需替换为 [专属API Key](#)。

Bash

```
claude mcp add --transport http datapro https://datapro.hqd.cn-beijing.volces.com/mcp \
--header "X-Agent-Plan-Key: <Agent-Plan-Key>"
```

3. 在终端执行以下命令启动 Claude Code，然后使用专业数据集。

Bash

```
claude
```

OpenClaw

- 1. 安装并配置 OpenClaw，具体参见 [OpenClaw](#)。
- 2. 执行以下命令添加 MCP Server。其中 `<Agent-Plan-Key>` 需替换为 [专属API Key](#)。

```
openclaw mcp set datapro '{
 "url": "https://datapro.hqd.cn-beijing.volces.com/mcp",
 "transport": "streamable-http",
 "headers": {
```

```
Bash
{"X-Agent-Plan-Key": "<Agent-Plan-Key>"
}
```

3. 在终端执行以下命令使配置生效。

```
Bash

openclaw gateway restart
```

4. 在项目目录下，执行如下命令查看已安装的 `datapro` MCP。

```
Bash

mcporter list
```

5. 启动 OpenClaw 后，即可使用专业数据集。

## TRAE

1. 安装并配置 TRAE，具体参见 [TRAE](#)。
2. 启动 TRAE 并打开对应项目。
3. 在设置页面，选择左侧菜单 **MCP**，在 MCP 页面右上角选择**添加-手动配置**。
4. 在手动配置弹窗添加以下 MCP 配置。其中 `<Agent-Plan-Key>` 需替换为[专属API Key](#) [🔗](#)。

```
JSON

{
 "mcpServers": {
 "https://datapro.hqd.cn-beijing.volces.com/mcp": {
 "url": "https://datapro.hqd.cn-beijing.volces.com/mcp",
 "headers": {
 "X-Agent-Plan-Key": "<Agent-Plan-Key>"
 }
 }
 }
}
```

5. MCP 安装完成后，在 AI 对话框选择 **Builder with MCP** 模式使用专业数据集。

## Hermes Agent

1. 安装并配置 Hermes Agent，具体参见 [Hermes Agent](#)。
2. 启动 Hermes Agent。
3. 通过对话形式安装 MCP 配置。其中 `<Agent-Plan-Key>` 需替换为[专属API Key](#) [🔗](#)。

## YAML

帮我安装下面这个专业数据库MCP:

```
mcp_servers:
 datapro:
 url: "https://datapro.hqd.cn-beijing.volces.com/mcp"
 transport: "http"
 headers:
 X-Agent-Plan-Key: "<Agent-Plan-Key>"
```

4. 安装完成后，即可使用专业数据集。

## OpenCode

1. 安装并配置 OpenCode，具体参见 [OpenCode](#)。
2. 配置环境变量 `AGENT_PLAN_KEY`，并将其值替换为[专属API Key](#) [🔗](#)。

## Bash

```
export AGENT_PLAN_KEY="<Agent-Plan-Key>"
```

3. 编辑配置文件 `~/.config/opencode/opencode.json`，新增以下 MCP 配置。

## JSON

```
{
 "$schema": "https://opencode.ai/config.json",
 "mcp": {
 "datapro": {
 "type": "remote",
 "url": "https://datapro.hqd.cn-beijing.volces.com/mcp",
 "enabled": true,
 "headers": {
 "X-Agent-Plan-Key": "{env:AGENT_PLAN_KEY}"
 }
 }
 }
}
```

4. 启动 OpenCode 后，即可使用专业数据集。

## Codex

1. 安装并配置 Codex，具体参见 [Codex CLI](#)。
2. 编辑配置文件 `~/.codex/config.toml`，新增以下 MCP 配置。其中 `<Agent-Plan-Key>` 需替换为[专属API Key](#) [🔗](#)。

## TOML

```
[mcp_servers.datapro]
url = "https://datapro.hqd.cn-beijing.volces.com/mcp"
transport = "http"
http_headers = { "X-Agent-Plan-Key" = "<Agent-Plan-Key>" }
```

3. 启动 Codex 后，输入 `/mcp` 命令验证MCP被识别成功后，即可使用专业数据集。

## 计费说明

- **未开启超额后付费**：请求调用产生的费用使用套餐内 AFP 进行抵扣，不消耗其他资源包或账户余额。抵扣规则参见[套餐内 AFP 抵扣规则](#)。
- **开启超额后付费**：在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息（如 Base URL、API Key、模型名称），系统将自动切换至超额后付费模式继续使用，并产生后付费账单。具体规则与操作参见[超额后付费规则](#)和[超额后付费管理](#)。

## 实践教程

### 搭建销售团队工作台

基于 Agent Plan + Codex + AI Native 应用开发底座 + 豆包搜索 + 专业数据集 + Agent 记忆，把散落在飞书会话、云文档里的私域销售资料和外部专业信息，搭成一个能查公司、看档案、问历史的销售团队定制工作台。

## 服务协议

- [高质量数据集服务专用条款](#) 
- [高质量数据集服务等级协议](#) 

## 6.3 Agent 记忆

火山引擎 OpenViking Service 为 Agent Plan Harness 层提供 Agent 记忆服务。OpenViking Service 是基于[开源 OpenViking](#) 内核构建的全托管云服务，是一款面向 AI Agents 的上下文数据库，以“虚拟文件系统 + 分层上下文 + 目录递归检索 + 可观测与自迭代”为核心思想，通过将 Memory、Resource、Skill 统一抽象为文件的理念，解决 AI Agent 在执行对话和长周期任务时的复杂上下文管理难题，为 AI Agent 开发者提供结构化、可追溯且低成本的上下文工程解决方案。

### 能力介绍

OpenViking Service 适用于需要长程记忆、复杂知识管理和多 Agent 协作的各类 AI 应用场景，核心价值在于提供结构化、可迭代的上下文管理能力。

- **Agent 长期记忆升级**：作为 Agent 产品的记忆插件，为 Agent 提供记忆沉淀、检索和自进化能力，解决长周期任务中常见的遗忘和高昂成本问题。
- **全局知识资产沉淀**：支持将多个代码仓库、文档库等知识源统一导入 OpenViking Service，构建可语义检索的知识库，赋能代码助手与研发场景。
- **多 Agent 协作共享**：多个独立 Agent 可共享同一个 OpenViking Service 实例作为记忆与知识中枢，实现高效协同。
- **多租户安全隔离**：利用多租户共享与隔离能力，为团队或企业构建安全可控的知识与记忆中心，让智能体在可控的上下文环境中执行任务。

### 使用限制

- 仅支持完成了实名认证的企业用户和个人用户使用。
- 仅支持在北京地域使用 Agent Plan 套餐的 AFP 额度抵扣火山引擎 OpenViking Service 资源的使用费用。
- 仅支持 Agent Plan 个人版、OpenViking Service 个人版。
- 如果您是子账号，首次使用时需联系主账号授权 [VikingdbFullAccess](#) 策略。
- 火山引擎官网账号未欠费（账户余额  $\geq 0$ ）。

## 使用 OpenViking Service

### 步骤一：订阅套餐并开通 Agent 记忆

1. 在 [开通管理-Agent Plan](#) 页面购买 Agent Plan 个人版套餐。套餐说明参见[套餐概览](#)。
2. 在控制台进入 [使用配置](#) 页签，单击 **配置 Harness**，开启 Agent 记忆的抵扣开关。

开通管理

模型 | 应用组件库 | 云产品 | 高级创作权益包 | **Agent Plan** | Agent Plan 企业版 | Coding Plan | Coding Plan 企业版

管理素材资产



步骤二：创建数据库

在 使用配置 - 配置 Harness 区域，找到 Agent 记忆卡片，单击 配置使用 进入 OpenViking Service 页面，根据页面提示开通服务并创建数据库。

说明

创建数据库的页面左下角会展示 AFP 抵扣信息。

开通管理

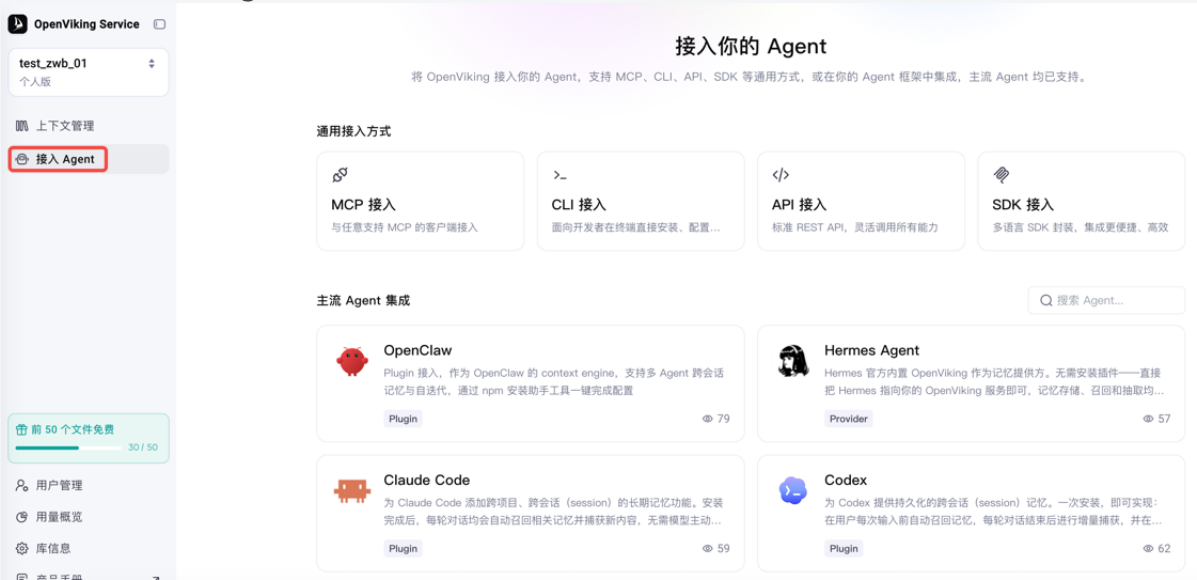


步骤三：接入 Agent

OpenViking Service 支持 MCP、CLI、API、SDK 接入您的自研 Agent 框架，同时对主流 Agent 框架进行生态适配。

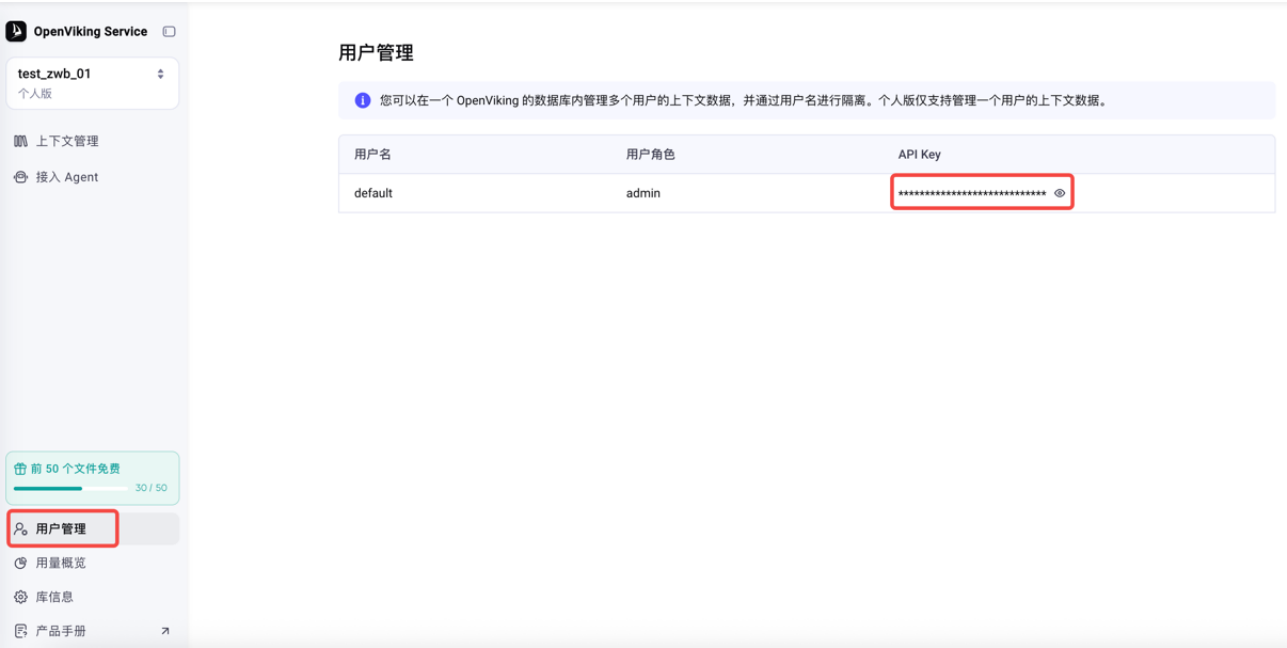
说明

当前文档仅展示了常用 Agent 接入步骤，更多 Agent 的接入步骤可在 [OpenViking Service 控制台](#) 页面单击 接入 Agent 查看。



认证说明

Agent 接入 OpenViking 时，必须使用 OpenViking 的 API Key 作为唯一认证，并关联到对应的数据。OpenViking API Key 可在 [OpenViking Service 控制台](#) 的 用户管理 页面查看与复制。



## 主流 Agent 集成

### Claude Code

#### 说明

**集成方式：**Plugin 接入，为 Claude Code 添加跨项目、跨会话（session）的长期记忆功能。安装完成后，每轮对话均会自动召回相关记忆并捕获新内容，无需模型主动调用任何工具。

#### 步骤 1：安装

Bash

```
bash <(curl -fsSL https://raw.githubusercontent.com/volcengine/OpenViking/main/examples/claude-code-memory-plugin/setup-helper/install.sh)
```

该脚本将自动检查依赖项、配置 OpenViking 连接并完成插件安装，支持重复运行（幂等操作）。安装完成后，需在当前终端激活 claude 的封装函数（wrapper），或者直接打开一个新的终端窗口：

Bash

```
source ~/.openviking/openviking-repo/examples/claude-code-memory-plugin/setup-helper/wrapper.sh
```

#### 说明



通过自定义命令启动 Claude Code？例如包装脚本 `cc-custom`，或“基础命令 + 子命令”形式的多词启动器——在安装时的“Extra launch commands”一步填入（或运行脚本时传入 `OPENVIKING_CC_WRAP_EXTRA='cc-custom'`），即可让它们一并注入凭据。

使用一段时间后，即便在全新的对话中提及过往的话题，Claude Code 也能准确回忆起来。

## 步骤 2：验证

Bash

```
type claude
```

### 说明

若输出的是一个路径而非 shell function，说明 wrapper 尚未生效，请先 `source` 那行 wrapper（或新开一个终端）再启动；否则 `claude` 会静默连到本地 127.0.0.1 且不带鉴权。

在 `type claude` 显示为 shell function 的终端中运行 `claude` 启动，随后：

- 输入 `/plugins`：在 Installed 列表中应能找到 `openviking-memory`（其子项 `openviking MCP` 应显示为已连接状态）。
- 输入 `/mcp`：OpenViking 对应的条目应显示您的服务器 URL 及有效的认证信息。
- 输入 `/openviking-memory:ov`：查看服务器状态、身份信息、召回/注入的统计数据以及功能开关状态。若插件未正常工作，可设置环境变量 `OPENVIKING_DEBUG=1`，并查看日志文件 `~/openviking/logs/cc-hooks.log` 以排查问题。

## OpenClaw

### 说明

**集成方式：**Plugin 接入，作为 OpenClaw 的 context engine，支持多 Agent 跨会话记忆与自迭代，通过 npm 安装助手工具一键完成配置。

**前提条件：**您已经在本地或云服务器上部署了 OpenClaw 2026.5.2 或更新版本。

### 步骤 1：安装 OpenViking

在你的 OpenClaw 的机器终端，执行以下命令以安装 OpenViking Plugin。

Bash

```
openclaw plugins install clawhub:@openviking/openclaw-plugin && openclaw openviking setup
```

### 步骤 2：配置信息

根据提示配置以下信息：

- Base URL：`https://api.vikingdb.cn-beijing.volces.com/openviking`
- API Key：在 OpenViking Service 控制台的用户管理页面复制 API Key，具体参见[认证说明](#)。

### 步骤 3：重启 OpenClaw

复制以下指令到 Agent 终端以重启 OpenClaw，重启后控制台将自动判断 Agent 接入状态。

```
Bash
openclaw gateway restart
```

Hermes Agent

**说明**  
**集成方式：**Hermes 官方内置 OpenViking 作为记忆提供方。无需安装插件，直接把 Hermes 指向您的 OpenViking 服务即可，记忆存储、召回和抽取均原生支持。

**步骤1：配置**  
运行 Hermes 记忆配置向导：

```
Bash
hermes memory setup
```

- 配置以下信息：
- OpenViking 服务 URL：自托管服务器 或 火山引擎 OpenViking Service
  - API Key：火山引擎 OpenViking Service 时填 API Key，具体参见[认证说明](#)。
  - 租户 account / user / agent ID：多租户部署时使用

配置保存在 Hermes 的 `config.yaml` 和 `.env` 文件中。

**步骤2：验证**

```
Bash
hermes memory status
```

配置完成后，Hermes 自动使用 OpenViking 作为长期记忆——`viking_remember`、`viking_recall` 等记忆工具即刻可用。

Codex

**说明**  
**集成方式：**Plugin 接入，为 Codex 提供持久化的跨会话（session）记忆。一次安装即可实现：每次输入前自动召回记忆、每轮对话结束后增量捕获，并在上下文压缩前将记忆提交给抽取器。

**步骤 1：安装**

Bash

```
bash <(curl -fsSL https://raw.githubusercontent.com/volcengine/OpenViking/main/examples/codex-memory-plugin/setup-helper/install.sh)
```

该脚本会自动检查依赖、配置 OpenViking 连接并注册插件。所有步骤均具有幂等性，可安全地重复执行。

安装完成后，在当前终端激活 codex 封装函数，或重新打开一个终端窗口。

Bash

```
source ~/.openviking/openviking-repo/examples/codex-memory-plugin/setup-helper/wrapper.sh
```

```
首次启动需执行 /hooks 进行一次安全审批
codex
```

### 说明

通过自定义命令启动 Codex？例如包装脚本 `codex-custom`，或“基础命令 + 子命令”形式的多词启动器——在安装时的“Extra launch commands”一步填入（或运行脚本时传入 `OPENVIKING_CODEX_WRAP_EXTRA='codex-custom'`），即可让它们一并注入凭据。

## 步骤 2：验证

Bash

```
type codex
```

### 说明

若上一步输出的是一个路径而非 `shell function`，说明 wrapper 尚未生效，请先 `source` 那行 wrapper（或新开一个终端）再启动；否则 codex 启动时拿不到 `OPENVIKING_API_KEY`，会报 `MCP server is not logged in`。

进入 Codex 后，插件将在每次输入前自动召回记忆。将环境变量 `OPENVIKING_DEBUG` 设置为 1，可将事件日志输出至 `~/.openviking/logs/codex-hooks.log`。

## CLI 接入

### 手动终端安装

步骤1：安装 OpenViking CLI 并进入配置流程

Bash

```
npm i -g @openviking/cli && ov config
```

## 步骤2: 配置信息

按提示填写以下信息：

- Base URL: `https://api.vikingdb.cn-beijing.volces.com/openviking`
- API Key: 在 OpenViking Service 控制台的[用户管理](#)页面复制 API Key，具体参见[认证说明](#)。

## 步骤3: 查看 CLI 用法

Bash

```
ov --help
```

## 通过 AI Agent 安装

将下方提示词复制给您的 AI 助手（ Claude Code、Codex、Cursor、TRAE 等 ）， 它会自动完成 OpenViking CLI 安装、配置和用法学习：

### 说明

`<OPENVIKING_API_KEY>` 需替换为 OpenViking Service 控制台的[用户管理](#)页面的 API Key，具体参见[认证说明](#)。

Plain

请先向用户询问 OpenViking API Key，并记为 OPENVIKING\_API\_KEY。

请在 `~/openviking/ovcli.conf` 写入以下内容：

```
{
 "url": "https://api.vikingdb.cn-beijing.volces.com/openviking",
 "api_key": "<OPENVIKING_API_KEY>"
}
```

如发现 `~/openviking/ovcli.conf` 已存在且内容冲突，请先询问用户是否备份原文件，并在得到确认后覆盖。

请安装 OpenViking CLI：

```
npm i -g @openviking/cli
```

安装完成后，请运行：

```
ov --help
```

请探索 CLI 用法，并把 OpenViking CLI 的使用方式写入你的长期记忆。

## MCP 接入

### 步骤1: MCP 配置

以下为 MCP 配置示例，将 `<OPENVIKING_API_KEY>` 替换为 OpenViking API Key，具体参见[认证说明](#)。

JSON

```
{
 "mcpServers": {
 "ov-mcp-server": {
 "url": "https://api.vikingdb.cn-beijing.volces.com/openviking/mcp",
 "headers": {
 "Authorization": "Bearer <OPENVIKING_API_KEY>"
 }
 }
 }
}
```

### 步骤2: 测试 MCP 工具连通性

Bash

```
ov health
```

## SDK 接入

### 步骤1: 安装 OpenViking

Bash

```
pip install openviking --upgrade --force-reinstall
```

### 步骤2: 初始化客户端

将控制台用户管理页面中展示的 API Key 复制到你的 Agent 运行环境中，建议通过环境变量或安全配置中心进行注入，而不要直接硬编码在代码里。具体参见[认证说明](#)。

Bash

```
export OPENVIKING_API_KEY="your-api-key"
```

```
import os
import openviking as ov
```

```
python
client = ov.SyncHTTPClient(
 url="https://api.vikingdb.cn-beijing.volces.com/openviking",
 api_key=os.environ["OPENVIKING_API_KEY"],
 agent_id="your-agent-id", # 自定义 agent_id, 用于隔离 Agent 侧上下文
 timeout=120.0,
)
client.initialize()
```

### 步骤3：写入资源

资源写入用于把外部知识导入 OpenViking Service。SDK 可以直接接收本地文件路径、目录路径或远端 URL，并自动处理上传细节。

Python

```
file_path = "your-file-path"
resource_to = "your-resource-path" # e.g. viking://resources/demo/api.md
reason = "your-reason" # e.g. External API documentation

result = client.add_resource(
 path=file_path,
 to=resource_to,
 reason=reason,
 wait=True,
)

print(result)
```

#### 说明

- to 必须位于 viking://resources/... 作用域下。
- 如果 path 是本地文件，SDK 会自动先上传再调用服务端接口。
- 如果 `wait=True`，接口会等待资源处理完成后返回，更适合导入后马上检索的场景。
- 如果不希望阻塞，也可以使用默认异步模式，然后显式调用 `client.wait_processed()`。

### 步骤4：写入记忆

记忆写入建议通过会话方式完成。下面示例演示如何将一条用户消息写入会话，并通过提交触发后续记忆提取。

```
text = "your-message-text" # e.g. I am a developer

session = client.create_session()
session_id = session["result"]["session_id"] if "result" in session else session["session_id"]

client.add_message(
```

```

Python
session_id,
 "user",
 parts=[{"type": "text", "text": text}],
)

result = client.commit_session(session_id)
print(result)

```

## API 接入

如果您希望通过平台网关、工作流引擎、低代码平台直接调用接口，推荐使用 HTTP API 接入。

### 步骤1：写入资源

对于 HTTP 接口，本地文件不能直接作为 path 传给 `/api/v1/resources`，需先上传本地文件获取 temp\_file\_id，再调用 `/api/v1/resources`。

```

import json
from pathlib import Path

import requests

url = "https://api.vikingdb.cn-beijing.volces.com/openviking"
api_key = "your-api-key" #可在用户管理页面复制
agent_id = "your-agent-id" #自定义agent-id
file_path = Path("your-file-path") # e.g. test.txt
resource_to = "your-resource-path" # e.g. viking://resources/demo/test.txt
reason = "your-reason" # e.g. External API documentation

headers = {
 "Content-Type": "application/json",
 "Authorization": "Bearer " + api_key,
 "X-OpenViking-Agent": agent_id,
}

def post_json(path: str, payload: dict, timeout: float):
 response = requests.post(
 f"{url}{path}",
 headers=headers,
 json=payload,
 timeout=timeout,
)
 response.raise_for_status()
 return response.json()

```

```

Python
with file_path.open("rb") as file:
 result = requests.post(
 f"{url}/api/v1/resources/temp_upload",
 headers={
 "Authorization": "Bearer " + api_key,
 "X-OpenViking-Agent": agent_id,
 },
 files={"file": (file_path.name, file, "application/octet-stream")},
 timeout=120.0,
)

 result.raise_for_status()
 result = result.json()
 print(json.dumps(result, ensure_ascii=False, indent=2))

temp_file_id = result["result"]["temp_file_id"]

result = post_json(
 "/api/v1/resources",
 {
 "temp_file_id": temp_file_id,
 "source_name": file_path.name,
 "to": resource_to,
 "reason": reason,
 },
 120.0,
)

print(json.dumps(result, ensure_ascii=False, indent=2))

```

## 步骤2：写入记忆

通过 API 写入记忆时，同样遵循“创建会话 - 添加消息 - 提交会话”的流程。

```

import json
import requests

url = "https://api.vikingdb.cn-beijing.volces.com/openviking"
api_key = "your-api-key" #可在【用户管理页面】复制
agent_id = "your-agent-id" #自定义agent-id
text = "your-message-text" # e.g. I am a developer

headers = {
 "Content-Type": "application/json",
 "Authorization": "Bearer " + api_key,
 "X-OpenViking-Agent": agent_id,
}

```



```
}
Python

def post_json(path: str, payload: dict, timeout: float):
 response = requests.post(
 f"{url}{path}",
 headers=headers,
 json=payload,
 timeout=timeout,
)
 response.raise_for_status()
 return response.json()

session = post_json("/api/v1/sessions", {}, 360.0)
session_id = session["result"]["session_id"]

post_json(
 f"/api/v1/sessions/{session_id}/messages",
 {
 "role": "user",
 "parts": [{"type": "text", "text": text}],
 },
 360.0,
)

result = post_json(
 f"/api/v1/sessions/{session_id}/commit",
 {"telemetry": False},
 360.0,
)

print(json.dumps(result, ensure_ascii=False, indent=2))
```

## 计费说明

- **未开启超额后付费**：请求调用产生的费用使用套餐内 AFP 进行抵扣，不消耗其他资源包或账户余额。抵扣规则参见[套餐内 AFP 抵扣规则](#)。
- **开启超额后付费**：在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息，系统将自动切换至超额后付费模式继续使用，并产生后付费账单。具体规则与操作参见[超额后付费规则](#)和[超额后付费管理](#)。

## 实践教程

## 搭建销售团队工作台

基于 Agent Plan + Codex + AI Native 应用开发底座 + 豆包搜索 + 专业数据集 + Agent 记忆，把散落在飞书会话、云文档里的私域销售资料和外部专业信息，搭成一个能查公司、看档案、问历史的销售团队定制工作台。

## 常见问题

- 我想保留 OpenViking 服务，仅想从 Agent Plan 抵扣切换到官网付费，要如何操作？  
在 Agent Plan 页面关闭抵扣后，立即进入[控制台](#)重新开通 OpenViking 商品即可。
- 我想保留 OpenViking 服务，仅想从官网付费切换到 Agent Plan 抵扣，要如何操作？  
下单个人版 Agent Plan 套餐，并在 Agent Plan 页面开启抵扣即可。
- Agent Plan 额度不足且未开通超额后付费，导致服务无法使用，如何恢复？  
开启超额后付费立即恢复，通过官网出账；或升级 Agent Plan 套餐扩充 AFP 额度，待下一个扣费周期扣减成功即可恢复，最长 1 小时；或升级后 oncall 客服人工恢复。

## 服务协议

- [OpenViking Service 专用条款](#)
- [Harness 能力说明和产品专用条款](#)

## 6.4 AI Native 应用开发底座

AI Native 应用开发底座基于火山引擎 Supabase 实现，作为 Agent Plan 的 Harness 为 AI 工具（Claude Code、OpenClaw、TRAE 等）提供云端数据库与后端基础设施，使 Agent 在开发应用时可直接通过 MCP Server 或 Skill 方式调用数据库、认证、存储、边缘函数等能力，无需手动搭建后端环境。

### 能力介绍

火山引擎 Supabase，全称是 AI 原生 BaaS 平台 Supabase 版，是基于开源 Supabase 打造，100% 兼容 Supabase 开源用法，包括 PostgreSQL Database、Authentication、Storage、Edge Function、Realtime、AI 助手、API Docs 等，具备云原生服务特有的高弹性低价格等特点。基于火山引擎数据库 Serverless 架构，实现了更好可用性、性能和成本优势，可以高性价比一站式地满足用户在 AI Agent 开发构建、SaaS 服务开发、原型系统快速构建等场景的诉求。

支持的核心能力如下：

- **Database**：基于 Serverless 的 PostgreSQL，支持秒级扩缩容（Scale to Zero）、数据分支管理及丰富插件。
- **Authentication**：提供完整的认证体系，支持短信/邮箱、各类第三方授权、单点登录，并允许自定义逻辑。
- **Storage**：提供对象存储服务，用于轻松管理文件的上传、下载及精细的权限控制。
- **Edge Function**：边缘函数服务，支持在靠近用户的边缘节点快速执行各类自定义服务端逻辑。
- **Realtime**：提供实时数据同步功能，支持将数据库变更结果进行实时推送与广播。
- **MCP Server**：基于 MCP 协议的专属服务，支持通过自然语言直接输入与管理各类数据库资源。
- **前端部署**：对接火山引擎 Pages，支持“推送即发布”（Push-to-Deploy）的一站式发布能力。开发者将代码推送至 Git 仓库后，平台可自动完成构建、部署和线上预览，打通“自然语言生成代码”到“应用发布”的最后一环，帮助用户降低部署运维成本，快速验证和交付应用。

### 使用限制

- 仅支持完成了实名认证的企业用户和个人用户使用。
- 仅支持在北京地域使用 Agent Plan 套餐的 AFP 额度抵扣火山引擎 Supabase 资源的使用费用。
- 首次使用时，请注意：
  - 如您是主账号：请为账号授权 **ServiceRoleForAIDAP** 角色。
  - 如您是子账号：请联系主账号授权 **ServiceRoleForAIDAP** 角色，并为您再授予 IAM 的 **AIDAPFullAccess** 预设策略或包含 AIDAP 权限点的自定义策略。

## 使用火山引擎 Supabase

### 步骤一：订阅套餐

1. 在 [开通管理-Agent Plan](#) 页面购买 Agent Plan 个人版套餐。套餐说明参见[套餐概览](#)。

2. 在控制台进入 **使用配置** 页签，单击 **配置Harness**，开启AI Native 应用开发底座的抵扣开关。  
开通管理



步骤二：安装与更新 CLI

- **安装**  
执行以下命令，即可完成在 MCP Server、Skill 和 CLI 三种方式下的安装。

```
Bash
npx @byted-supabase/cli@latest install
```

- **更新**  
如安装过程中收到 “A new version of Byted Supabase CLI is available” 的提示，按照指引或执行以下命令即可完成更新。

```
Bash
byted-supabase-cli update
```

步骤三：登录 CLI

登录 / 初始配置

- **方式 1：浏览器 OAuth 登录（推荐）**

```
Bash
byted-supabase-cli login --is-agent-plan
```

- **方式 2：用 AK/SK 配置默认 profile**

```
Bash
byted-supabase-cli configure set --ak <AK> --sk <SK> --is-agent-plan
```

## 企业版切换为个人版

如需从企业切换成个人版，只设置 `--is-agent-plan` 即可。

Bash

```
byted-supabase-cli configure agent-plan --profile <profile-name> --is-agent-plan
```

`--profile <profile-name>` 用于指定已有的 profile 名称。profile 由 `configure set` 或 `login` 命令创建，直接传入名称即可。

## 关闭 AgentPlan 默认设置

Bash

```
byted-supabase-cli configure agent-plan --profile <profile-name> --is-agent-plan=false
```

`--profile <profile-name>` 用于指定已有的 profile 名称。profile 由 `configure set` 或 `login` 命令创建，直接传入名称即可。

## 步骤四：在工具中使用火山引擎 Supabase

目前支持通过 Skill、CLI、MCP Server 三种方式使用火山引擎 Supabase。

### Skill 接入方式

通过 Skill 方式为 AI 工具快速安装并接入火山引擎 Supabase。

#### Claude Code

1. 安装并配置 Claude Code，具体步骤见 [Claude Code](#)。
2. 在终端执行以下命令为 Claude Code 安装并接入火山引擎 Supabase Skill。

Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples --s byted-supabase --agent claude-code
```

3. 安装完成后，执行 `claude` 命令启动 Claude Code。
4. 在 Claude Code 中通过自然语言使用火山引擎 Supabase 能力，具体参见[使用自然语言管理资源](#) [🔗](#)。

#### OpenCode

1. 安装并配置 OpenCode，具体步骤见 [OpenCode](#)。
2. 在终端执行以下命令为 OpenCode 安装并接入火山引擎 Supabase Skill。

Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples -s byted-supabase --agent opencode
```

3. 安装完成后，执行 `opencode` 命令启动 OpenCode。
4. 在 OpenCode 中通过自然语言使用火山引擎 Supabase 能力，具体参见[使用自然语言管理资源](#)。

## OpenClaw

1. 安装并配置 OpenClaw，具体步骤见 [OpenClaw](#)。
2. 在终端执行以下命令为 OpenClaw 安装并接入火山引擎 Supabase Skill。

Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples -s byted-supabase --agent openclaw
```

3. 安装完成后，执行以下命令启动 OpenClaw。

Bash

```
openclaw tui
```

4. 在 OpenClaw 中通过自然语言使用火山引擎 Supabase 能力，具体参见[使用自然语言管理资源](#)。

## TRAE

1. 安装并配置 TRAE，具体步骤见 [TRAE](#)。
2. 在终端执行以下命令安装 `byted-supabase` Skill，安装过程中按需选择 agent。

Bash

```
npx skills add https://skills.volces.com/skills/bytedance/agentkit-samples -s byted-supabase
```

3. 安装完成后，通过 TRAE 打开对应的项目，在[设置-技能与命令](#)页面确认已安装 `byted-supabase` Skill。
4. 在 TRAE 中通过自然语言使用火山引擎 Supabase 能力，具体参见[使用自然语言管理资源](#)。

## CLI 接入方式

通过 CLI 方式可在终端直接管理 Workspace 等资源，常用命令如下：

- 创建 Workspace

Bash

```
byted-supabase-cli projects create <your_workspace_name> --is-agent-plan
```

- 查询 Workspace 列表

Bash

```
byted-supabase-cli projects list
```

- 查询某个 Workspace 的详细信息

Bash

```
byted-supabase-cli projects list --workspace-id <Workspace-id>
```

## MCP 接入方式

以下为常用 AI 工具的 MCP 接入方案，其他工具可按需调整配置文件格式。

### Claude Code

1. 安装并配置 Claude Code，具体步骤见 [Claude Code](#)。
2. 编辑配置文件 `~/.claude.json`，添加 MCP 配置。

JSON

```
{
 ...
 "mcpServers": {
 "byted-supabase-mcp": {
 "command": "byted-supabase-cli",
 "args": [
 "mcp",
 "serve",
 "--agent-plan"
]
 }
 }
 ...
}
```

#### 说明

配置保存后，建议使用在线 JSON 验证工具或本地命令检查 JSON 格式是否合法。

3. 在终端执行 `claude` 命令启动 Claude 服务后，输入 `/mcp` 后看到 `byted-supabase-mcp`，说明配置成功。
4. MCP 安装完成后即可使用火山引擎 Supabase 能力。

## OpenCode

1. 安装并配置 OpenCode，具体步骤见 [OpenCode](#)。
2. 编辑配置文件 `~/config/opencode/opencode.json`，添加 MCP 配置。

JSON

```
{
 ...
 "mcp": {
 "byted-supabase-mcp": {
 "command": "byted-supabase-cli",
 "args": [
 "mcp",
 "serve",
 "--agent-plan"
]
 }
 }
 ...
}
```

### 说明

配置保存后，建议使用在线 JSON 验证工具或本地命令检查 JSON 格式是否合法。

3. 在终端执行 `opencode` 命令启动 OpenCode 服务后，输入 `/mcp` 后看到 `byted-supabase-mcp` 状态为 Enabled，说明配置成功。
4. MCP 安装完成后即可使用火山引擎 Supabase 能力。

## OpenClaw

1. 安装并配置 OpenClaw，具体步骤见 [OpenClaw](#)。
2. 在终端执行如下命令安装 MCPorter。

Bash

```
npm install -g mcporter
```

3. 编辑配置文件 `~/openclaw/openclaw.json`，添加 MCP 配置。

```
{
 ...
 "mcp": {
 "servers": {
 "byted-supabase-mcp": {
```



```
JSON {
 "command": "byted-supabase-cli",
 "args": [
 "mcp",
 "serve",
 "--agent-plan"
]
},
{
 "skills": {
 "entries": {
 "mcporter": {
 "enabled": true
 }
 }
 }
},
...
}
```

#### 说明

配置保存后，建议使用在线 JSON 验证工具或本地命令检查 JSON 格式是否合法。

4. 在终端执行如下命令使配置生效。

Bash

```
openclaw gateway restart
```

5. 在项目目录下，执行如下命令查看已安装的 `byted-supabase-mcp` MCP。

Bash

```
mcporter list
```

6. 启动 OpenClaw，随后即可使用火山引擎 Supabase 能力。

## TRAE

1. 安装并配置 TRAE，具体步骤见 [TRAE](#)。
2. 在设置页面，选择左侧菜单 **MCP**，在 MCP 页面右上角选择**添加-手动配置**。
3. 在手动配置弹窗添加 MCP 配置信息。

JSON

```
{
 "mcpServers": {
 "byted-supabase-mcp": {
 "command": "byted-supabase-cli",
 "args": [
 "mcp",
 "serve",
 "--agent-plan"
]
 }
 }
}
```

4. MCP 安装完成后，在 AI 对话框选择 **Agent** 模式使用火山引擎Supabase。

## 计费说明

- **未开启超额后付费**：请求调用产生的费用使用套餐内 AFP 进行抵扣，不消耗其他资源包或账户余额。抵扣规则参见[套餐内 AFP 抵扣规则](#)。
- **开启超额后付费**：在套餐周期内额度未用尽时优先抵扣套餐额度；当套餐周期内额度用尽时，无需修改任何配置信息，系统将自动切换至超额后付费模式继续使用，并产生后付费账单。具体规则与操作参见[超额后付费规则](#)和[超额后付费管理](#)。

## 实践教学

### 搭建销售团队工作台

基于 Agent Plan + Codex + AI Native 应用开发底座 + 豆包搜索 + 专业数据集 + Agent 记忆，把散落在飞书会话、云文档里的私域销售资料和外部专业信息，搭成一个能查公司、看档案、问历史的销售团队定制工作台。

### 搭建资料管理与AI 问答工具

基于 Agent Plan + AI Native 应用开发底座，搭建一个面向个人和小团队的轻量资料管理与 AI 问答工具。该工具支持文件上传、共享协作、通知提醒，帮助用户存储资料、发现资料，是一个全栈 App。

## 常见问题

- 如何获取 Workspace 的连接信息？  
在 [AI 原生 BaaS 平台 Supabase 版](#) 的 Workspace 详情页，可查看 Supabase URL、API Key 及数据库连接串。
- Workspace 暂停后数据会丢失吗？  
不会。暂停仅停止算力计费，数据存储空间仍会保留。恢复 Workspace 后可继续使用。

# 服务协议

- [产品和服务测试协议](#) 
- [Harness 能力说明和产品专用条款](#) 

## 6.5 ArkClaw

ArkClaw 是火山引擎提供的云端 AI 智能体（Agent）服务，能帮助您**一键云端部署 OpenClaw**，享有一对一专属 ECS 资源，免去复杂配置、7 × 24 在线，让您零门槛使用 OpenClaw，还能无缝使用您订阅的火山方舟 Agent Plan，告别 Token 按量计费焦虑。

### 用户权益

- 订阅方舟 **Agent Plan Medium 及以上套餐**：赠送 ArkClaw 轻量版。套餐到期时，您创建的 ArkClaw 将被删除，其中的数据将为您保留 24 小时。

#### 说明

体验中心提供的 ArkClaw 服务要求同一主账号同一套餐只能开通一个 ArkClaw，主账号下的子账号会共享数据记录。

### 步骤一：购买 Agent Plan

进入 [Agent Plan 控制台](#)，按需购买 Agent Plan Medium 及以上套餐。套餐介绍请参见[套餐概览](#)。

### 步骤二：创建并使用 ArkClaw

1. 登录火山方舟[体验中心](#)。
2. 在左侧目录选择 “Agent > ArkClaw”。
3. 单击 “立即创建” 按钮，提交创建申请。

#### 注意

- 使用子账号创建 ArkClaw 前，需先联系账号管理员为子账号添加 `ArkClawXFullAccess` 权限，权限配置完成后再单击**立即创建**创建 ArkClaw。
- 可以在页面左上角切换套餐，当前默认是已购买的 Agent Plan 的套餐。



4. 待创建成功后，即可在方舟中与 ArkClaw 进行对话。

注意

根据[使用技巧](#)，对 ArkClaw 进行个性化配置与管理，如配置消息渠道、切换使用的模型、定义人格等。



## 使用技巧

---

- [查看并配置定时任务](#) : 不仅支持实时解答问题，还支持完成设置的定时任务，主动发送提醒。
- [使用快捷指令](#) : 预置了常用的操作命令，例如：新建对话、查看系统状态及清空上下文等。
- [配置消息渠道](#) : 支持与您的 IM 工具（例如飞书机器人、钉钉机器人）进行快捷集成。
- [学习并使用技能](#) : ArkClaw 已预置部分常用技能（Skills），可根据任务需求自动匹配调用。
- [使用网盘功能上传/下载文件](#) : 将[对象存储（TOS）](#) 服务的TOS桶作为网盘，实现本地与云端的文件快速上传/下载操作。

## 服务规则

---

[ArkClaw 服务规则](#) 

# 7. 实践指南

## 7.1 使用 Agent Plan 开发短视频网站

本文档提供使用 Agent Plan 接入 AI 工具快速打造抖音同款短视频网站的完整实践指南，涵盖从套餐购买、环境配置、需求定义全模态素材生产到完整项目开发全流程。通过本文，您可以在分钟级别完成项目核心开发，全程无需手动编写代码。

### 项目概述

#### 项目效果



#### 核心使用能力

- 语言模型：Agent Plan 支持 Seed 2.0 系列、GLM 系列、Minimax 系列、DeepSeek 系列、Kimi 系列等多种优秀模型，可灵活切换或使用 Auto 模式调度。
- 生图生视频模型：Agent Plan 提供生图生视频模型，包含字节 SOTA 水平的 Seedance、Seedream，支持通过 Skill 接入并使用。
- 联网搜索：Agent Plan 提供联网搜索专属 Harness，支持通过 Skill 或 MCP 方式接入。
- 多工具兼容：Agent Plan 兼容多生态，无缝接入 OpenClaw、Hermes Agent、Claude Code 等主流编程及 AI Agent 工具。

#### 成本预估

完成该项目基础开发预估使用 14,900+ AFP，大约是 3% Max 套餐月额度。实际消耗以具体使用情况为准，可能会存在差异。

## 前置准备

### 一、购买 Agent Plan

进入 [Agent Plan 控制台](#) 购买 Agent Plan 套餐。套餐详情参见[套餐概览](#)。

**注意**

推荐购买 Medium 及以上套餐，Small 套餐不支持生视频模型。

### 二、配置工具

完成套餐购买后，需要获取 Agent Plan 支持的专属 Base URL、专属 API Key 及模型，完成工具配置后才能正常使用并抵扣套餐额度。

#### 需要配置的信息

- 专属 Base URL：
    - 兼容 Anthropic 接口协议工具：`https://ark.cn-beijing.volces.com/api/plan`
    - 兼容 OpenAI 接口协议工具：`https://ark.cn-beijing.volces.com/api/plan/v3`
  - 专属 API Key：[获取专属 API Key](#)，其他方舟 API Key 无法在 Agent Plan 中使用。
- 开通管理



- 支持模型：见[支持模型及Harness](#)。

### 接入 AI 工具

选择支持的 AI 工具并配置 Agent Plan，具体步骤参见[二、选择 AI 工具](#)。完成文本生成模型配置后，需要安装生图、生视频、联网搜索 Skill，具体如下：

- 安装生图生视频 Skill：[配置视觉模型](#)
- 安装联网搜索 Skill：在控制台配置 **Harness** 领取联网权益，具体步骤参见 [联网搜索（Beta版）](#)。

### 实践流程



## 一、明确项目需求

具体的项目需求可参考下面内容。

请帮我实现一个短视频网站项目：

- 1、功能调研：首先使用联网搜索skill调研主流短视频平台的Web端设计，总结关键功能。
- 2、页面设计：网页风格简洁时尚，可参考提供截图的页面布局设计。参考截图如下：



- 3、生成素材：请调用已安装的生图skill绘制网站logo，并使用生视频Skill，生成首页的3条视频素材，主题分别为自然风光、美食推荐、美妆护肤。

## 二、执行任务

启动工具后，提交上述短视频网站开发项目的需求，AI工具（以 Claude Code 为例）将自动完成以下关键步骤。

### 1.调用联网搜索 Skill

通过调用联网搜索 Skill 进行功能调研。

```

Claude Code v2.1.132
ark-code-latest · API Usage Billing
/Users/bytedance

请帮我实现一个短视频网站项目：
1、功能调研：首先使用联网搜索skill调研主流短视频平台的Web端设计，总结关键功能
2、页面设计：网页风格简洁时尚，可参考提供截图的页面布局设计。
3、生成素材：请调用已安装的生图skill绘制网站logo，并使用生视频Skill，生成首页的3条视频素材，主题分别为自然风光、美食推荐、美妆护肤 [Image #2]

Skill(byted-web-search)
 Successfully loaded skill

$ Bash(cd /Users/bytedance/.claude/skills/byted-web-search && python3 scripts/web_search.py "主流短视频平台Web端功能设计 抖音 快手 B站 关键功能总结"

+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
warnings.warn(
 ... +69 lines (ctrl+o to expand)

主流短视频平台Web端关键功能总结

基于搜索结果，主流短视频平台（抖音、快手、B站）的Web端核心功能可归纳为：

主流短视频平台Web端关键功能总结

基于搜索结果，主流短视频平台（抖音、快手、B站）的Web端核心功能可归纳为：

1. 内容浏览体系
- 布局模式：抖音采用单列全屏沉浸式上下滑，快手支持双列缩略图选择，B站采用分区瀑布流布局
- 导航系统：顶部部分类导航（全部、美食、旅行、美妆、知识等），侧边栏功能入口（推荐、关注、我的、直播等）
- 推荐算法：个性化内容推荐，支持搜索功能、热门榜单、话题挑战
- 播放控制：高清画质切换、倍速调节、音量控制、全屏播放、进度条拖动

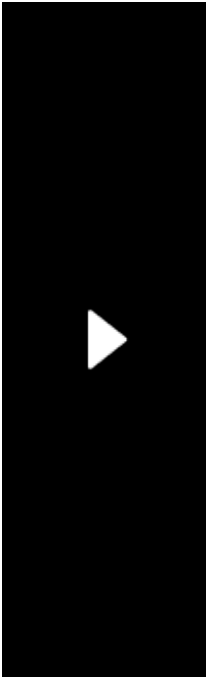
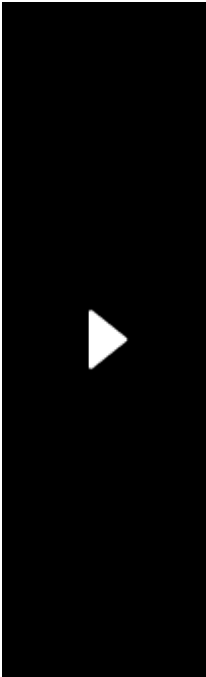
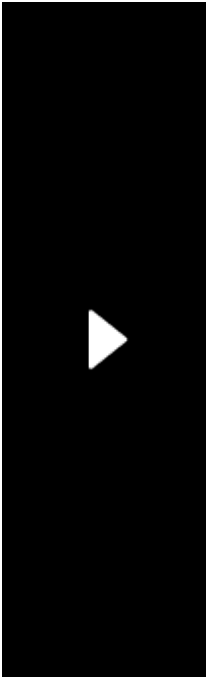
2. 互动功能模块
- 基础互动：点赞、评论、收藏、转发、分享到第三方平台
- 社交体系：关注/粉丝系统、私信功能、@用户功能、评论回复
- 特色互动：B站弹幕系统、抖音特效礼物打赏、快手老铁社区互动

3. 创作者服务

```

### 2.调用生图、生视频 Skill 生成素材

通过调用Seedream、Seedance Skill生成网站所需的多模态素材，例如网站Logo设计、短视频内容、视频封面素材等。生成素材示例如下：

使用模型	doubao-seedream-5.0-lite	doubao-seedance-2.0	doubao-seedance-2.0	doubao-seedance-2.0
生成素材				

```

✅ 网站 logo 已生成成功！保存路径： /Users/bytedance/Desktop/Seedream-Images/2026-05-07/seedream_1778158856_1.png

接下来生成首页 3 条视频素材：

• Skill(byted-ark-seedance-skill)
 ↳ Successfully loaded skill

• Bash(cd /Users/bytedance/.claude/skills/byted-ark-seedance-skill && node scripts/seedance-wrapper.js create --prompt
 "自然风光视频。 高清画质， 壮丽山川河流， 蓝天白云， 自然美景， 镜头缓慢移动， 节奏舒缓， 适合短视频平台")
 ↳ 🧠【智能意图识别】自动路由至模型： doubao-seedance-2.0
 {
 "id": "cgt-20260507210207-hkq2v"
 ... +12 lines (ctrl+o to expand)

✅ 自然风光视频生成任务已提交！任务 ID: cgt-20260507210207-hkq2v， 大约需要 3-5 分钟完成。

现在提交第二个美食推荐主题的视频：

• Bash(node scripts/seedance-wrapper.js create --prompt "美食推荐短视频， 诱人的美食特写， 色香味俱全的菜品展示， 镜头缓慢推拉， 暖色调， 让人有食欲， 适合短视频平台" --duration
 15 --ratio "16:9" --resolution "1080p" --api-k...)
 ↳ 🧠【智能意图识别】自动路由至模型： doubao-seedance-2.0
 {
 "id": "cgt-20260507210224-np814"
 ... +12 lines (ctrl+o to expand)

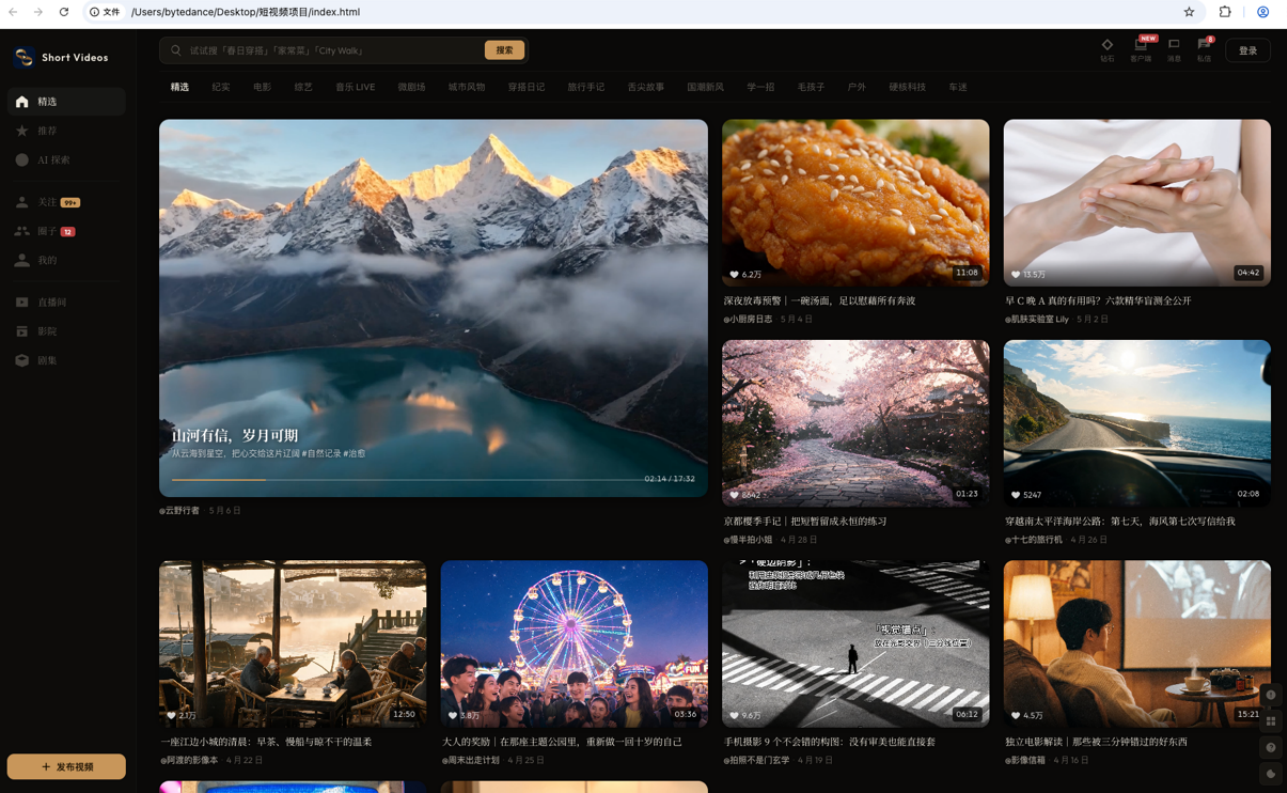
✅ 美食推荐视频生成任务已提交！任务 ID: cgt-20260507210224-np814

现在提交第三个美妆护肤主题的视频：
- Bash(node scripts/seedance-wrapper.js create --prompt "美妆护肤短视频， 精致的妆容展示， 护肤步骤演示， 时尚潮流， 适合短视频平台")
```

3. 网站生成与预览

通过 AI 工具自动完成网站基础开发工作， 后续可根据需求调整细节或部署上线。

您可以选用前端设计 Skill 进行页面美化设计。



三、查询用量消耗

进入 [Agent Plan 控制台](#) 查询 AFP 用量。实际用量消耗根据具体使用情况而定，详情参见[积分计费机制（AFP）](#)。

实现该项目预估消耗的 AFP 明细如下：

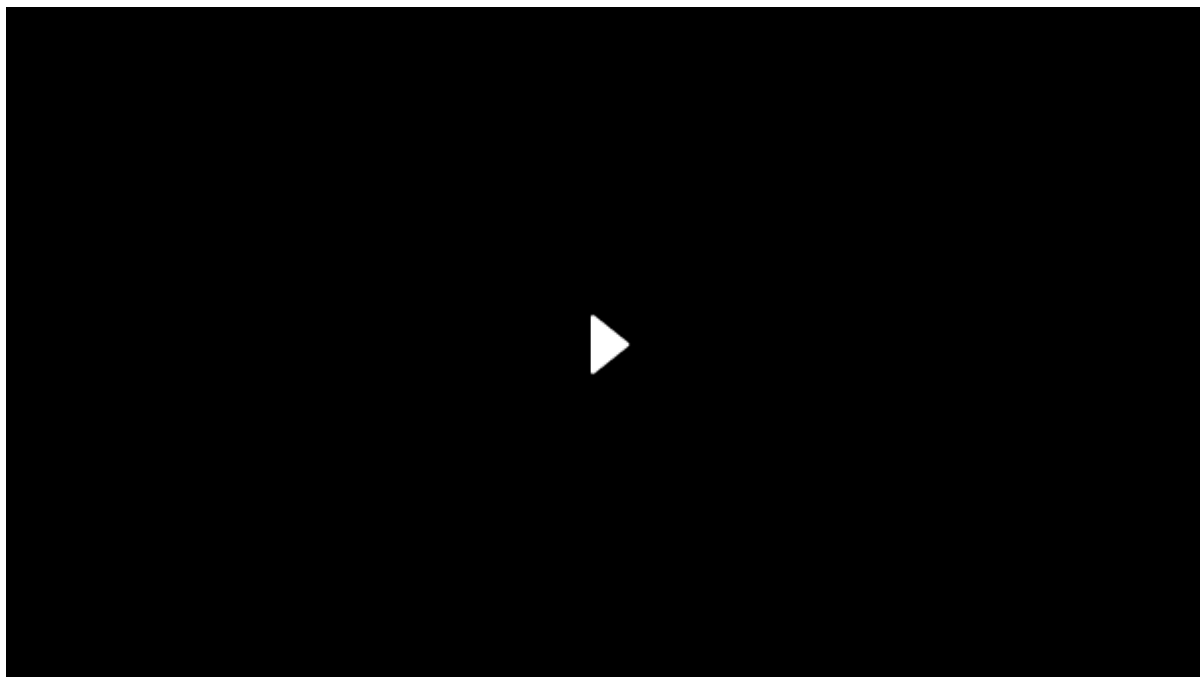
消耗项	消耗量	预估 AFP 消耗
语言模型	开发一个完整的项目、对话消耗。  不同层级模型抵扣系数不同。	1250 AFP
生图模型	7 张 seedream-5.0-lite 生成的图片素材。  一张图消耗 99 AFP。	693 AFP
生视频模型	3个 seedance-2.0 生成的 5秒视频素材。  约消耗 50w-60w token。 计算公式：消耗的原始 token / 10,000 * 抵扣系数	13000 AFP

## 7.2 使用 Agent Plan 开发学习教育网站

本文档以绘声绘色学古诗-教育网站为例，介绍如何通过 Agent Plan 一站式调用全模态模型及 Harness 完成产品落地。借助套餐内的多模态模型，快速产出带人声吟诵的诗境动画短片、古风意境画卷等优质素材，并通过优秀编程模型完成高质量网站代码交付。

### 项目概述

### 项目效果



### 核心使用能力

- 语言模型：Agent Plan 支持 Seed 2.0 系列、GLM 系列、Minimax 系列、DeepSeek 系列、Kimi 系列等多种优秀模型，可灵活切换或使用 Auto 模式调度。
- 生图生视频模型：Agent Plan 提供生图生视频模型，包含字节 SOTA 水平的 Seedance、Seedream，支持通过 Skill 接入并使用。
- 联网搜索：Agent Plan 提供联网搜索专属 Harness，支持通过 Skill 或 MCP 方式接入。
- 多工具兼容：Agent Plan 兼容多生态，无缝接入 OpenClaw、Hermes Agent、Claude Code 等主流编程及 AI Agent 工具。

### 成本预估

完成该项目基础开发预估使用 19,700+ AFP，大约是 4.6% Max 套餐月额度。实际消耗以具体使用情况为准，可能会存在差异。

### 前置准备

#### 一、购买 Agent Plan

进入 [Agent Plan 控制台](#) 购买 Agent Plan 套餐。套餐详情参见[套餐概览](#)。

**注意**

推荐购买 Medium 及以上套餐，Small 套餐不支持生视频模型。

二、配置工具

完成套餐购买后，需要获取 Agent Plan 支持的专属 Base URL、专属 API Key 及模型，完成工具配置后才能正常使用并抵扣套餐额度。

需要配置的信息

- 专属 Base URL：
    - 兼容 Anthropic 接口协议工具：`https://ark.cn-beijing.volces.com/api/plan`
    - 兼容 OpenAI 接口协议工具：`https://ark.cn-beijing.volces.com/api/plan/v3`
  - 专属 API Key：[获取专属 API Key](#)，其他方舟 API Key 无法在 Agent Plan 中使用。
- 开通管理



- 支持模型：见[支持模型及Harness](#)。

接入 AI 工具

选择支持的 AI 工具并配置 Agent Plan，具体步骤参见[二、选择 AI 工具](#)。完成文本生成模型配置后，需要安装生图、生视频、联网搜索 Skill，具体如下：

- 安装生图生视频 Skill：[配置视觉模型](#)
- 安装联网搜索 Skill：在控制台配置 **Harness** 领取联网权益，具体步骤参见 [联网搜索（Beta版）](#)。

实践流程

一、明确项目需求

具体的项目需求可参考下面内容。

Plain

帮我实现一个学古诗教育网站

通过沉浸式的诗境动画，让用户"看见诗、听见诗、读懂诗"

1、联网搜索3首代表性中小学古诗词，作为精选诗词库

2、网页设计：精致的典雅古风，网站图片素材可使用生图skill进行生成，诗文用衬线体模拟书法韵味

3、每首诗歌的学习模块包括：

- 诗歌原文、诗词解析

- 诗境动画短片：使用生视频skill生成带有人声朗诵诗句的意境短片，帮助用户学习

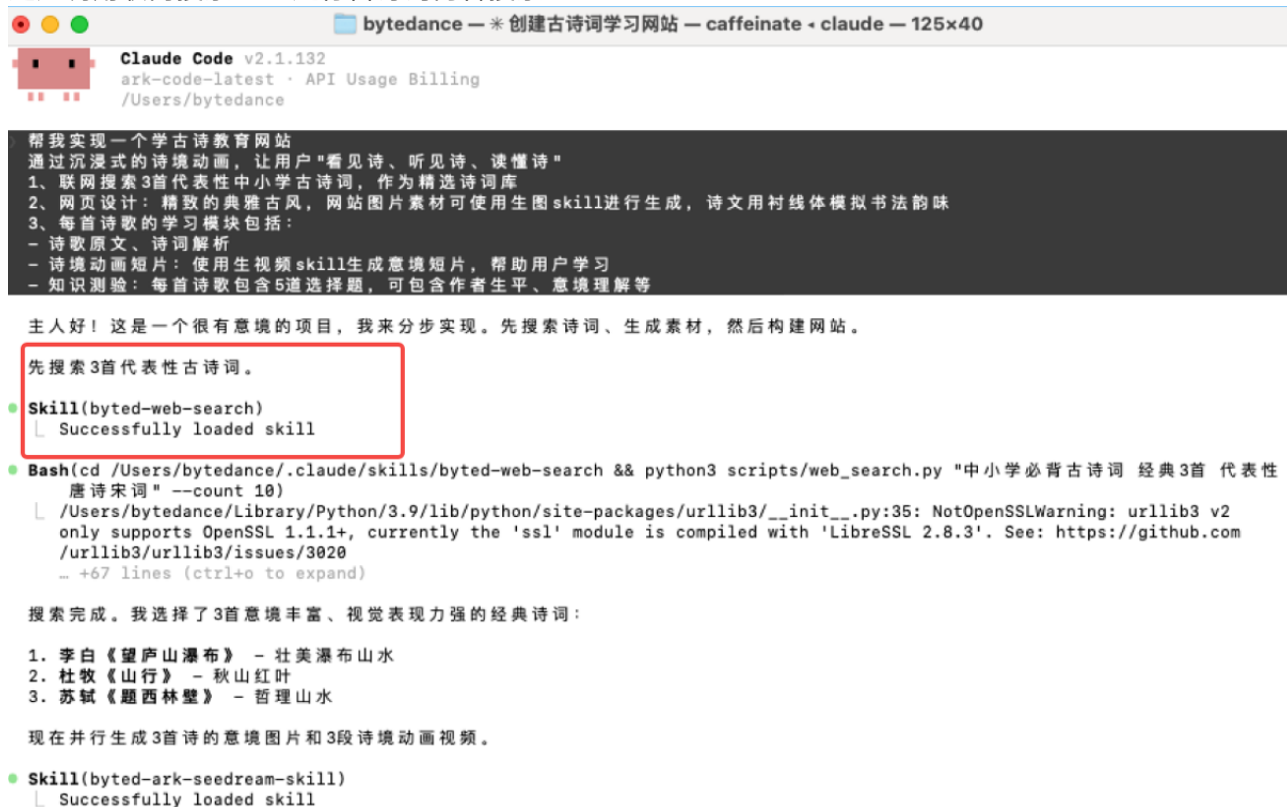
- 知识测验：每首诗歌包含5道选择题，可包含作者生平、意境理解等

## 二、执行任务

启动工具后，提交上述古诗学习教育网站开发项目的需求，AI 工具（以 Claude Code 为例）将自动完成以下关键步骤。

### 1.调用联网搜索 Skill

通过调用联网搜索 Skill 进行古诗词内容搜索。



```
bytedance — * 创建古诗词学习网站 — caffeinate · claude — 125x40

Claude Code v2.1.132
ark-code-latest · API Usage Billing
/Users/bytedance

帮我实现一个学古诗教育网站
通过沉浸式的诗境动画，让用户"看见诗、听见诗、读懂诗"
1、联网搜索3首代表性中小学古诗词，作为精选诗词库
2、网页设计：精致的典雅古风，网站图片素材可使用生图skill进行生成，诗文用衬线体模拟书法韵味
3、每首诗歌的学习模块包括：
- 诗歌原文、诗词解析
- 诗境动画短片：使用生视频skill生成意境短片，帮助用户学习
- 知识测验：每首诗歌包含5道选择题，可包含作者生平、意境理解等

主人好！这是一个很有意境的项目，我来分步实现。先搜索诗词、生成素材，然后构建网站。

先搜索3首代表性古诗词。
Skill(byted-web-search)
 Successfully loaded skill
Bash(cd /Users/bytedance/.claude/skills/byted-web-search && python3 scripts/web_search.py "中小学必背古诗词 经典3首 代表性唐诗宋词" --count 10)
 /Users/bytedance/Library/Python/3.9/lib/python/site-packages/urllib3/__init__.py:35: NotOpenSSLWarning: urllib3 v2 only supports OpenSSL 1.1.1+, currently the 'ssl' module is compiled with 'LibreSSL 2.8.3'. See: https://github.com/urllib3/urllib3/issues/3020
 ... +67 lines (ctrl+o to expand)

搜索完成。我选择了3首意境丰富、视觉表现力强的经典诗词：

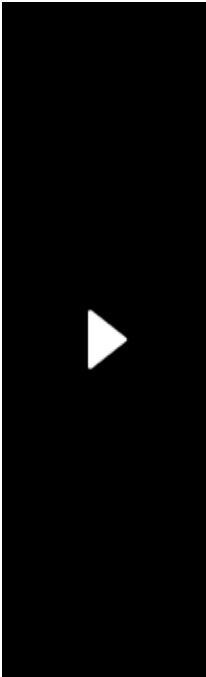
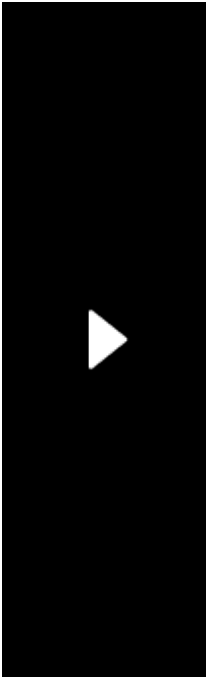
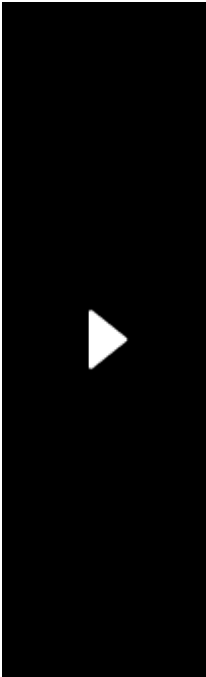
1. 李白《望庐山瀑布》 - 壮美瀑布山水
2. 杜牧《山行》 - 秋山红叶
3. 苏轼《题西林壁》 - 哲理山水

现在并行生成3首诗的意境图片和3段诗境动画视频。
Skill(byted-ark-seedream-skill)
 Successfully loaded skill
```

### 2.调用生图、生视频 Skill 生成素材

通过调用Seedream、Seedance Skill生成网站所需的多模态素材，例如带有人声吟诵的动画短片、古风意境画卷等。



使用模型	doubao-seedream-5.0-lite	doubao-seedance-2.0	doubao-seedance-2.0	doubao-seedance-2.0
生成素材				

调用生视频 Skill 过程如下：

bytedance — \* 创建古诗词学习网站 — claude — 105x24

✔ 5秒版本三首诗的诗境动画已全部提交！

新任务列表（5秒版）：

序号	诗词	任务 ID	时长
1	《望庐山瀑布》李白	cgt-20260508003437-9xmxw	5秒
2	《山行》杜牧	cgt-20260508003445-xk7kl	5秒
3	《题西林壁》苏轼	cgt-20260508003611-5d69c	5秒

⌚ 生成中，预计 2-3 分钟/首。完成后会自动下载并替换到网站 videos 目录！

\* Brewed for 2m 13s

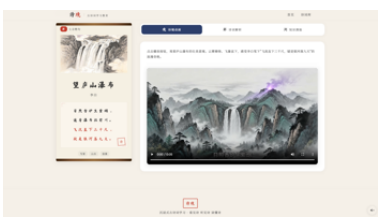
> `check the video generation progress`

? for shortcuts

3.网站生成与预览

通过 AI 工具自动完成网站基础开发工作，后续可根据需求调整细节或部署上线。

您可以选用前端设计 Skill 进行页面美化设计。

首页	诗词库	诗歌学习页
		

三、查询用量消耗

进入 [Agent Plan 控制台](#) 查询 AFP 用量。实际用量消耗根据具体使用情况而定，详情参见[积分计费机制 \(AFP\)](#)。

实现该项目预估消耗的 AFP 明细如下：

消耗项	消耗量	预估 AFP 消耗
语言模型	开发一个完整的项目、对话消耗。 不同层级模型抵扣系数不同。	3500 AFP
生图模型	3 张 seedream-5.0-lite 生成的图片素材。 一张图消耗 99 AFP。	297 AFP
生视频模型	4个 seedance-2.0 生成的 5秒视频素材。 约消耗 60w-70w token。 计算公式：消耗的原始 token / 10,000 * 抵扣系数	16000 AFP



## 8. 常见问题

### 关于 Agent Plan

#### Agent Plan 和 Coding Plan 有什么区别？

Agent Plan 是在 Coding Plan 基础升级推出的全新订阅包。具体区别如下：

对比	Agent Plan	Coding Plan
核心定位	主打 Agent 场景，向多模态及 Harness 发展的全新订阅式大模型服务套餐包。	主打 Coding 场景，面向个人开发者推出的订阅式 Coding 大模型服务套餐包。
模型范围	语言模型、向量化模型、视觉模型（生图模型、生视频模型）、语音模型	语言模型、向量化模型
Harness	豆包搜索、AI Native 应用开发底座、Agent记忆	/
Base URL 与 API Key	在 <a href="#">Agent Plan 控制台</a> 获取专属 Base URL 和专属 API Key。	在 <a href="#">Coding Plan 控制台</a> 获取专属 Base URL 和 API Key（方舟 API Key）。
套餐信息	4 档套餐，灵活可选择	2 档套餐
计费方式	按照 Agent 燃料值（AFP）消费抵扣，额度用量清晰可查。	预估模型调用次数

## 9. 活动及公告

### 9.1 Agent/Coding Plan 指定模型抵扣系数限时折扣活动

2026年6月10日18:00:00至2026年7月15日00:00:00活动期间，Agent/Coding Plan 个人版用户使用 deepseek-v4-pro、kimi-k2.6、kimi-k2.7-code、glm-5.2 模型时，模型抵扣系数在现有抵扣系数基础上享最低 2.5 折优惠。活动到点自动生效，无需用户申请。

#### 活动时间

2026年6月10日18:00:00至2026年7月15日00:00:00

#### 活动对象

所有 Agent/Coding Plan 个人版用户，活动到点自动生效，无需申请。

#### 适用产品与模型

适用产品	适用模型	优惠规则
Agent/Coding Plan 个人版	deepseek-v4-pro	活动期间，模型抵扣系数在现有抵扣系数基础上享 4 折优惠，可使用配额为原用量的 250%。
	kimi-k2.6	
	kimi-k2.7-code	新模型上线，模型抵扣系数在现有抵扣系数基础上享 2.5 折优惠，可使用配额为原用量的 400%。
	glm-5.2 (glm-latest)	
	注意  glm-5.2 是智谱最新旗舰模型，支持 1M 上下文窗口，长程任务效果表现突出，多项权威评测榜单成绩稳居前列。	

#### 参与方式

活动到点自动生效。

- 您可以访问[方舟 Agent Plan 活动](#) 了解活动详情并订阅 Agent Plan 套餐。
- 您可以访问[方舟 Coding Plan 活动](#) 了解活动详情并订阅 Coding Plan 套餐。

### 特别说明

- 本活动仅调整适用模型在活动期间的抵扣系数，不改变套餐内其他模型、Harness、套餐额度刷新规则与使用限制。
- 活动结束后，适用模型的抵扣系数恢复为原有规则。
- 火山引擎可以根据运营情况对活动进行变更或调整，包括不限于活动规则、活动有效期等，请以页面最新内容及火山引擎通知为准。

## 9.2 Agent Plan Small & Medium 套餐 2.5 折普惠活动

2026年6月8日至2026年8月8日活动期间内，所有用户购买 Agent Plan Small、Medium 套餐，最多可享首两个月 2.5 折优惠，第三个月起恢复原价。**名额有限，先到先得**；优惠资格在新购、续费、升级场景间共享，使用后不因升档或后续退订等操作重新获得。

### 活动时间

2026年6月8日至2026年8月8日

### 活动对象

所有用户，**名额有限，先到先得**

### 适用产品与档位

注意

- 活动期间内，每个账号最多能享受两个月的特惠套餐，新购、续费、升配共享同一优惠资格。
- 若用户首次仅订阅了一个月的特惠套餐，第二个月的特惠套餐相关操作（包含新购、续费、升配）需在首月套餐购买成功次日起才可进行。

档位	原价	折扣价	折扣享受条件
Small	40元	9.9元	2026年6月8日至2026年8月8日，首两个月套餐可享。       · 新用户：    ◦ 如果当前没有订阅套餐，购买前两个月套餐 2.5 折，第三个月起原价。        · 老用户：    ◦ 若当前已有订阅 Small 套餐，续费或升配至 Medium 套餐，前两个月套餐 2.5 折（不满 1 个月按 1 个月计算），第三个月起原价。    ◦ 若当前已有订阅 Medium 套餐，续费前两个

档位	原价	折扣价	折扣享受条件
Medium	200元	49.9元	月套餐 2.5折， 第三个月起原 价。

### 优惠资格规则

- 活动优惠资格按用户维度发放，仅能在同一账号内享受。
- 新购、续费、升配共享同一优惠资格。
- 用户使用优惠资格后，后续操作 **不会**因续费、升配或换档重新获得优惠。
- 名额有限，先到先得。

### 特别说明

- 如多个账户通过同一手机号、同一证件号、同一账号 ID，或其他经火山引擎排查多个账户为同一实际控制人的，均视为“同一用户”。
- 对恶意抢占活动资源、批量刷单及其他任何采用非常规手段影响活动公平性和违反服务协议及规则的行为，火山引擎有权取消从事违法违规行为的用户活动资格并回收奖励包资源，必要时追究违规用户的法律责任。
- 火山引擎可以根据运营情况对活动进行变更或调整，包括但不限于活动规则、商品数量、活动有效期等，请以页面最新内容及火山引擎通知为准。用户在活动规则变更或调整前已发放的服务和权益不受影响。

## 9.3 Agent Plan Medium 套餐 seedance 2.0 系列模型调整公告

### 调整内容

为优化 Agent Plan 套餐能力配置，自2026年6月8日起，Agent Plan Medium 套餐将不再支持 seedance 2.0 系列模型调用，包含 seedance 2.0 与 seedance 2.0 fast 模型。本次调整仅涉及 Agent plan Medium 套餐中的 seedance 2.0 系列模型能力，不影响其他已支持模型及其他档位套餐现有权益。

### 影响范围

2026年6月8日00:00:00 前已订阅并生效的 Agent Plan Medium 套餐用户。

#### 说明

2026年6月8日00:00:00 后新购、续费、升配的 Medium 套餐，不包含在本次影响范围内。时间以套餐生效时间为准。

### 处理方案

如你受本次调整实际影响，可于**2026年6月8日至2026年6月15日**前往[火山方舟控制台](#)，将符合条件的 Medium 套餐免费升级 Large 套餐，逾期不再受理。

#### 注意

本次补偿仅有1次免费升配机会，若因错误操作导致资格失效，资格不再恢复。具体资格以控制台页面展示及系统校验结果为准。平台将结合套餐生效情况及相关使用情况进行资格校验。

订阅情况	是否符合免费升级条件
2026年6月8日00:00:00 前已订阅并生效的 Medium 套餐周期	适用
2026年6月8日00:00:00 后新购的 Medium 套餐	不适用
2026年6月8日00:00:00 后续费形成的新套餐周期	不适用
2026年6月8日00:00:00 后升配的 Medium 套餐	不适用

## 9.4 功能发布公告

### 202607

发布日期	功能说明	帮助文档
07.01	* Small、Medium 套餐支持 AI Native 应用开发底座。	<a href="#">套餐概览</a>

### 202606

发布日期	功能说明	帮助文档
06.30	* 模型下线：glm-5.1，deepseek-v3.2 模型下线。	<a href="#">支持的模型</a>
06.25	* 模型新增：支持 doubao-seedance-2.0-mini 视频生成模型，仅 Large、Max 套餐支持。	<a href="#">支持的模型</a>
06.21	· 联网搜索（Beta）版升级为豆包搜索。  · 新增支持 AI Native 应用开发底座、Agent 记忆。	<a href="#">套餐概览</a>
06.17	* 模型新增：支持 glm-5.2、kimi-k2.7-code 模型。	<a href="#">支持的模型</a>
06.11	· 模型新增：语音模型 doubao-seed-tts-2.0、doubao-seed-asr-2.0。  · 新增支持专业数据集，支持开启超额后付费。	<a href="#">套餐概览</a>
06.08	· 模型新增：支持 minimax-m3 模型。  · 模型下线：minimax-m2.5、kimi-k2.5、glm-4.7 模型下线。	<a href="#">支持的模型</a>

### 202605

发布日期	功能说明	帮助文档
05.15	* 模型新增：支持 <code>deepseek-v4-flash</code> 、 <code>deepseek-v4-pro</code> 模型。	<a href="#">支持的模型</a>
05.07	* 方舟 Agent Plan 正式上线，支持全模态模型及 Harness，采用精细化积分计费模式。	<a href="#">套餐概览</a>